



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

GRADO EN INGENIERÍA INFORMÁTICA

Curso Académico 2024/2025

Trabajo Fin de Grado

**Desarrollo de una aplicación para el seguimiento de contenidos multimedia en
Scala mediante programación funcional.**

Autor: Diego Carrera Hernando

Director: Juan Manuel Serrano Hidalgo

RESUMEN

Este Trabajo de Fin de Grado aborda el desarrollo del Back-End de una aplicación web orientada a la gestión y valoración de distintos tipos de contenidos multimedia, como películas, series, libros y videojuegos. Inspirada en plataformas como Letterboxd o Goodreads, pero buscando un enfoque más unificado, la aplicación permite a los usuarios realizar acciones como registrar contenidos vistos o leídos, escribir reseñas, asignar puntuaciones, crear listas personalizadas e interactuar con otros usuarios.

El proyecto se ha desarrollado utilizando el lenguaje de programación Scala, haciendo uso de la librería Tapir para la definición, implementación y exposición de los distintos endpoints que conforman el sistema. Además, dado que la aplicación no cuenta aún con una capa de persistencia, la obtención de la información sobre los contenidos se ha realizado mediante la integración de APIs (Interfaces de Programación de Aplicaciones) externas como TMDB (The Movie Database), IGDB (Internet Game Database) y Google Books.

Durante el desarrollo, se han abordado aspectos como el análisis del dominio, la definición de la arquitectura del sistema y la instrumentación de endpoints tanto para funcionalidades propias como para la comunicación con servicios externos. El trabajo ha supuesto un proceso continuo de aprendizaje, especialmente en lo relativo a la programación funcional en Scala, el uso de Tapir y la gestión de codificadores y decodificadores de datos.

Finalmente, se identifican diversas líneas de trabajo futuro, como la incorporación de una base de datos para la persistencia de datos, el desarrollo de una interfaz Front-End, la generación automática de documentación OpenAPI y la implementación de un sistema de autenticación de usuarios.

PALABRAS CLAVE

Aplicación web, Scala, programación funcional, Tapir, endpoints, APIs externas, solicitudes HTTP, contenidos multimedia, gestión de usuarios y reseñas.

ABSTRACT

This Final Degree Project focuses on the development of the Back-End for a web application aimed at managing and reviewing different types of multimedia content, such as movies, TV series, books, and video games. Inspired by platforms like Letterboxd or Goodreads, but with a more unified approach, the application allows users to perform actions such as recording consumed content, writing reviews, assigning ratings, creating personalized lists, and interacting with other users.

The project has been developed using the Scala programming language, leveraging the Tapir library for defining, implementing, and exposing the various endpoints that form the system. Additionally, since the application does not yet include a persistence layer, information about multimedia content has been obtained by integrating external APIs (Application Programming Interface) such as TMDb (The Movie Database), IGDB (Internet Game Database), and Google Books.

Throughout the development process, several aspects have been addressed, including domain analysis, system architecture definition, and the instrumentation of endpoints both for internal functionalities and external service communication. The project has involved a continuous learning process, especially regarding functional programming in Scala, the use of Tapir, and the management of data encoders and decoders.

Finally, several lines of future work are identified, such as the incorporation of a database for data persistence, the development of a user-friendly Front-End interface, the automatic generation of OpenAPI documentation, and the implementation of a user authentication system.

KEYWORDS

Web application, Scala, functional programming, Tapir, endpoints, third-party APIs, HTTP requests, multimedia content, users and reviews management.

ÍNDICE DE CONTENIDOS

Resumen	2
Palabras clave	2
Abstract	3
Keywords	3
Índice de contenidos.....	4
Índice de figuras	7
1. Introducción	14
2. Objetivos.....	15
2.1. Tecnologías, herramientas y metodología.....	16
2.1.1 Tecnologías	16
2.1.2. Herramientas	16
2.1.3. Metodología	17
3. Descripción informática.....	18
3.1. Análisis y arquitectura del sistema.....	18
3.1.1. Funcionalidades del sistema.....	18
3.1.2. Modelo del dominio	21
3.1.3. Análisis funcional.....	27
3.1.4. Arquitectura del sistema.....	30
3.2. Instrumentación de endpoints del dominio propio	33
3.2.1. Definición de un endpoint	35

3.2.1.1.	Conceptos básicos de un endpoint.....	35
3.2.1.2.	Endpoints base.....	37
3.2.1.3.	Parámetros de E/S y códecs.....	38
3.2.2.	Implementación de la lógica de un endpoint.....	43
3.2.3.	Exposición de un endpoint	44
3.3.	Instrumentación de endpoints que se comunican con servicios externos	48
3.3.1.	Definición de un endpoint	48
3.3.2.	Creación de un cliente	50
3.3.3.	Utilización del endpoint	53
4.	Experimentos y validación.....	55
4.1.	Endpoint createUser	55
4.2.	Endpoint getAllEntries	60
4.3.	Endpoint searchBook	62
4.4.	Endpoint deleteBook	64
5.	Conclusiones	66
5.1.	Líneas futuras	68
	Bibliografía.....	70
	Apéndices	73
1.	Estado del Arte	73
1.1.	Letterboxd	73
1.2.	Goodreads.....	74

1.3.	Backlogg	75
1.4.	Serializd	76
1.5.	Filmaffinity	77
1.6.	Simkl	79
1.7.	HowLongToBeat	80
1.8.	Metacritic	81
1.9.	Conclusión	82
1.10.	Sitios web de las aplicaciones analizadas	83
2.	Sitios web oficiales de las tecnologías y herramientas utilizadas	84
3.	Otros casos de uso	85
4.	Instrumentación de otros endpoints	92
4.1.	Endpoint de tipo POST	92
4.2.	Endpoint de tipo PUT	95
4.3.	Endpoint de tipo DELETE	97
5.	Experimentos adicionales	98
5.1.	Endpoints relacionados con contenidos de TMDB	98
5.2.	Endpoints relacionados con contenidos de IGDB	115
5.3.	Endpoints relacionados con contenidos de Google Books	124
5.4.	Otros endpoints de interés	127

ÍNDICE DE FIGURAS

Figura 1. Diagrama de clases con todas las entidades.....	22
Figura 2. Diagrama de clases con entidades del conjunto de clases de usuarios.	23
Figura 3. Diagrama de clases para relaciones entre <i>Usuario</i> y conjunto de clases de conversaciones.	23
Figura 4. Diagrama de clases para relaciones entre <i>Usuario</i> y entidades del conjunto de clases social.....	24
Figura 5. Diagrama de clases para relaciones entre entidades del conjunto de clases social...	25
Figura 6. Diagrama de clases para relaciones entre <i>Usuario</i> y clases del conjunto de contenidos multimedia.....	26
Figura 7. Diagrama de clases para relaciones entre clases del conjunto de contenidos y entidades del conjunto social.....	27
Figura 8. Diagrama de casos de uso de la página principal.	28
Figura 9. Diagrama de casos de uso de la herramienta de búsqueda.	29
Figura 10. Diagrama de casos de uso relacionados con los contenidos multimedia.....	30
Figura 11. Definición del tipo <i>PublicEndpoint</i>	36
Figura 12. Definición del endpoint <i>getAllReviews</i>	36
Figura 13. Definición de la interfaz <i>UserError</i> y de las clases que la extienden.....	36
Figura 14. Clase <i>Review</i>	36
Figura 15. Definición del endpoint base <i>reviewsBaseEndpoint</i>	37
Figura 16. Definición del endpoint base <i>httpMethodEndpoint</i>	37
Figura 17. Definición del endpoint base <i>getBaseEndpoint</i>	38
Figura 18. Definición del endpoint base <i>baseEndpoint</i>	38

Figura 19. Definición del <i>EndpointInput querySortBy</i>	39
Figura 20. Definición del <i>EndpointInput queryCategories</i>	39
Figura 21. Definición del códec <i>categoriesCodec</i>	39
Figura 22. Definición del <i>EndpointOutput listOfReviewsOutput</i>	40
Figura 23. Definición del <i>Encoder listMediaUnionEncoder2</i>	40
Figura 24. Fragmento de un JSON que muestra una instancia de <i>MovieId</i> convertida a dicho formato.	41
Figura 25. Fragmento de un JSON que muestra una instancia de la tupla (<i>TvShowId</i> , <i>TvSeasonNumber</i> , <i>TvEpisodeNumber</i>) convertida a dicho formato.	41
Figura 26. Definición del <i>Decoder listMediaUnionDecoder2</i>	42
Figura 27. Definición del <i>Schema listMediaUnionSchema2</i>	42
Figura 28. Implementación de la lógica del endpoint <i>getAllReviews</i>	43
Figura 29. Definición de la ruta que asocia el endpoint <i>getAllReviews</i> con la implementación de su lógica.	44
Figura 30. Variable que recoge las rutas de todos los endpoints relacionados con las reseñas.	46
Figura 31. Variable que recoge las rutas de todos los endpoints de la aplicación.	46
Figura 32. Objeto <i>Main</i> de la aplicación.	46
Figura 33. Definición del endpoint <i>requestMovie</i>	48
Figura 34. Definición del endpoint base <i>movieBaseEndpoint</i>	48
Figura 35. Definición del endpoint base <i>tmdbBaseEndpoint</i>	49
Figura 36. Clase <i>MovieFromTMDB</i>	49

Figura 37. Implementación de cliente (<i>TMDBClient</i>) para realizar peticiones a la API de TMDB.	50
Figura 38. Definición de la configuración básica del cliente.	51
Figura 39. Comprobación de coincidencia en la estructura entre los argumentos recibidos por el cliente.	52
Figura 40. Función que genera la solicitud HTTP a partir del endpoint.	52
Figura 41. Lanzamiento de la petición HTTP desde el cliente.	53
Figura 42. Implementación de la lógica del endpoint <i>getMovie</i> , realizando varias solicitudes a la API de TMDB.	54
Figura 43. Cuerpo de petición introducido al endpoint <i>createUser</i>	56
Figura 44. Llamada al endpoint <i>createUser</i> con resultado exitoso.	56
Figura 45. Objeto JSON devuelto por el endpoint <i>createUser</i> con los datos del nuevo usuario.	57
Figura 46. Resultado de ejecutar el endpoint <i>createUser</i> introduciendo un identificador de usuario que ya existe dentro de la aplicación.	58
Figura 47. Resultado de ejecutar el endpoint <i>createUser</i> introduciendo un identificador de usuario no válido.	58
Figura 48. Cuerpo de petición, con un nombre de usuario existente, introducido al endpoint <i>createUser</i>	58
Figura 49. Resultado de ejecutar el endpoint <i>createUser</i> introduciendo un nombre de usuario que ya existe dentro de la aplicación.	58
Figura 50. Cuerpo de petición, con un email existente, introducido al endpoint <i>createUser</i> . .	58
Figura 51. Resultado de ejecutar el endpoint <i>createUser</i> introduciendo un email que ya existe dentro de la aplicación.	59
Figura 52. Usuario 2 de los datos de prueba.	59

Figura 53. Usuario 4 de los datos de prueba.	60
Figura 54. Petición exitosa al endpoint <i>getAllEntries</i> con parámetros <i>sort_by</i> y <i>categories</i> ...	60
Figura 55. Respuesta del endpoint <i>getAllEntries</i> mostrando solo objetos <i>Entry</i> que hacen referencia a películas y libros, ordenados de más recientes a menos.	62
Figura 56. Petición al endpoint <i>searchBook</i>	63
Figura 57. Objeto JSON devuelto por la aplicación al llamar al endpoint <i>searchBook</i> para buscar el libro <i>The Old Man and The Sea</i>	64
Figura 58. Petición al endpoint <i>deleteBook</i> para un libro que ya ha sido marcado previamente con ese estado de progreso.	65
Figura 59. Petición al endpoint <i>deleteBook</i> para un libro que no ha sido marcado previamente con ese estado de progreso.	65
Figura 60. Diagrama de casos de uso relacionados con las puntuaciones.	85
Figura 61. Diagrama de casos de uso relacionados con las reseñas.	86
Figura 62. Diagrama de casos de uso relacionados con los registros.	87
Figura 63. Diagramas de casos de uso relacionados con las listas de contenidos.....	88
Figura 64. Diagrama de casos de uso relacionados con los <i>me gusta</i>	89
Figura 65. Diagrama de casos de uso relacionados con los comentarios.	89
Figura 66. Diagramas de casos de uso relacionados con los usuarios.	90
Figura 67. Definición del endpoint <i>createReview</i>	92
Figura 68. Definición del endpoint base <i>reviewBaseEndpoint</i>	92
Figura 69. Definición del endpoint base <i>postEndpoint</i>	93
Figura 70. Definición del endpoint base <i>postBaseEndpoint</i>	93
Figura 71. Definición del <i>EndpointInput jsonReview</i>	94

Figura 72. Definición del <i>EndpointOutput reviewOutput</i> .	94
Figura 73. Implementación de la lógica del endpoint <i>createReview</i> .	94
Figura 74. Definición del endpoint <i>editReview</i> .	95
Figura 75. Definición del endpoint base <i>putBaseEndpoint</i> .	95
Figura 76. Definición del <i>EndpointInput pathReviewId</i> .	96
Figura 77. Clase <i>ReviewId</i> .	96
Figura 78. Definición del códec <i>reviewIdCodec</i> .	96
Figura 79. Implementación de la lógica del endpoint <i>editReview</i> .	96
Figura 80. Definición del endpoint <i>deleteReview</i> .	97
Figura 81. Definición del endpoint base <i>deleteBaseEndpoint</i> .	97
Figura 82. Implementación de la lógica del endpoint <i>deleteReview</i> .	98
Figura 83. Llamada al endpoint <i>searchTvShow</i> .	99
Figura 84. Objeto JSON devuelto por la aplicación al llamar al endpoint <i>searchTvShow</i> para buscar la serie <i>The Bear</i> .	101
Figura 85. Ejecución del endpoint <i>getTvShow</i> .	101
Figura 86. DTO utilizado para recoger información de una serie de TMDB.	102
Figura 87. Objeto JSON devuelto por la aplicación al llamar al endpoint <i>getTvShow</i> para obtener información de la serie <i>The Bear</i> .	108
Figura 88. Petición al endpoint <i>getEntriesForTvShow</i> .	108
Figura 89. Objeto JSON obtenido como resultado de la llamada al endpoint <i>getEntriesForTvShow</i> para la serie <i>The Bear</i> .	110
Figura 90. Resultado de ejecutar el endpoint <i>getMediaListsForTvShow</i> sin que exista ninguna lista que albergue ese contenido.	110

Figura 91. Ejecución del endpoint <i>getTvSeason</i>	110
Figura 92. Objeto JSON devuelto por la aplicación al llamar al endpoint <i>getTvSeason</i> para obtener información de una temporada de la serie <i>The Bear</i>	113
Figura 93. Ejecución del endpoint <i>getTvEpisode</i>	113
Figura 94. Objeto JSON devuelto por la aplicación al llamar al endpoint <i>getTvEpisode</i> para obtener información de una temporada de la serie <i>The Bear</i>	115
Figura 95. Ejecución del endpoint <i>searchVideogame</i>	115
Figura 96. Objeto JSON devuelto por la aplicación al llamar al endpoint <i>searchVideogame</i> para buscar el videojuego <i>Sifu</i>	119
Figura 97. Ejecución del endpoint <i>getVideogame</i>	119
Figura 98. Objeto JSON devuelto por la aplicación al llamar al endpoint <i>getVideogame</i> para obtener información del videojuego <i>Sifu</i>	123
Figura 99. Cuerpo de petición (JSON) para el endpoint <i>editMediaList</i> , remarcando aquello que realmente se está editando en la lista.	124
Figura 100. Llamada al endpoint <i>editMediaList</i>	124
Figura 101. Resultado de ejecutar el endpoint <i>editMediaList</i> introduciendo un identificador en el cuerpo de la petición diferente al especificado en la ruta del endpoint.	124
Figura 102. Solicitud al endpoint <i>getBook</i>	124
Figura 103. Objeto JSON devuelto por la aplicación al llamar al endpoint <i>getBook</i> para obtener información del libro <i>The Old Man and The Sea</i>	125
Figura 104. Solicitud al endpoint <i>addBook</i> para el estado de progreso <i>Completado</i>	126
Figura 105. Objeto JSON devuelto por el endpoint <i>addBook</i> , remarcando aquello que realmente se ha añadido a la categoría <i>Completado</i>	127
Figura 106. Petición al endpoint <i>addBook</i> para un libro que ya ha sido marcado previamente con ese estado de progreso.	127

Figura 107. Petición al endpoint <i>getUser</i>	127
Figura 108. Objeto JSON devuelto por la aplicación al llamar al endpoint <i>getUser</i>	130
Figura 109. Petición al endpoint <i>getUserReplies</i>	131
Figura 110. Objeto JSON devuelto por la aplicación al llamar al endpoint <i>getUserReplies</i> . ..	131
Figura 111. Petición al endpoint <i>getAllMedia</i> , especificando que se quiere obtener los contenidos marcados como completados.	132
Figura 112. Objeto JSON devuelto por la aplicación al llamar al endpoint <i>getAllMedia</i>	132

1. INTRODUCCIÓN

En los últimos años, debido al auge del consumo de contenido a través de plataformas digitales, así como al deseo de las personas de compartir sus opiniones acerca de los contenidos que disfrutan, se han popularizado las aplicaciones o páginas web que permiten a sus usuarios generar interacciones en torno a ellos. Tales son ejemplos como Letterboxd, red social que gira alrededor de las películas; Serializd, cuyo concepto es el mismo, pero orientado a series de televisión; Goodreads, cuya temática es, en este caso, los libros; o HowLongToBeat, que se enfoca en los videojuegos.

Sin embargo, ninguna de estas aplicaciones o páginas web engloba todas estas formas de ocio. Quizás, la que más se aproxime sea Metacritic, aunque su gran popularización entre aficionados a los videojuegos hace que estos sean su principal atractivo y que se enfoque más en este tipo de contenido. Un análisis más detallado de estas aplicaciones, así como de otras similares, puede encontrarse en el apéndice dedicado al **Estado del Arte**.

Motivado por esta carencia de una solución unificada, decidí plantear el desarrollo de una aplicación que ofreciera dicha funcionalidad. Es decir, permitir a sus usuarios calificar y reseñar películas, series, libros y videojuegos, así como crear rankings e interactuar con otros usuarios.

Este TFG, por tanto, aborda la creación de la aplicación web con la funcionalidad descrita, enfocándome en la parte de Back-End y, concretamente, en la definición, creación, implementación y ejecución de los *endpoints* que compondrán la aplicación, así como en la documentación de estos mismos.

La aplicación se creará usando el lenguaje de programación Scala, que permite un uso cómodo y sencillo de la programación funcional (Scala, s.f.), además de la biblioteca Tapir para definir y documentar los endpoints (SoftwareMill, 2024; SoftwareMill, s.f.; Varvarigos, 2024), sobre la cual se hará especial énfasis, explicando cómo se comporta y las funcionalidades que ofrece.

Por otra parte, como la aplicación no contará con persistencia para guardar películas, series, videojuegos y libros, estos elementos se obtendrán realizando consultas a diferentes APIs proveedoras de estos contenidos, por lo que la utilización de estos servicios de terceros será una parte importante del proyecto.

2. OBJETIVOS

El principal objetivo del Trabajo de Fin de Grado es, como ya se ha comentado en la **Introducción**, crear el Back-End de una aplicación web que integre funcionalidades similares a las de plataformas populares como Letterboxd o Goodreads. Esta aplicación permitirá a los usuarios registrar y gestionar sus experiencias y opiniones relacionadas con películas, series (y sus temporadas y episodios), videojuegos y libros (a partir de ahora nos referiremos a todos estos como “contenidos multimedia” o “contenidos”, por abreviar), así como interactuar con otros usuarios de diversas formas, pero siempre partiendo de estos contenidos multimedia como eje principal.

Por otra parte, la aplicación se va a desarrollar en el lenguaje de programación Scala, utilizando, para ello, una librería llamada Tapir, que permite definir e implementar endpoints con programación funcional. Cabe recalcar que lo que se desarrollará de la aplicación es solo la parte de Back-End, entendiendo con esto todo lo necesario para poder ejecutar dichos endpoints. Por tanto, en este proyecto no se contempla ni el desarrollo del Front-End, ni de un sistema de persistencia para los datos generados al poner en funcionamiento los distintos endpoints.

Si entendemos todo lo anterior como un objetivo global a alcanzar, para alcanzar este objetivo me he propuesto tres hitos a realizar:

- Análisis de la aplicación a desarrollar. Esto incluye la definición de las funcionalidades que se quieren implementar, la descripción de las entidades que modelarán el dominio de la aplicación, la descripción de los casos de uso que se pueden dar, y un análisis de la arquitectura del sistema.
- Instrumentación de endpoints del dominio propio de la aplicación. Incluye todos los endpoints necesarios para gestionar los distintos elementos del dominio que se maneja internamente.
- Instrumentación de los endpoints que se comunican con servicios externos. Al no implementarse un sistema de persistencia, será necesario obtener los distintos contenidos (y toda la información relevante sobre estos) de algunas APIs de terceros que, precisamente, proveen este tipo de información (y que se describirán más adelante). Es por ello por lo que también es necesario implementar endpoints con los que no interactúa el usuario final, pero que la aplicación ejecuta internamente para realizar

peticiones a estas APIs. La ejecución de estos endpoints tiene lugar al interactuar con los endpoints propios de la herramienta de búsqueda (en caso de usarse para buscar contenidos), así como cuando el usuario quiera acceder a un contenido directamente desde una lista, una reseña, una puntuación o un registro, por ejemplo.

2.1. Tecnologías, herramientas y metodología

En esta sección se explica, a grandes rasgos, cómo se ha tratado de conseguir los objetivos previamente propuestos. Para ello, se describirán las tecnologías y las herramientas que se han empleado en el proyecto, así como la metodología utilizada para abordar el desarrollo de la aplicación.

2.1.1 Tecnologías

Como ya se ha mencionado anteriormente en el capítulo, el lenguaje de programación elegido para desarrollar el sistema es Scala. Esta elección responde a dos motivos principales: el primero, una motivación personal por profundizar en el conocimiento de este lenguaje, impulsada por el interés y la curiosidad de comprobar sus capacidades y su idoneidad para el desarrollo de aplicaciones web; y el segundo, por tratarse de un lenguaje que promueve la programación funcional y facilita la creación de sistemas escalables.

Por otra parte, las librerías que, en mayor medida, se han usado en este Trabajo de Fin de Grado son cuatro: Tapir, para instrumentar endpoints mediante programación funcional; http4s, para construir servicios HTTP (como clientes y servidores) siguiendo un enfoque puramente funcional; Circe, para la codificación y decodificación entre *case classes* y objetos JSON; y Cats Effect, para la gestión de efectos secundarios y recursos de manera funcional.

2.1.2. Herramientas

Las herramientas que se han usado para facilitar la creación de la aplicación en su conjunto son las siguientes:

- IntelliJ IDEA: Se ha elegido este IDE como entorno de desarrollo debido a que ofrece muchas facilidades para programar con Scala, y también por la familiaridad que ya tenía con esta herramienta al haberla usado anteriormente en el grado.
- APIs externas: Para obtener detalles sobre contenidos multimedia que pudiesen ser utilizados como parte de la aplicación, se han elegido las APIs de TMDb (películas y series), IGDB (videojuegos) y Google Books (libros).

- Postman: Permite ejecutar solicitudes HTTP a una URL con los parámetros deseados, lo cual facilita la realización de todo tipo de pruebas frente a otras opciones, como el uso de comandos a través de la terminal de Windows.
- ChatGPT: Se ha utilizado esta herramienta, principalmente, para la resolución de dudas de programación, pues para algunas de ellas no conseguía encontrar respuesta en Internet, o no al menos que pudiese entender fácilmente. Por ello, me ha facilitado bastante el entender mejor algunos conceptos y varias de las tecnologías utilizadas en la aplicación.
- Git y GitHub: Para el control de versiones y la gestión del código fuente, se ha utilizado Git, creando ramas específicas para cada nueva funcionalidad o grupo de cambios. Esto ha permitido desarrollar de forma controlada, realizar integraciones graduales y mantener un historial claro de la evolución del proyecto. El repositorio ha estado alojado en GitHub, lo que ha facilitado tanto el almacenamiento remoto como el seguimiento de los avances. La dirección de dicho repositorio es la siguiente: <https://github.com/dcarh/TFGDiegoCarrera>. Para ver la versión final de la aplicación, habrá que consultar la rama *release*.

Tanto las herramientas aquí descritas, como las tecnologías listadas en la subsección anterior, tienen referenciados sus respectivos sitios web oficiales en el anexo **Sitios web oficiales de las tecnologías y herramientas utilizadas**.

2.1.3. Metodología

En cuanto a la metodología empleada, no se ha seguido ningún marco de trabajo formal como Scrum o Kanban. En su lugar, el desarrollo se ha abordado de manera iterativa y exploratoria, comenzando por la implementación de pequeñas funcionalidades que resultaban alcanzables en cada momento. A partir de estas primeras versiones, se ha procedido a su ampliación y adaptación al resto del dominio de la aplicación. Este proceso se ha repetido de forma progresiva: partiendo de soluciones básicas, validándolas y posteriormente generalizándolas y enriqueciéndolas para cubrir nuevos casos de uso y funcionalidades más complejas.

3. DESCRIPCIÓN INFORMÁTICA

Esta sección ha sido estructurada en función de los objetivos previamente definidos. Es decir, incluirá todo lo relativo al análisis de la aplicación y de su arquitectura, a la instrumentación de los endpoints del dominio propio, y a la instrumentación de los endpoints que permiten la comunicación con servicios externos.

3.1. Análisis y arquitectura del sistema

Antes de proceder con las explicaciones pertinentes acerca de la implementación de ambos tipos de endpoints, es oportuno realizar un análisis para comprender mejor cuáles son las necesidades de la aplicación a nivel funcional y a nivel estructural, con el objetivo de definir qué debe hacer el sistema, cómo interactúan los distintos actores con él, y qué elementos conforman su estructura interna. Con tal propósito, he decidido dividir esta sección en los siguientes apartados:

- Funcionalidades del sistema. Se enumeran y describen todas las funcionalidades que darán forma a la aplicación.
- Modelo del dominio. A través de un diagrama de clases se presentarán las entidades clave y sus relaciones dentro de la aplicación.
- Análisis funcional. Mediante diagramas de casos de uso, se identificarán las principales interacciones entre los usuarios y la aplicación.
- Arquitectura del sistema. Se explica la lógica de comunicación y acceso a los recursos del sistema.

Habiendo visto todo esto, será más fácil comprender las bases conceptuales necesarias para tratar aspectos posteriores de la implementación de endpoints.

3.1.1. Funcionalidades del sistema

En el contexto de una aplicación como la que he descrito en la sección de Objetivos, considero que las funcionalidades que debería cubrir son las siguientes:

- Reseñas. El usuario debe poder crear (y editar o eliminar) reseñas de los contenidos que consume. Una reseña contendrá el texto en el que el usuario expresa su opinión acerca de ese contenido, y puede recibir *me gusta* y comentarios. También, el usuario puede especificar si la reseña contiene destripes o si quiere permitir comentarios sobre la reseña.

- Puntuaciones. El usuario debe poder puntuar los distintos contenidos presentes en la aplicación con un número entre el 0 y el 10, ambos inclusive.
- Registros. El usuario debe poder crear (y editar o eliminar) registros de los contenidos que va consumiendo. Estos registros pueden incluir una reseña, una puntuación, el estado de progreso respecto al contenido (si se ha completado, si está en progreso, si se ha pausado dicho progreso o si se ha abandonado), la fecha en la que se ha iniciado el progreso, la fecha en la que se ha finalizado ese progreso (en caso de haberlo hecho) y el sistema en el que se ha consumido (solo en caso de que sea un videojuego). También se puede especificar si dicho contenido ya se ha completado otras veces en el pasado.
- Listas de contenidos. El usuario debe poder crear (y editar o eliminar) colecciones que incluyan contenidos según el criterio que se desee. Estas listas podrán tener un nombre y una descripción, el conjunto de contenidos incluidos (que se podrán ordenar, añadir, eliminar). Aparte, el usuario podrá especificar si quiere permitir comentarios a la lista, así como la visibilidad (todo el público, solo seguidores o nadie) de esta y si se trata o no de un ranking. Por otra parte, otros usuarios podrán publicar un comentario respecto a la lista (en caso de que se permita) y también podrán dar *me gusta*.
- Comentarios. El usuario podrá publicar comentarios (como los de cualquier red social que se nos ocurra) en listas de contenidos, en reseñas y en otros comentarios. Otros usuarios podrán dar *me gusta* a estos comentarios.
- *Me gusta*. El usuario podrá dar *me gusta* a reseñas, listas de contenidos o comentarios de otros usuarios.
- Seguimiento de contenidos. Además de hacerlo mediante los registros anteriormente mencionados, el usuario también podrá marcar el estado de seguimiento de un contenido sin necesidad de crear un registro. Los estados de seguimiento son:
 - *Completado*. Indica que el usuario ha finalizado el progreso de un contenido. Por ejemplo, cuando se termina una película o serie.
 - *En progreso*. Indica que el usuario está actualmente progresando con un contenido. Es decir, que está viendo una serie, jugando a un videojuego o leyendo a un libro. No necesariamente tiene que marcarse en el momento exacto de realizar la acción, sino que la intención es que este estado se mantenga desde que el usuario empieza el contenido hasta que lo acaba, pase el tiempo que pase entre medias, ya sean varias horas o varias semanas. Por tanto, este estado está disponible para aquellos

contenidos cuyo progreso normalmente se extiende a periodos de tiempo más largos de un par de horas: series (y sus respectivas temporadas), videojuegos y libros.

- *Progreso pausado*. Indica que el usuario, por el motivo que sea, ha dejado de progresar durante un tiempo con el contenido referenciado, pero tiene pensado retomarlo en algún momento. Respecto a los contenidos disponibles para este estado, aplica de la misma manera que *En progreso*.
- *Abandonado*. Indica que el usuario ha dejado un contenido sin terminar definitivamente. Aplica a todos los contenidos.
- *Pendiente*. Indica que el usuario, en algún momento, desea consumir el contenido especificado. Este estado no está disponible en los registros debido a que, en realidad, no indica el progreso realizado respecto a un contenido, sino únicamente la intención del usuario de iniciarlo en algún momento.
- Conversaciones. El usuario podrá mantener conversaciones privadas con otros usuarios. También podrá archivar o eliminar chats.
- Seguimiento de otros usuarios. El usuario podrá seguir a otros usuarios y, de la misma forma, el usuario podrá ser seguido por otros usuarios.
- Bloqueo de usuarios. El usuario podrá bloquear usuarios y ser bloqueado por estos.
- Contenidos favoritos. En la información del usuario habrá una sección llamada *Favoritos*, en la que el usuario puede especificar su película favorita, su serie favorita, su videojuego favorito, y su libro favorito.
- Perfil de usuario. El usuario contará con un perfil de usuario que consta de nombre de usuario, contraseña, biografía, email y localización. Esta información se podrá editar en todo momento.
- Información de usuario. Reuniendo todo lo visto en los anteriores puntos, un usuario podrá ver su perfil de usuario, sus contenidos favoritos y toda la información respecto a sus interacciones con los contenidos y otros usuarios. Es decir, podrá ver la siguiente información:
 - Contenidos favoritos.
 - Contenidos completados.
 - Contenidos en progreso.
 - Contenidos cuyo progreso se ha pausado.

- Contenidos abandonados.
- Contenidos pendientes.
- Listas de contenidos.
- Registros.
- Reseñas.
- Puntuaciones.
- *Me gusta* dados.
- Comentarios.
- Seguidores.
- Usuarios seguidos.
- Usuarios bloqueados.
- Conversaciones (no archivadas).
- Conversaciones archivadas.
- Herramienta de búsqueda. El usuario podrá utilizar una herramienta que le permita buscar contenidos, listas y otros usuarios en la aplicación mediante coincidencia de texto con el nombre introducido.

3.1.2. Modelo del dominio

El modelo del dominio representa, a nivel conceptual, las principales entidades que forman parte de la aplicación, así como las relaciones existentes entre ellas, de manera que se puedan entender fácilmente las características esenciales del problema que nos atañe sin entrar en detalles demasiado técnicos.

Para llevar a cabo dicho objetivo, he realizado un diagrama de clases UML (Unified Modeling Language) que describe de forma estructurada las clases del sistema, sus atributos, y la forma en que se relacionan entre ellas. Este modelo se ha elaborado teniendo en cuenta las funcionalidades previamente descritas, extrayendo de ellas las entidades que nos permitirán implementar una lógica de aplicación que se corresponda con dichas funcionalidades, dando forma al sistema.

Como hay un gran número de relaciones entre las clases existente, el diagrama resultante es demasiado grande e ilegible como para mostrarlo en una sola imagen. Por ello, he tomado dos decisiones: la primera, generar un diagrama con todas las clases y sin relaciones entre ellas; y la segunda, partir el hipotético diagrama con todas las clases y todas las relaciones, en diagramas que muestran las relaciones entre clases siguiendo una cierta temática.

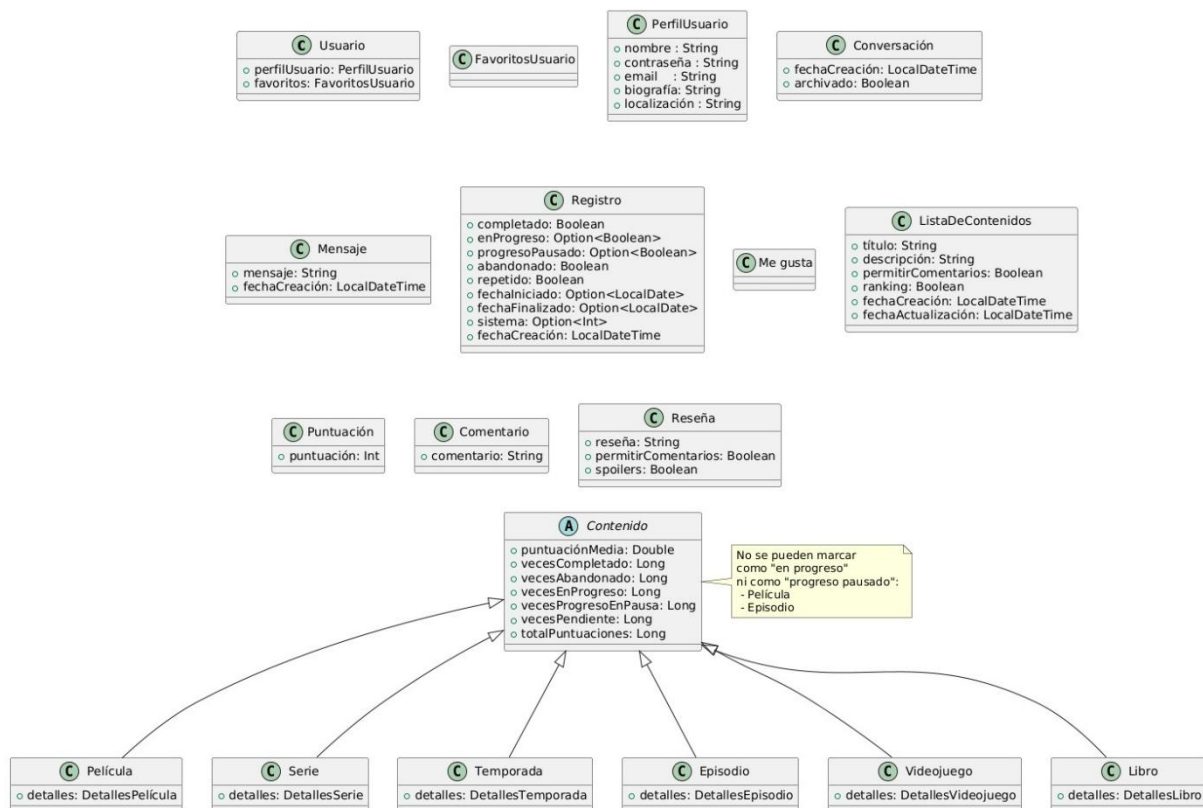


Figura 1. Diagrama de clases con todas las entidades.

Para empezar, tenemos el diagrama con todas las clases que se han podido extraer de los casos de uso, y sin relaciones entre estas (únicamente la clase abstracta *Contenido*, que engloba a los distintos contenidos multimedia), mostrado en la **Figura 1**.

Podemos distinguir las clases *Usuario*, *FavoritosUsuario* y *PerfilUsuario*, a las que a partir de ahora me referiré en conjunto como “usuarios”; *Conversación* y *Mensaje*, a las que me voy a referir en conjunto como “conversaciones”; *Registro*, *Me gusta*, *ListaDeContenidos*, *Puntuación*, *Comentario* y *Reseña*, a las que me referiré en conjunto como “social”; y *Contenido*, *Película*, *Serie*, *Temporada*, *Episodio*, *Videojuego* y *Libro*, a las que, en conjunto, me voy a referir como “contenidos multimedia” o “contenidos”. Todo esto sea por simplificar un poco la terminología a la hora de abordar los siguientes diagramas.

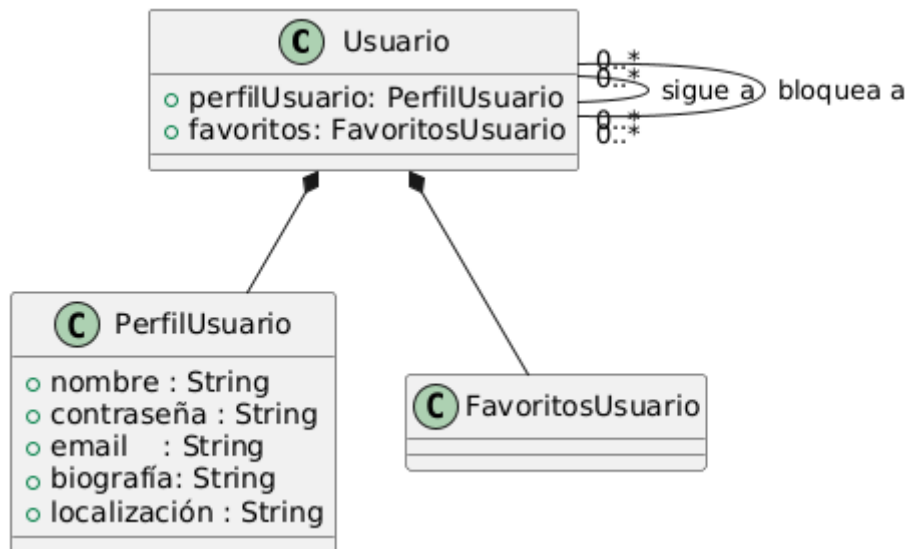


Figura 2. Diagrama de clases con entidades del conjunto de clases de usuarios.

En la **Figura 2** se pueden ver las clases del conjunto de clases de usuarios. La clase *Usuario* es dueña de *PerfilUsuario* y *FavoritosUsuario* (pudiendo tener solo una de cada), y se relaciona consigo misma de la siguiente manera:

- Un usuario puede seguir a ninguno o múltiples usuarios y puede ser seguido por ninguno o múltiples usuarios.
- Un usuario puede bloquear a ninguno o múltiples usuarios y puede ser bloqueado por ninguno o múltiples usuarios.

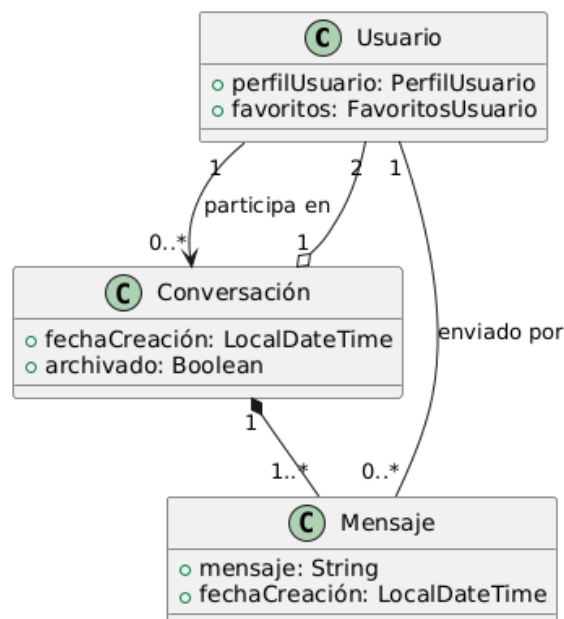


Figura 3. Diagrama de clases para relaciones entre *Usuario* y conjunto de clases de conversaciones.

El diagrama de la **Figura 3** muestra las relaciones entre la clase *Usuario* y las clases del conjunto de conversaciones:

- Un usuario puede participar en ninguna o en múltiples conversaciones.
- Una conversación está compuesta por dos usuarios.
- Un usuario puede enviar múltiples mensajes (o ninguno)
- Una conversación contiene uno o más mensajes.

Por otra parte, hay un diagrama que detalla las relaciones entre la clase *Usuario* y las entidades del conjunto de clases social (**Figura 4**).

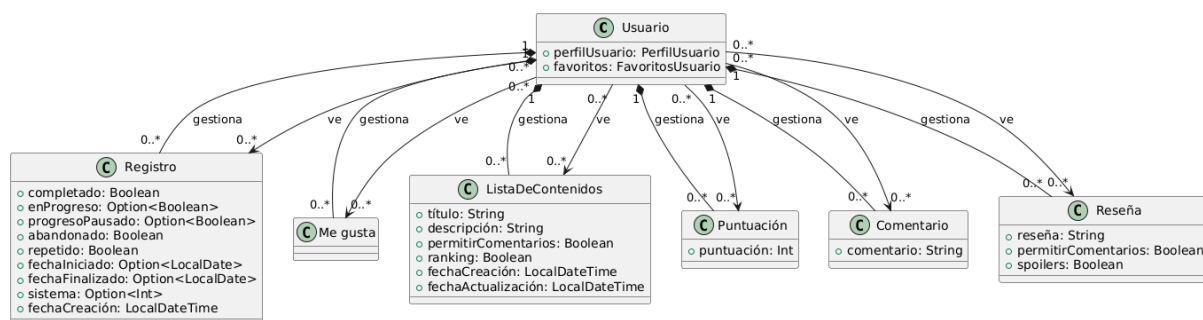


Figura 4. Diagrama de clases para relaciones entre *Usuario* y entidades del conjunto de clases social.

En este diagrama las relaciones que existen son muy sencillas, pues describen que *Usuario* puede ver o gestionar (en relación de composición):

- Múltiples registros (o ninguno).
- Múltiples *me gusta* (o ninguno).
- Múltiples listas de contenidos (o ninguna).
- Múltiples puntuaciones (o ninguna).
- Múltiples comentarios (o ninguno).
- Múltiples reseñas (o ninguna).

La diferencia entre que un usuario pueda ver o gestionar una de estas clases reside en si es o no el dueño de estas. Si un usuario, por ejemplo, ha creado una reseña de una película, podrá editarla o eliminarla si quiere, y esta reseña no puede existir sin el usuario. Por otra parte, cualquier usuario podrá ver cualquier reseña si así lo desea, sin necesidad de que sea suya.

Aprovechando que el anterior diagrama incluía el conjunto de clases social, ahora podemos ver uno (**Figura 5**) que detalla las relaciones entre las clases de este conjunto.

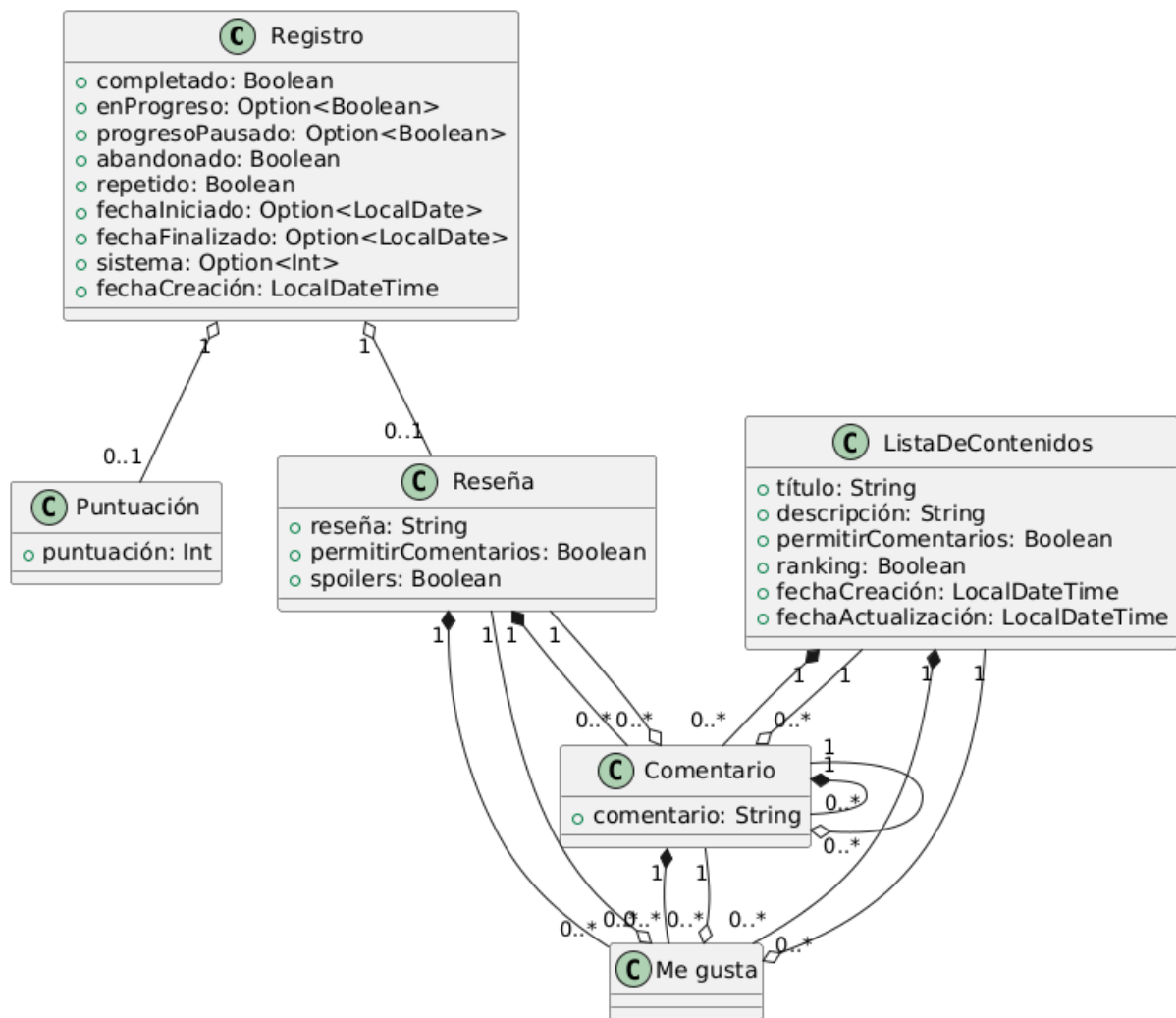


Figura 5. Diagrama de clases para relaciones entre entidades del conjunto de clases social.

Las relaciones se pueden resumir de la siguiente manera:

- Un registro puede (o no) tener una puntuación o una reseña, aunque estas pueden existir fuera del registro.
- Tanto las reseñas, como los comentarios y las listas de contenidos pueden tener múltiples (o ningún) comentario o *me gusta*. Estos comentarios o *me gusta* solo pueden hacerse sobre objetos de dichas clases, y no pueden existir por sí mismos fuera de estas (de ahí la relación de composición).
- Por otra parte, tanto los comentarios como los *me gusta* referencian a una reseña, a una lista de contenidos o a un comentario, por lo que estaríamos hablando de una relación de agregación simple.

Por otra parte, se muestra en la **Figura 6** las relaciones entre la clase *Usuario* y las clases del conjunto de contenidos multimedia.

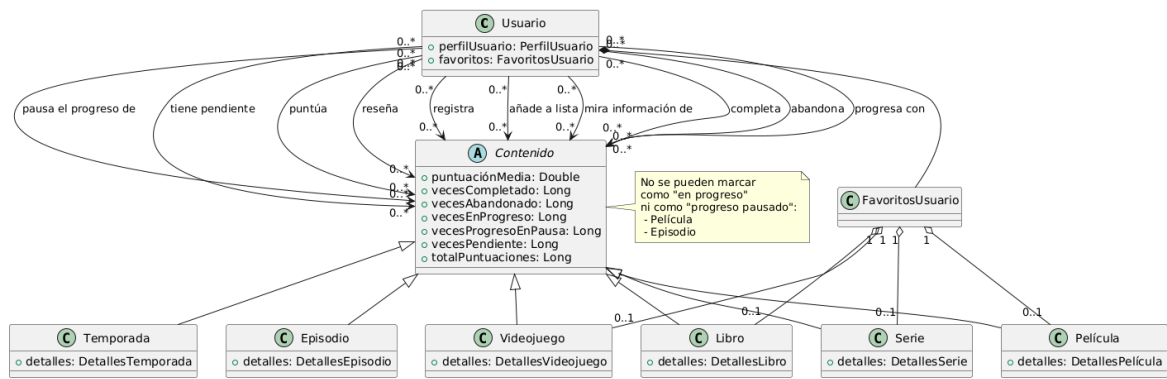


Figura 6. Diagrama de clases para relaciones entre *Usuario* y clases del conjunto de contenidos multimedia.

Como ya hemos visto antes, la clase *Usuario* es dueña de *FavoritosUsuario*, que está compuesta por una película, una serie, un libro y un videojuego. Por otra parte, podemos apreciar que todos los usuarios pueden llevar a cabo las siguientes acciones sobre los contenidos presentes:

- Completar
- Progresar
- Pausar progreso
- Abandonar
- Tener pendiente
- Puntuar
- Reseñar
- Registrar
- Mirar información

Por último, en el diagrama de la **Figura 7** aparecen las relaciones entre las clases del conjunto de contenidos, y las clases del conjunto social (solo aquellas clases que ejercen algún tipo de relación). Se puede apreciar que un contenido multimedia puede ser referenciado por múltiples (o ninguna) instancias de clases del conjunto social.

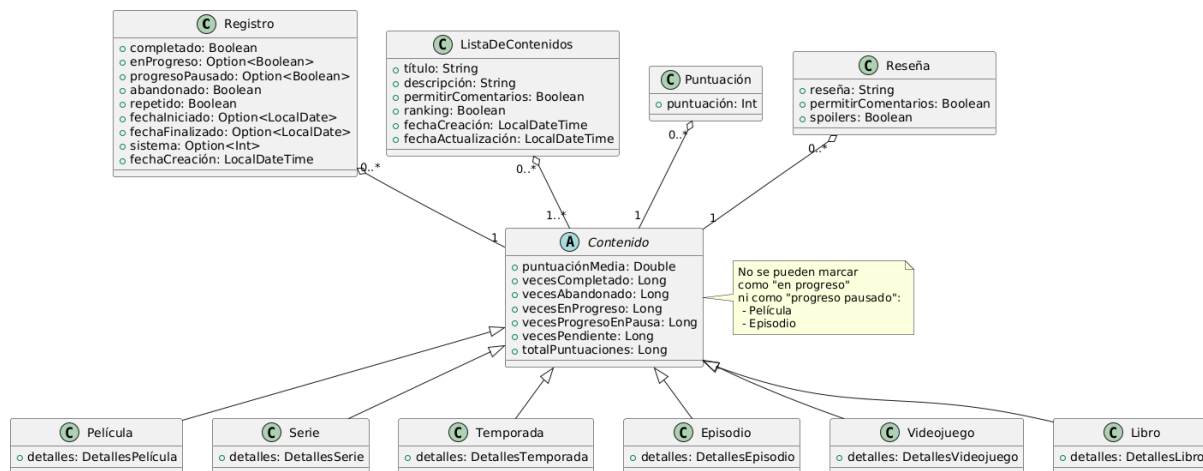


Figura 7. Diagrama de clases para relaciones entre clases del conjunto de contenidos y entidades del conjunto social.

3.1.3. Análisis funcional

Los diagramas de casos de uso que se mostrarán tratarán de explicar, a alto nivel, las posibilidades que ofrecerá la aplicación a los distintos actores involucrados en su uso, en tanto que todas las funcionalidades descritas anteriormente en la sección de objetivos puedan cumplirse. Esto se hará desde el punto de vista del usuario, por lo que no entrará en detalles respecto a lo que puede hacer la aplicación internamente para llevar a cabo esas interacciones.

Antes de proceder con dichos diagramas, es conveniente describir a los distintos actores involucrados en el uso de la aplicación. En este caso, son solo dos: los usuarios autenticados y los usuarios no autenticados.

- Un usuario autenticado es todo aquel usuario que tenga una cuenta en la aplicación, con su correspondiente email, nombre de usuario, contraseña, etc.; y que haya iniciado sesión. Este usuario podrá acceder a todas las posibles acciones que ofrece la aplicación.
- El usuario no autenticado es aquel que no ha procedido a iniciar sesión en la aplicación, ya sea porque aún no haya querido hacerlo o porque no disponga de una cuenta, en cuyo caso debería primero registrarse. Este usuario tendrá acceso a una parte muy limitada de las funcionalidades mencionadas.

En cuanto a los casos de uso, se encuentran repartidos en diagramas según su temática y también según las clases del modelo del dominio en torno a las cuales se producen. Para empezar, tenemos el diagrama de casos de uso que se pueden dar en la página principal del

sistema (**Figura 8**), que es desde donde parten el resto de las posibles acciones. Este será el único diagrama en el que aparezca el actor *Usuario no autenticado*.

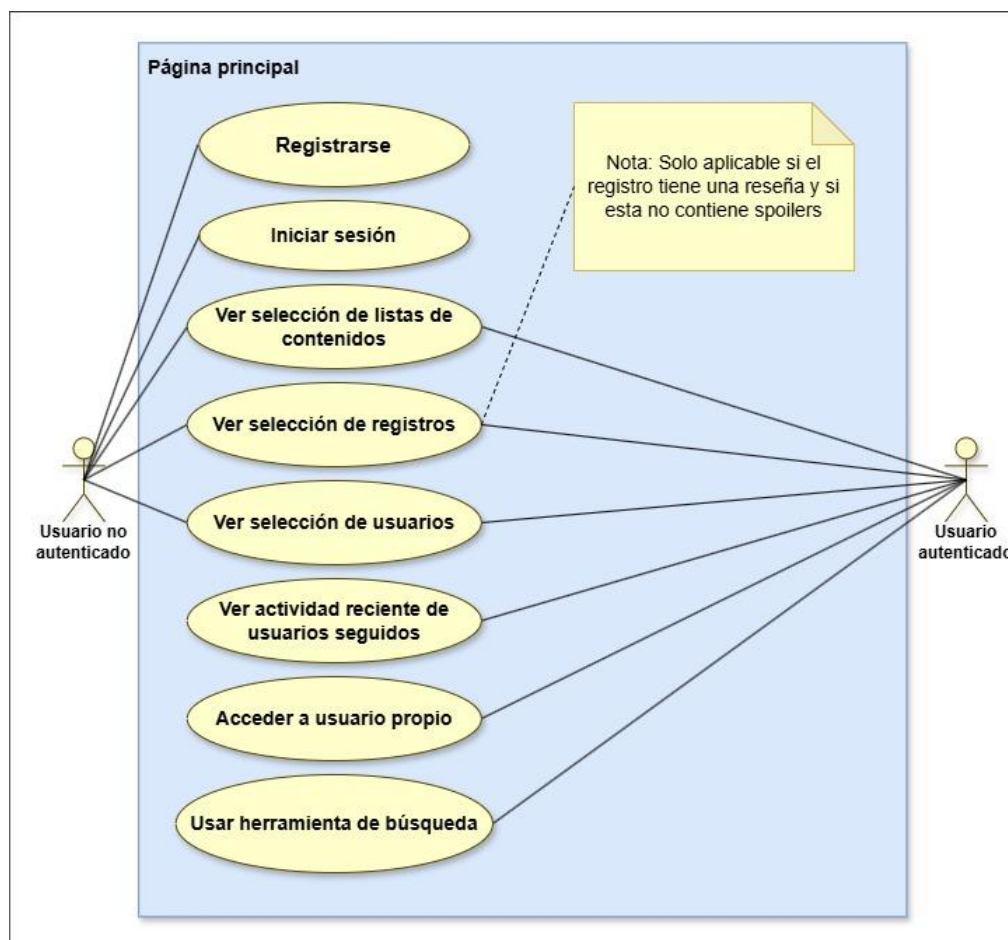


Figura 8. Diagrama de casos de uso de la página principal.

Como se puede observar, en esta pantalla se pueden llevar a cabo acciones relacionadas con la autenticación, la visualización de contenido destacado para ambos tipos de actores, e interacciones algo más avanzadas que requieren haberse autenticado.

Uno de los casos de uso de esta pantalla es el de usar la herramienta de búsqueda, que tiene su propio diagrama (**Figura 9**). En él se muestran las posibilidades a la hora de buscar cosas en la aplicación. Por una parte, podemos buscar todo tipo de contenidos multimedia, excluyendo temporadas y episodios de series, dado que, para ello, deberíamos acceder antes a una serie en concreto. Estos contenidos multimedia han de buscarse por separado. Por ejemplo, si deseo buscar una serie, la aplicación solo encontrará series.

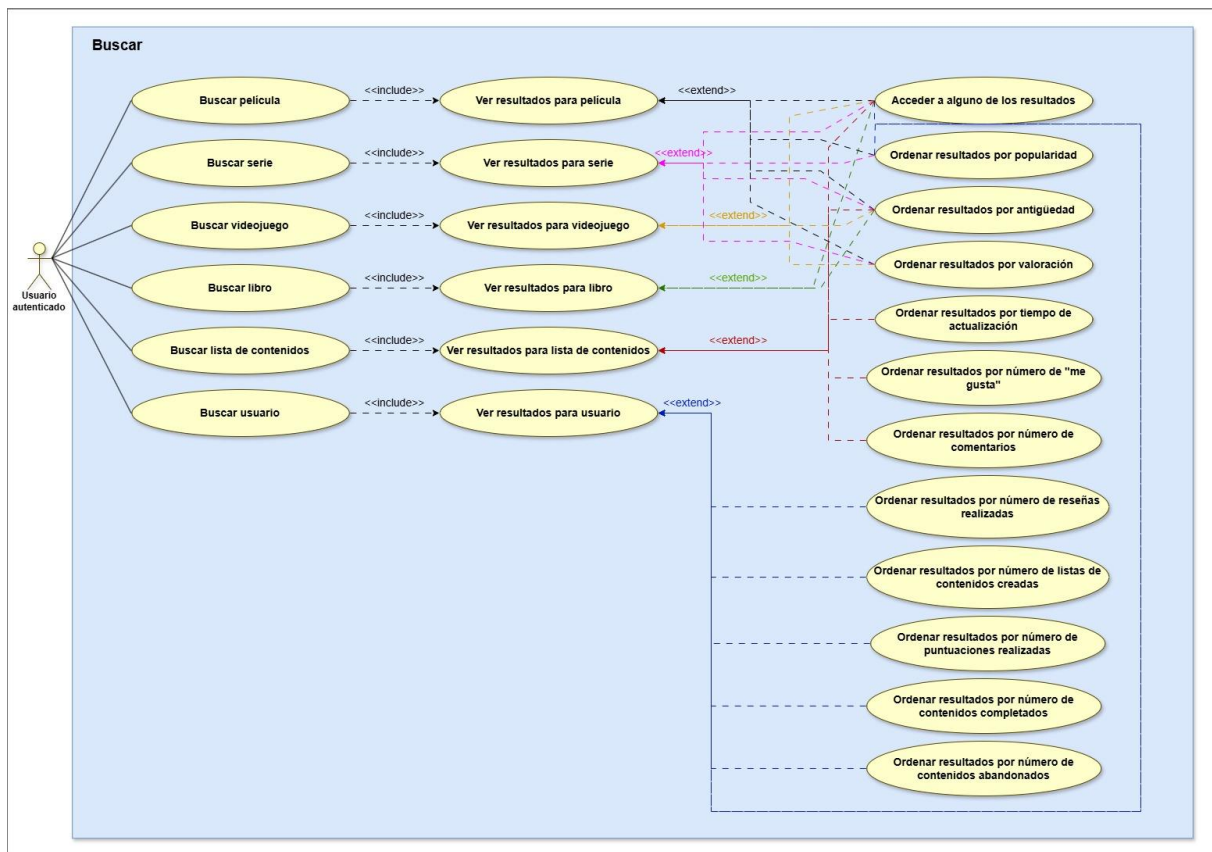


Figura 9. Diagrama de casos de uso de la herramienta de búsqueda.

Además de contenidos multimedia, también podremos buscar usuarios y listas de contenidos creadas por los usuarios.

Las búsquedas se realizarán por coincidencia de texto con el nombre del elemento buscado, y los resultados podrán ordenarse atendiendo a distintos criterios según lo que se haya buscado (los criterios varían según el elemento debido a diferencias en la información que ofrecen).

El siguiente diagrama por mostrar es el que se refiere a los contenidos multimedia (**Figura 10**). El diagrama en cuestión muestra las acciones que podemos realizar al acceder a un contenido (esto, en una interfaz, lo haríamos al pinchar en un contenido que se nos muestre como resultado en la herramienta de búsqueda, por ejemplo). El usuario puede ver los detalles del contenido (tanto los proporcionados por la API, como los generados por la aplicación), así llevar a cabo algunas acciones como crear una reseña o, incluso, ver interacciones que han generado otros usuarios.

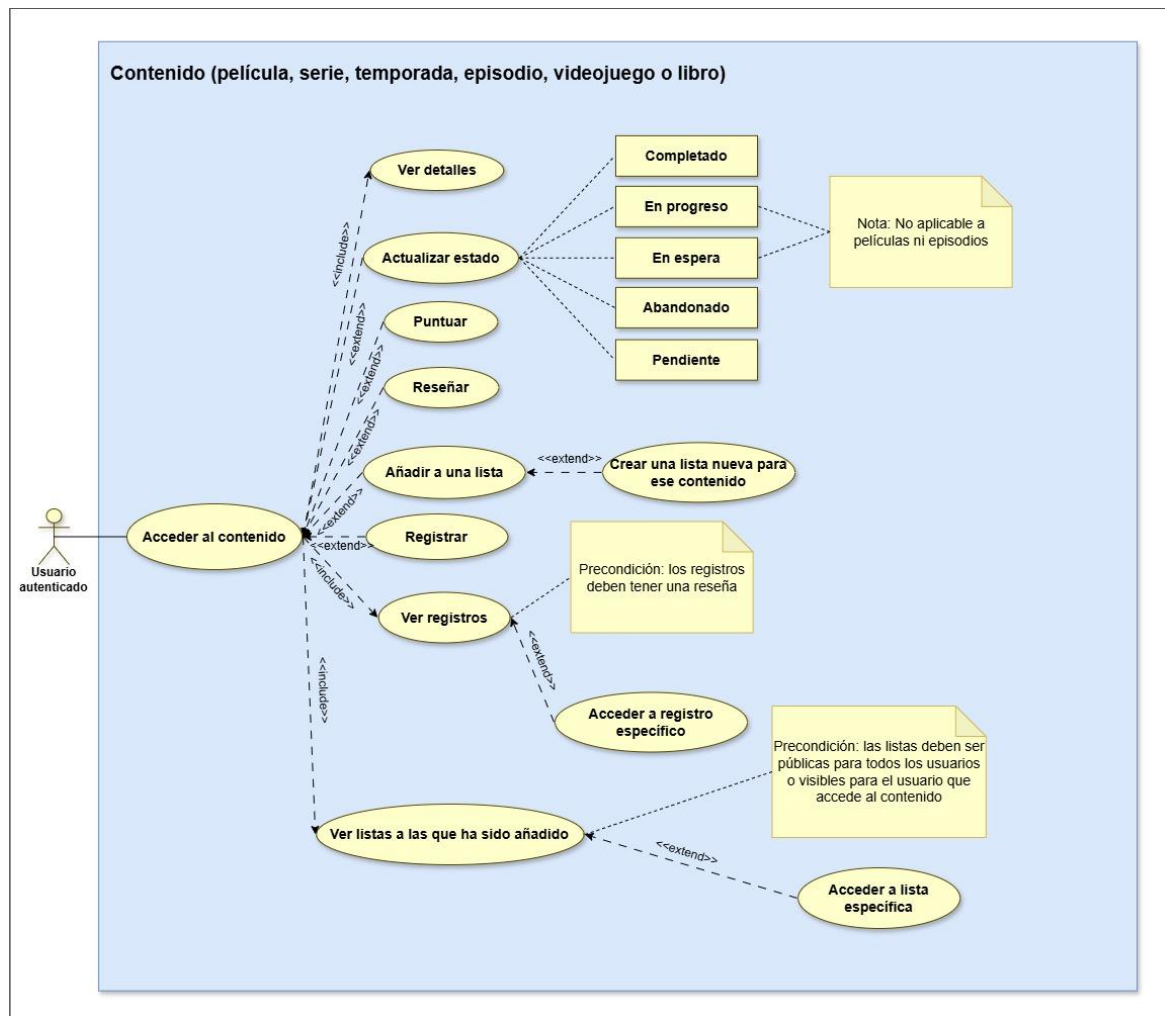


Figura 10. Diagrama de casos de uso relacionados con los contenidos multimedia.

3.1.4. Arquitectura del sistema

El sistema desarrollado adopta una arquitectura de tipo cliente-servidor, en la cual se ha implementado únicamente la parte correspondiente al servidor. Este servidor ha sido desarrollado en Scala, utilizando la librería Tapir para la definición de los endpoints HTTP que constituyen la interfaz de comunicación con el sistema.

Si bien en las fases iniciales del proyecto se planteó la posibilidad de seguir el patrón Model-View-Controller (MVC), la evolución del desarrollo ha llevado a una estructura más orientada a servicios, sin interfaz de usuario (View) ni persistencia de datos (Model), por lo que no se puede hablar de una implementación completa del patrón MVC. No obstante, la organización interna del Back-End mantiene una separación clara de responsabilidades, lo cual refleja cierta inspiración en dicho patrón.

Los componentes principales de la arquitectura son los siguientes:

- Servidor (Back-End). El servidor se organiza en varios niveles funcionales:
 - Endpoints: Definidos utilizando Tapir, describen las operaciones disponibles para el cliente, especificando sus métodos HTTP, parámetros, cuerpos de petición y respuestas.
 - Lógicas: Contienen la implementación de la lógica de negocio asociada a cada operación del sistema.
 - Rutas: Asociaciones entre los endpoints definidos y su lógica de negocio correspondiente. Actúan como un punto intermedio entre la capa de definición y la lógica funcional.
 - Persistencia. No se ha incorporado un sistema de almacenamiento persistente. En su lugar, se han definido clases y estructuras de datos pre-pobladas, que permiten simular el comportamiento de una fuente de datos. Como resultado, las operaciones realizadas al ejecutar los endpoints del dominio propio de la aplicación no persisten información entre peticiones, sino que se simulan para permitir la validación del flujo de ejecución.
- Cliente (Front-End). El sistema, en su versión actual, no incluye una interfaz de usuario gráfica. En su lugar, las interacciones con el Back-End se realizan mediante la herramienta Postman, que permite lanzar peticiones HTTP a los endpoints expuestos. Esto representa un caso típico de uso en una arquitectura cliente-servidor, en la que el cliente es externo y no está embebido dentro del propio sistema.
- Integración con APIs externas. Uno de los aspectos clave del sistema es su capacidad para interactuar con fuentes de información externas. Se han implementado clientes específicos para realizar peticiones HTTP a las siguientes APIs:
 - TMDB: Se trata de la API de una gran base de datos de películas y series que es de uso libre siempre que no sea con usos comerciales (aunque la gran mayoría de aplicaciones que he visto la usan). Para usarla, uno debe registrarse para así obtener una clave de API con la que poder llevar a cabo las peticiones HTTP que se quieran hacer. TMDB cuenta con un límite de peticiones de, aproximadamente, 50 por segundo y, en cuanto a la información que proporciona, se puede obtener el nombre, la fecha de salida, el idioma original, el reparto, la productora, entre otros muchos datos. Otras opciones que se han tenido en cuenta son la de IMDB, cuyo ha sido declinado por no tener un plan de uso gratis de la API, y OMDb, que tiene muy buenas funcionalidades, pero unas prestaciones menores que las de TMDB, sobre todo por el límite de 10 peticiones por minuto.

- IGDB: API proveedora de contenido de videojuegos que cuenta con un catálogo de unos 430.000 videojuegos y permite 4 peticiones por segundo. Usando esta API, se puede encontrar información tal como el nombre de un videojuego, su año de salida, las consolas o sistemas para los que está disponible, videojuegos similares, etc. Otros posibles servicios cuyo uso se ha contemplado, pero que finalmente se ha descartado, son MobyGames (un límite de peticiones de 1 petición por cada 10 segundos, lo cual puede resultar un poco escaso), RAWG (provee menos información que IGDB y solo permite 20.000 peticiones al mes, lo cual se traduce a algo menos de una petición cada dos minutos) y CheapShark, que sirve para comparar precios de juegos digitales de PC y que podría utilizarse en caso de querer ampliar funcionalidades del proyecto, pero que no se adhiere a lo que se busca en este desarrollo.
- Google Books: Es la API proveedora de información de libros desarrollada por Google. Los principales motivos para su elección son su un límite diario de 100.000 peticiones y lo extensa que puede llegar a ser la información que proporciona. De las otras APIs que se han tenido en cuenta, quizá la más interesante sea la de Open Library, que es gratis y no tiene límite de peticiones, pero cuya información proporciona acerca de los libros es más limitada. En cuanto al resto de las opciones que se han barajado (ISBNdb, WorldCat, Bookshare, etc.), o estaban dirigidas más a entidades e instituciones, o requerían coordinación directa con sus desarrolladores para utilizarlas. Dicho esto, es importante mencionar el hecho de que, tanto la API de Google Books como las otras que se han mencionado, normalmente no proporcionan información respecto a los títulos, sino a las ediciones de estos. Es decir, al buscar un título, no buscarán un libro con su fecha de publicación original y datos generales de este, sino que más bien buscará ediciones de dicho libro, con la fecha de publicación de dicha edición, el idioma, la editora, etc. Esto no es un inconveniente demasiado grande, pero dista ligeramente de la lógica de la aplicación que quiero crear.

Los servicios que se acaban de ver y cuya elección se ha justificado enriquecen los datos ofrecidos por la aplicación sin necesidad de almacenar información de forma local, lo que simplifica el desarrollo y permite centrarse en la lógica de consulta y transformación de datos.

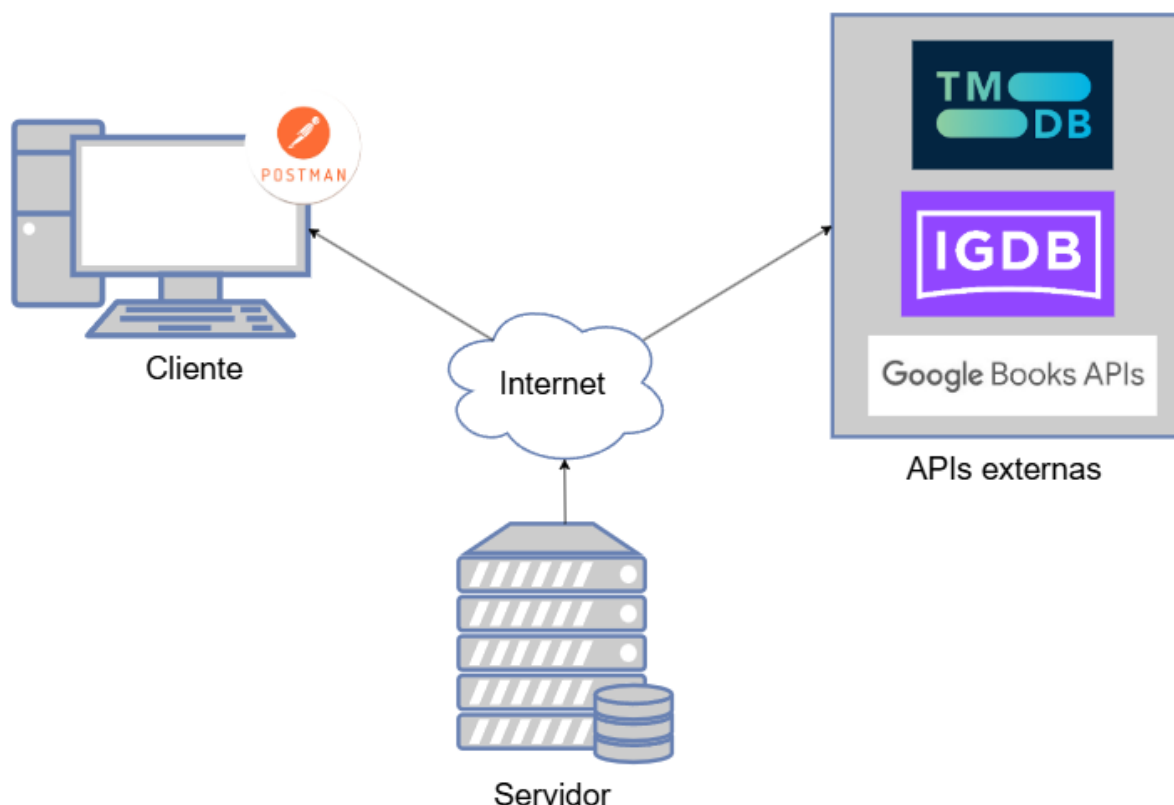


Figura 11. Diagrama de la arquitectura implementada.

En resumen, el sistema implementado se basa en una arquitectura cliente-servidor, con un Back-End modular que expone su funcionalidad mediante endpoints HTTP. Aunque se consideró inicialmente una arquitectura MVC, la falta de una capa de persistencia y de una vista (Front-End) limita su aplicación. A pesar de ello, se ha mantenido una clara separación de responsabilidades, lo que facilita la mantenibilidad y escalabilidad futura del sistema.

3.2. Instrumentación de endpoints del dominio propio

Tanto si existe una interfaz gráfica como si no, la manera de interactuar con la aplicación propuesta es a través de endpoints que hay que definir, implementar y exponer. Estos conectan a los usuarios con la lógica de la aplicación, permitiendo activar las operaciones necesarias para que el usuario obtenga un resultado deseado al realizar una determinada acción, como lo podría ser crear una reseña (el resultado sería que la reseña creada apareciese de manera visible en aquellos sitios que debe aparecer, y que los usuarios puedan interactuar con ella si así lo desean).

Para ello, se ha optado por utilizar la librería Tapir, una librería funcional para definir y consumir endpoints de APIs HTTP en Scala. Esta decisión se ha tomado tras analizar varias alternativas disponibles en el ecosistema Scala, como Akka HTTP, http4s o Play Framework, y considerar los requisitos del proyecto, así como las ventajas que Tapir ofrece en comparación.

Tapir permite definir los endpoints de manera declarativa y fuertemente tipada, describiendo no solo la lógica de negocio, sino también el contrato de cada operación HTTP de forma centralizada y reutilizable. Esta aproximación resulta especialmente ventajosa en entornos donde la seguridad de tipos y la coherencia entre cliente y servidor son prioritarias, como es el caso de este trabajo.

Una de las principales razones para elegir Tapir es su integración directa con otras herramientas funcionales del ecosistema Scala, como Cats Effect o http4s, lo que facilita la composición modular y funcional de la lógica de negocio. Además, Tapir permite generar automáticamente clientes HTTP, evitando duplicidades y garantizando la consistencia entre implementación y documentación.

Comparado con otras alternativas:

- Akka HTTP es un conjunto de librerías para la construcción de servicios HTTP en Scala, basado en el modelo de actores de Akka y Akka Streams, lo que facilita el desarrollo de sistemas concurrentes y escalables (Lightbend, s.f.). Su diseño de bajo nivel, que combina en una misma capa la definición de rutas y su lógica de manejo, ofrece gran flexibilidad, pero también puede suponer una mayor carga de complejidad y curva de aprendizaje en comparación con soluciones más declarativas como Tapir.
- http4s es un framework HTTP para Scala que adopta un enfoque puramente funcional, basado en tipos y orientado a trabajar con efectos, especialmente en combinación con bibliotecas como Cats Effect. Proporciona un control exhaustivo sobre el flujo de las peticiones y respuestas, lo que lo hace muy adecuado para sistemas que requieren composición funcional y manejo seguro de efectos. Sin embargo, no ofrece una capa declarativa para la definición de endpoints de alto nivel, lo que puede incrementar la cantidad de código necesario para describir la interfaz de un servicio (http4s, s.f. -a).
- Play Framework es un framework completo para el desarrollo de aplicaciones web en Scala y Java. Proporciona soporte integrado para el manejo de rutas, controladores, vistas, acceso a bases de datos y más, todo ello optimizado para entornos reactivos, sin estado, con uso eficiente de CPU, memoria e hilos (Play Framework Team, s.f.). No obstante, esta característica puede resultar excesiva cuando el objetivo es únicamente definir y exponer endpoints HTTP de forma ligera, como es el caso de este proyecto, donde se ha optado por Tapir, una librería más específica para este propósito.

En resumen, Tapir ha sido seleccionada por su capacidad para describir endpoints de forma expresiva, reutilizable y segura, facilitando al mismo tiempo la integración con otras herramientas funcionales. Esto ha permitido mantener una base de código clara, mantenible y coherente con los principios de diseño funcional adoptados a lo largo del proyecto.

Dicho lo cual, a continuación, se detallan los pasos seguidos para definir, implementar y exponer varios endpoints relacionados con la gestión de las reseñas que se realizan en el sistema, de manera que, a partir de estos ejemplos, se pueda entender fácilmente el proceso seguido para todos los endpoints del dominio propio de la aplicación.

3.2.1. Definición de un endpoint

Entre las cosas que podemos definir en un endpoint, se encuentran el nombre, la descripción, el método HTTP, la ruta por la que se accede al endpoint, los parámetros de entrada (ya sean de ruta, de consulta o el cuerpo de la petición) y los valores devueltos al realizar la llamada, entre los que nos podemos encontrar códigos de estado, mensajes de éxito o error, u objetos JSON (JavaScript Object Notation).

3.2.1.1. Conceptos básicos de un endpoint

Lo primero de todo es entender la naturaleza de estos endpoints. Para poder utilizarlos posteriormente, los definiremos como variables del tipo *PublicEndpoint*, que es un alias del tipo *Endpoint* proporcionado por la librería Tapir (SoftwareMill, 2024 -b). El tipo *Endpoint* es en realidad *Endpoint[A, I, E, O, R]*, de manera que:

- *A* es el tipo del parámetro de entrada de seguridad.
- *I* es el tipo del parámetro de entrada “normal”.
- *E* es el tipo del valor devuelto en caso de error.
- *O* es el tipo del valor devuelto en caso de petición exitosa.
- *R* es el tipo de las capacidades requeridas por las entradas y salidas del endpoint. Como, en el caso de esta aplicación, ningún endpoint necesita de alguna de esas capacidades, este tipo estará cubierto siempre con un *Any*.

El tipo *PublicEndpoint* lo que hace es cubrir siempre el tipo *A* de *Endpoint[A, I, E, O, R]* con *Unit*, omitiendo así el parámetro de entrada de seguridad para cualquier caso; por lo que podemos concluir que $PublicEndpoint[I, E, O, R] = Endpoint[Unit, I, E, O, R]$, tal y como podemos ver en la **Figura 11**.

```
type PublicEndpoint[INPUT, ERROR_OUTPUT, OUTPUT, -R] = Endpoint[Unit, INPUT, ERROR_OUTPUT, OUTPUT, R]
```

Figura 11. Definición del tipo *PublicEndpoint*.

Por tanto, en los endpoints que definamos vamos a sustituir los tipos *I*, *E* y *O* por los tipos que deseemos tener en la entrada, en la salida de error, y en la salida de éxito, respectivamente. En el caso del endpoint mostrado en la **Figura 12**, tendríamos como entrada una dupla de *String* y *List[String]* opcionales que, en este caso serían dos parámetros de consulta.

```
val getAllReviews: PublicEndpoint[(Option[String], Option[List[String]]), UserError, List[Review], Any] =
  reviewsBaseEndpoint(
    "getAllReviews",
    "This endpoint returns a list with all the reviews in the app",
    "GET"
  )
  .in(QueryInputs.querySortBy)
  .in(QueryInputs.queryCategories)
  .out(SocialOutputs.listOfReviewsOutput)
```

Figura 12. Definición del endpoint *getAllReviews*.

El valor devuelto en caso de error sería de tipo *UserError* que es una interfaz usada para englobar las clases de errores de usuario que se pueden dar en la aplicación, como *BadRequest*, por ejemplo. Dicha interfaz y las clases que la extienden se muestran en la **Figura 13**.

```
sealed trait UserError
case class BadRequest(what: String) extends UserError
case class NotFound(what: String) extends UserError
case class Unauthorized(realm: String) extends UserError
case class Conflict(what: String) extends UserError
case class Unknown(code: Int, msg: String) extends UserError
case class NoContent() extends UserError
```

Figura 13. Definición de la interfaz *UserError* y de las clases que la extienden.

Por otra parte, el valor devuelto si la petición se ha completado de manera satisfactoria, es un *List[Review]*. La clase *Review* representa, dentro del código, a la entidad *Reseña* descrita previamente en la sección de **Análisis y arquitectura del sistema**, y se muestra en la **Figura 14**.

```
case class Review(
  id: ReviewId,
  userId: UserId,
  reviewedMediaId: MovieId | TvShowId | (TvShowId, TvSeasonNumber) | (TvShowId, TvSeasonNumber, TvEpisodeNumber) | VideogameId | BookId,
  review: String,
  likesIds: List[LikeId],
  allowReplies: Boolean,
  repliesIds: List[ReplyId],
  spoilers: Boolean
)
```

Figura 14. Clase *Review*.

3.2.1.2.Endpoints base

Volviendo al endpoint *getAllReviews*, podemos ver que llama a un endpoint base, *reviewsBaseEndpoint* (mostrado en la **Figura 15**), al que se le pasan como parámetros tres cadenas de texto: el nombre, la descripción y el método HTTP que ejecuta el endpoint.

```
private val reviewsBaseEndpoint:
    (String, String, String) => PublicEndpoint[Unit, UserError, Unit, Any] =
    (name, description, method) => httpMethodEndpoint(name, description, "reviews", method)
```

Figura 15. Definición del endpoint base *reviewsBaseEndpoint*.

Los dos primeros parámetros se utilizan en Tapir para que, si se desea generar documentación de los endpoints de manera automática, estos tengan un nombre y una descripción textual. De todos modos, yo no los he incluido con el fin de generar esa documentación, sino con el propósito de que todos sean entendibles dentro del código.

El endpoint *reviewsBaseEndpoint* pasa los tres parámetros anteriores al endpoint *httpMethodEndpoint*, mostrado en la **Figura 16**, añadiéndole otro parámetro con la cadena de texto *reviews*, que es añadida a la ruta de *reviewsBaseEndpoint* y de todos los endpoints que lo utilicen.

```
val httpMethodEndpoint:
    (String, String, String, String) => PublicEndpoint[Unit, UserError, Unit, Any] =
    {
        case (name, description, path, "GET") => getBaseEndpoint(name, description, path)
        case (name, description, path, "POST") => postEndpoint(name, description, path)
        case (name, description, path, "POST_Igdb") => igdbPostEndpoint(name, description, path)
        case (name, description, path, "PUT") => putBaseEndpoint(name, description, path)
        case (name, description, path, "DELETE") => deleteBaseEndpoint(name, description, path)
    }
```

Figura 16. Definición del endpoint base *httpMethodEndpoint*.

El funcionamiento de *httpMethodEndpoint* es muy sencillo. Según el método HTTP que se le haya pasado, llama al endpoint base definido para ese método. En el caso de *getAllReviews*, se llamaría a *getBaseEndpoint*.

Este endpoint, así como los endpoints base correspondientes a los demás métodos HTTP, define los códigos de estado que pueden devolverse en caso de error y en caso de éxito. Para representar los posibles errores, se utiliza el mecanismo de variantes que ofrece Tapir a través de su método *oneOf*, permitiendo devolver distintos tipos de error como *Not Found*, *Bad Request* o un error desconocido (SoftwareMill, 2024 -c). Cada uno de estos errores está asociado a una clase específica que contiene el mensaje que se mostrará al usuario.

```
private val getBaseEndpoint:
    (String, String, String) => PublicEndpoint[Unit, UserError, Unit, Any] =
    (name, description, path) =>
        baseEndpoint(name, description, path)
            .get
            .errorOut(
                oneOf[UserError](
                    oneOfVariant(StatusCode.NotFound, ErrorOutputsTraits.notFoundOutput),
                    oneOfVariant(StatusCode.BadRequest, ErrorOutputsTraits.badRequestOutput),
                    oneOfDefaultVariant(ErrorOutputsTraits.unknownOutput)
                )
            )
            .out(statusCode(StatusCode.Ok))
```

Figura 17. Definición del endpoint base *getBaseEndpoint*.

Por otra parte, si se completa la ejecución del endpoint correctamente, se devuelve un código de estado *Ok*, junto con el objeto de tipo *O* definido en el endpoint que lo llama (en *getAllReviews* es de tipo *List[Review]*).

Por otra parte, *getBaseEndpoint* realiza una llamada al *baseEndpoint*, pasándole como parámetros el nombre, la descripción, y la ruta del endpoint.

```
private val baseEndpoint:
    (String, String, String) => PublicEndpoint[Unit, Unit, Unit, Any] =
    (name, description, path) =>
        endpoint
            .name(name)
            .description(description)
            .in(path)
```

Figura 18. Definición del endpoint base *baseEndpoint*.

Este endpoint, simplemente, aplica esos parámetros (**Figura 18**).

3.2.1.3. Parámetros de E/S y códecs

Si nos retrotraemos a la definición de *getAllReviews* (**Figura 12**), hay dos parámetros de entrada y uno de salida que aún no han sido abordados. Los parámetros de entrada se definen como valores de tipo *EndpointInput[T]*, una abstracción proporcionada por Tapir para representar entradas como parámetros de consulta (*query*), de ruta (*path*) o cuerpos JSON (*jsonBody*), entre otros (SoftwareMill, 2024 -d). En lo que respecta a los que se utilizan en endpoints del dominio propio de la aplicación, estos pueden ser de tres tipos distintos:

Tanto *querySortBy* (**Figura 19**), como *queryCategories* (**Figura 20**), son parámetros de consulta a los que se les puede dar valor al añadir *?sort_by=valor* y

?categories=valor1,valor2..., respectivamente, a la ruta del endpoint. En el supuesto de que no se incluyesen en la petición, tomarían el valor *None* por defecto.

```
val querySortBy: EndpointInput[Option[String]] =  
  query[Option[String]]("sort_by").default(None)
```

Figura 19. Definición del *EndpointInput querySortBy*.

```
val queryCategories: EndpointInput[Option[List[String]]] =  
  query[Option[List[String]]]("categories").default(None)
```

Figura 20. Definición del *EndpointInput queryCategories*.

Por otra parte, Scala no sabe cómo convertir texto plano a una *Option[List[String]]* (esto debe hacerlo cuando recibe una petición, pues los parámetros de consulta son recibidos en texto plano), por lo que debemos definir un códec que le indique como realizar esta operación, tal y se muestra en la **Figura 21** (SoftwareMill, 2024 -e).

```
implicit val categoriesCodec: Codec[String, Option[List[String]], TextPlain] =  
  Codec.string.mapDecode(s => DecodeResult.Value {  
    if (s.isEmpty) None else Some(s.split(",").toList)  
  })(  
    opt => opt.map(_.mkString(",")).getOrElse("")  
  )
```

Figura 21. Definición del códec *categoriesCodec*.

Aunque no haga falta a la hora de llamar a este endpoint, también se contempla la codificación de un *Option[List[String]]* a texto plano en este mismo códec. Por tanto, define dos operaciones:

- Cómo convertir un texto plano a *Option[List[String]]*. Si el texto no está vacío, lo divide usando las comas a modo de separador, e introduce los valores resultantes en una lista envuelta en un *Some*.
- Cómo convertir de *Option[List[String]]*. Si el *Option* tiene valor, crea una cadena de texto a partir del *List[String]*, introduciendo una coma entre cada elemento. Si el *Option* no tiene valor, se crea una cadena de texto vacía.

En cuanto al parámetro de salida del endpoint, se define como valor del tipo *EndpointOutput[T]* (**Figura 22**). Todos los parámetros de salida de este tipo definidos en el marco de la aplicación serán, o bien, *stringBody*, o *jsonBody*. Es decir, los parámetros de entrada del sistema son siempre cuerpos de petición en formato de texto o de JSON.


```
val listOfReviewsOutput: EndpointOutput[List[Review]] =
  jsonBody[List[Review]].description("The requested list of reviews")
```

Figura 22. Definición del *EndpointOutput* *listOfReviewsOutput*.

Al tratar con la entidad *Review* (Figura 14), uno de sus atributos (*reviewedMediaId*) representa un tipo unión que admite múltiples variantes: *MovieId*, *TvShowId*, una tupla con *TvShowId* y *TvSeasonNumber*, otra con un episodio, o bien *BookId* o *VideogameId*. Dado que esta información debe poder codificarse y decodificarse en formato JSON cuando se utiliza como parte de un *jsonBody*, es necesario definir manualmente un *Encoder* y un *Decoder*, siguiendo el enfoque recomendado por Circe para la creación de códecs personalizados (Typelevel, s.f. -a), y un *Schema*:

- *Encoder* transforma uno de esos tipos a JSON.
- *Decoder* transforma un JSON recibido en uno de esos tipos concretos.
- *Schema* hace que Tapir pueda reconocer correctamente las distintas formas válidas del tipo en la definición del endpoint.

El *Encoder* (Figura 23) se encarga de convertir una instancia del tipo unión a una representación JSON estructurada, en la que:

- El campo *type* indica de forma explícita el subtipo que se está codificando.
- El campo *value* contiene los datos concretos asociados a ese subtipo.

```
implicit val listMediaUnionEncoder2:
  Encoder[MovieId | TvShowId | (TvShowId, TvSeasonNumber) | (TvShowId, TvSeasonNumber, TvEpisodeNumber) | VideogameId | BookId] =
  Encoder.instance {
    case movieId: MovieId => Json.obj("type" -> "MovieId".asJson, "value" -> movieId.value.asJson)
    case tvShowId: TvShowId => Json.obj("type" -> "TvShowId".asJson, "value" -> tvShowId.value.asJson)
    case (tvShowId: TvShowId, seasonNumber: TvSeasonNumber) =>
      Json.obj(
        "type" -> "SeasonNumber".asJson,
        "value" -> Json.obj(
          "tvShowId" -> tvShowId.value.asJson,
          "seasonNumber" -> seasonNumber.value.asJson
        )
      )
    case (tvShowId: TvShowId, seasonNumber: TvSeasonNumber, episodeNumber: TvEpisodeNumber) =>
      Json.obj(
        "type" -> "EpisodeNumber".asJson,
        "value" -> Json.obj(
          "tvShowId" -> tvShowId.value.asJson,
          "seasonNumber" -> seasonNumber.value.asJson,
          "episodeNumber" -> episodeNumber.value.asJson
        )
      )
    case videogameId: VideogameId => Json.obj("type" -> "VideogameId".asJson, "value" -> videogameId.value.asJson)
    case bookId: BookId => Json.obj("type" -> "BookId".asJson, "value" -> bookId.value.asJson)
  }
```

Figura 23. Definición del *Encoder* *listMediaUnionEncoder2*.

Por ejemplo:

- Si se trata de un *MovieId*, el campo *type* tendrá el valor *MovieId* y *value* contendrá el *Long* correspondiente (**Figura 24**).
- Si se trata de una tupla que representa una temporada o episodio, el *value* será un objeto con varios campos (*tvShowId*, *seasonNumber*, etc.), tal y como se muestra en la **Figura 25**.

```
"reviewedMediaId": {  
  "type": "MovieId",  
  "value": 502033  
},
```

Figura 24. Fragmento de un JSON que muestra una instancia de *MovieId* convertida a dicho formato.

```
"reviewedMediaId": {  
  "type": "EpisodeNumber",  
  "value": {  
    "tvShowId": 136315,  
    "seasonNumber": 2,  
    "episodeNumber": 7  
  }  
},
```

Figura 25. Fragmento de un JSON que muestra una instancia de la tupla (*TvShowId*, *TvSeasonNumber*, *TvEpisodeNumber*) convertida a dicho formato.

Este enfoque con discriminador explícito mejora la claridad y evita ambigüedades en el intercambio de datos, lo que es especialmente importante cuando los tipos pueden compartir estructuras similares.

El *Decoder* (**Figura 26**), en cambio, realiza la operación inversa: recibe un JSON y, en función del valor del campo *type*, determina cuál es la variante del tipo unión que se está representando y cómo debe interpretarse el contenido del campo *value*.

- Si el *type* es *MovieId*, se espera que *value* contenga un *Long* que se convierte en una instancia de *MovieId*.
- Para casos más complejos como *EpisodeNumber*, se espera que *value* sea un objeto con varias claves, que se mapearán a las partes de la tupla correspondiente.

Gracias a este *Decoder*, se garantiza que cualquier JSON que cumpla con la estructura esperada pueda ser transformado en su correspondiente instancia en tiempo de ejecución (Typelevel, s.f. -a).

```
implicit val listMediaUnionDecoder2:
  Decoder[MovieId | TvShowId | (TvShowId, TvSeasonNumber) | (TvShowId, TvSeasonNumber, TvEpisodeNumber) | VideogameId | BookId] =
  Decoder.instance { cursor =>
    cursor.downField("type").as[String].flatMap {
      case "MovieId" => cursor.downField("value").as[Long].map(MovieId.apply)
      case "TvShowId" => cursor.downField("value").as[Long].map(TvShowId.apply)
      case "SeasonNumber" =>
        cursor.downField("value").as[Map[String, Long]].map { values =>
          (TvShowId(values("tvShowId")), TvSeasonNumber(values("seasonNumber")))
        }
      case "EpisodeNumber" =>
        cursor.downField("value").as[Map[String, Long]].map { values =>
          (
            TvShowId(values("tvShowId")),
            TvSeasonNumber(values("seasonNumber")),
            TvEpisodeNumber(values("episodeNumber"))
          )
        }
      case "VideogameId" => cursor.downField("value").as[Long].map(VideogameId.apply)
      case "BookId" => cursor.downField("value").as[String].map(BookId.apply)
      case other => Left(DecodingFailure(s"Unknown type: $other", cursor.history))
    }
  }
```

Figura 26. Definición del *Decoder* *listMediaUnionDecoder2*.

Finalmente, el *Schema* (Figura 27) se utiliza por Tapir para describir la estructura interna del tipo en el contexto de la definición de los endpoints (SoftwareMill, 2024 -f). Tapir no puede derivar esquemas automáticamente para tipos unión complejos, por lo que es necesario proporcionarlo explícitamente mediante *Schema.oneOfUsingField*.

- Se utiliza la función *extractDiscriminator2* para obtener el nombre del subtipo en tiempo de ejecución.
- A cada subtipo se le asocia su propio esquema utilizando *Schema.derived*, ya sea para tipos simples (*MovieId*, *BookId*, etc.) o compuestos (*(TvShowId, TvSeasonNumber)*...).

```
private val extractDiscriminator2: (MovieId | TvShowId | (TvShowId, TvSeasonNumber) | (TvShowId, TvSeasonNumber, TvEpisodeNumber) | VideogameId | BookId) => String = {
  case _: MovieId => "MovieId"
  case _: TvShowId => "TvShowId"
  case _: (TvShowId, TvSeasonNumber) => "SeasonNumber"
  case _: (TvShowId, TvSeasonNumber, TvEpisodeNumber) => "EpisodeNumber"
  case _: VideogameId => "VideogameId"
  case _: BookId => "BookId"
}

implicit val listMediaUnionSchema2: Schema[MovieId | TvShowId | (TvShowId, TvSeasonNumber) | (TvShowId, TvSeasonNumber, TvEpisodeNumber) | VideogameId | BookId] =
  Schema.oneOfUsingField(MovieId | TvShowId | (TvShowId, TvSeasonNumber) | (TvShowId, TvSeasonNumber, TvEpisodeNumber) | VideogameId | BookId, String)(
    extractDiscriminator2,
    identity
  )(
    "MovieId" -> Schema.derived[MovieId],
    "TvShowId" -> Schema.derived[TvShowId],
    "SeasonNumber" -> Schema.derived[(TvShowId, TvSeasonNumber)],
    "EpisodeNumber" -> Schema.derived[(TvShowId, TvSeasonNumber, TvEpisodeNumber)],
    "VideogameId" -> Schema.derived[VideogameId],
    "BookId" -> Schema.derived[BookId]
  )
```

Figura 27. Definición del *Schema* *listMediaUnionSchema2*.

Aunque Tapir permite aprovechar estos esquemas para generar documentación en formato OpenAPI, en el presente proyecto no se ha hecho uso de esta funcionalidad. Aun así, definir estos esquemas es igualmente necesario para que Tapir pueda procesar correctamente los tipos complejos al construir los endpoints y garantizar una validación adecuada en tiempo de ejecución.

3.2.2. Implementación de la lógica de un endpoint

Una vez que se ha hecho todo lo necesario para completar la definición de un endpoint satisfactoriamente, se puede implementar su lógica. Aquí tenemos libertad para programar el comportamiento que queramos para el endpoint, siempre y cuando respetemos los parámetros de entrada y de salida definidos para este (deben ser de los mismos tipos).

En el ejemplo de la **Figura 28** se recogen todas las reseñas de la aplicación, se les aplican los filtros que se hayan especificado y se ordenan según el criterio seleccionado en la consulta. Si el código se ejecuta exitosamente, se devuelve una lista de *Review* envuelta en un objeto *IO*. Si algo falla, se devuelve un *UserError*, que en este caso puede ser, o bien *BadRequest* o bien *Unknown*.

```
val getAllReviews: ((Option[String], Option[List[String]])) => IO[Either[UserError, List[Review]]] = {
  (sortByOption, categoryOption) => IO.pure {
    val reviews = ReviewRepository.getAll

    val filteredReviews = categoryOption match {
      case None => reviews
      case Some(categories) =>
        reviews.filter(
          review => review.reviewedMediaId match {
            case _: MovieId => categories.contains("movie")
            case _: TvShowId => categories.contains("tv_show")
            case (_: TvShowId, _: TvSeasonNumber) => categories.contains("season")
            case (_: TvShowId, _: TvSeasonNumber, _: TvEpisodeNumber) => categories.contains("episode")
            case _: VideogameId => categories.contains("videogame")
            case _: BookId => categories.contains("book")
          }
        )

    val sortedReviews = sortByOption match {
      case Some("least_liked") => Right(filteredReviews.sortBy(_.likesIds.size))
      case Some("most_liked") => Right(filteredReviews.sortBy(_.likesIds.size).reverse)
      case Some("least_replied") => Right(filteredReviews.sortBy(_.repliesIds.size))
      case Some("most_replied") => Right(filteredReviews.sortBy(_.repliesIds.size).reverse)
      case Some(unknown) => Left(BadRequest(s"Invalid sorting parameter: $unknown"))
      case None => Right(filteredReviews)
    }

    sortedReviews
  }
}.handleError {
  case ex: Exception => Left(Unknown(500, s"Unexpected error: ${ex.getMessage}"))
}
}
```

Figura 28. Implementación de la lógica del endpoint *getAllReviews*.

El tipo *IO*, que aparece tanto en esta como en otras partes del proyecto, es una herramienta clave en la gestión de efectos en aplicaciones funcionales (Typelevel, s.f. -c). En concreto, un valor de tipo *IO[A]* representa una descripción pura de una operación que, cuando se ejecute, devolverá un valor de tipo *A*, y cuya ejecución podría implicar efectos secundarios (como leer datos, escribir en una base de datos o gestionar errores).

Este tipo permite encapsular la descripción de estas operaciones de forma segura y controlada. Las funciones que devuelven un *IO* no ejecutan nada directamente, sino que simplemente describen lo que se quiere hacer. La ejecución real, que es impura, se pospone hasta que el programa lo indique explícitamente, normalmente en el punto de entrada principal, como en una implementación de *IOApp*.

Por ejemplo, en nuestro caso, el método *getAllReviews* devuelve un *IO[Either[UserError, List[Review]]]*. Esto significa que, al ejecutarlo, obtendremos o bien un error (por ejemplo, si el parámetro de ordenación no es válido), o bien una lista de reseñas ya filtradas y ordenadas. Mientras ese *IO* no se ejecute, no se ha realizado ningún trabajo.

Una de las principales ventajas de usar *IO* es que permite componer operaciones de manera monádica y gestionar los errores de forma limpia. Por ejemplo, mediante combinadores como *handleErrorWith* o *attempt*. Además, favorece un código más modular y predecible, evitando la aparición de efectos secundarios inesperados fuera del contexto de *IO*.

En resumen, *IO* nos sirve para trabajar con operaciones que pueden fallar o tener efectos secundarios, sin renunciar a las ventajas de la programación funcional, como la claridad, la modularidad o la facilidad para razonar sobre el comportamiento del programa.

3.2.3. Exposición de un endpoint

Una vez implementada la lógica deseada para el endpoint definido, lo único que queda es asociarlos. A lo que se muestra en la **Figura 29**, lo llamamos “ruta”, y con ello le estaríamos indicando a la aplicación cuál es el código que tiene que ejecutar cada vez que reciba una petición al endpoint.

```
private val getAllReviews: HttpRoutes[IO] =  
  Http4sServerInterpreter[IO]()  
    .toRoutes(ReviewsEndpoints.getAllReviews.serverLogic(ReviewsLogics.getAllReviews))
```

Figura 29. Definición de la ruta que asocia el endpoint *getAllReviews* con la implementación de su lógica.

En este caso, estaríamos diciéndole a la aplicación que, cuando se reciba una llamada al endpoint *getAllReviews*, se debe ejecutar la función homónima.

Para hacer esta asociación se utiliza la clase *Http4sServerInterpreter*, proporcionada por la librería Tapir. Esta clase permite transformar un endpoint definido con Tapir, junto con su lógica de ejecución (especificada mediante *serverLogic(...)*), en una ruta que http4s puede entender y gestionar. Es decir, convierte una descripción abstracta de un endpoint más su lógica funcional, en una ruta ejecutable dentro de un servidor real basado en http4s. Esto permite combinar la potencia de modelado de Tapir con la infraestructura de servidor que ofrece http4s (SoftwareMill, 2024 -h).

Este proceso se apoya en la librería http4s, una herramienta diseñada para construir aplicaciones web y servicios HTTP en Scala siguiendo un enfoque puramente funcional. Uno de los motivos por los que se ha elegido esta librería para el proyecto es precisamente ese: su diseño está basado en los principios de la programación funcional pura, utilizando estructuras inmutables y controlando los efectos mediante abstracciones como *IO* de Cats Effect. Además, es completamente modular, interoperable con otras librerías funcionales del ecosistema Scala (como Circe para JSON o la propia Tapir), y utiliza fs2 como motor de *streaming* para manejar datos de forma eficiente.

Una vez que se transforma un endpoint mediante *Http4sServerInterpreter*, lo que se obtiene es una instancia de tipo *HttpRoutes[IO]*.

Este tipo representa una colección de rutas HTTP que pueden gestionar peticiones dentro del contexto funcional proporcionado por *IO*. Es decir, cada ruta es una función parcial que, dado un *Request[IO]*, puede devolver un *Response[IO]*, si la petición coincide con alguna ruta definida. Esta forma de trabajar permite definir el comportamiento del servidor de forma declarativa, segura y totalmente funcional: no se ejecuta nada directamente, sino que se describe cómo debe responderse en cada caso.

Además, las distintas rutas pueden componerse entre sí muy fácilmente mediante operadores como *<+>*, lo cual permite agruparlas por funcionalidad. Por ejemplo, en este proyecto se agrupan todas las rutas relacionadas con reseñas en una sola variable (*reviewsRoutes*, **Figura 30**), y luego se hace lo mismo con otras funcionalidades (como usuarios, actividades, etc.).

```
val reviewsRoutes: HttpRoutes[IO] =
  getAllReviews <+>
  getReview <+>
  createReview <+>
  editReview <+>
  deleteReview
```

Figura 30. Variable que recoge las rutas de todos los endpoints relacionados con las reseñas.

Finalmente, todas estas agrupaciones se combinan en una variable global que representa el conjunto completo de rutas de la aplicación (**Figura 31**).

```
val httpRoutes: HttpRoutes[IO] =
  chattingRoutes <+>
  entriesRoutes <+>
  likesRoutes <+>
  movieRoutes <+>
  tvShowRoutes <+>
  tvSeasonRoutes <+>
  tvEpisodeRoutes <+>
  videogameRoutes <+>
  bookRoutes <+>
  mediaListsRoutes <+>
  ratingsRoutes <+>
  repliesRoutes <+>
  reviewsRoutes <+>
  searchRoutes <+>
  userRoutes <+>
  userFavouritesRoutes <+>
  userNetworkRoutes <+>
  userEntriesRoutes <+>
  userLikesRoutes <+>
  userMediaListsRoutes <+>
  userRatingsRoutes <+>
  userRepliesRoutes <+>
  userReviewsRoutes <+>
  userMediaRoutes
```

Figura 31. Variable que recoge las rutas de todos los endpoints de la aplicación.

Esta variable se pasa como parámetro al *BlazeServerBuilder* que aparece en el *Main* del proyecto, que es el encargado de levantar el servidor, abrir el puerto configurado, y empezar a gestionar peticiones reales a través de todas las rutas definidas.

```
object Main extends IOApp {
  override def run(args: List[String]): IO[ExitCode] = {
    BlazeServerBuilder[IO]
      .bindHttp(8080, "localhost")
      .withHttpApp(httpRoutes.orNotFound)
      .serve
      .compile
      .drain
      .as(ExitCode.Success)
  }
}
```

Figura 32. Objeto *Main* de la aplicación.

El *Main* del proyecto es el punto de entrada principal de la aplicación, y está definido extendiendo *IOApp*, una clase que ofrece una forma sencilla de lanzar programas basados en *IO*. En este caso, la función *run* se encarga de inicializar y ejecutar el servidor HTTP utilizando el método *BlazeServerBuilder[IO]*.

BlazeServerBuilder forma parte de *http4s* y permite crear servidores HTTP de forma declarativa, funcional y segura. En concreto, *BlazeServerBuilder*:

- Permite configurar el host y puerto en el que el servidor escuchará (en este caso, *localhost* y el puerto 8080).
- Recibe como parámetro la aplicación HTTP completa, que es el conjunto de rutas definido en *httpRoutes*, convertidas a un *HttpApp[IO]* mediante *orNotFound*.
- Se encarga de gestionar las conexiones, recibir peticiones, ejecutar la lógica asociada a cada endpoint y devolver las respuestas correspondientes.
- Utiliza *Blaze* como motor HTTP subyacente, una implementación no bloqueante y eficiente que está bien integrada con el resto de la librería.
- Se ejecuta dentro del contexto funcional de *IO*, por lo que toda su operación está controlada, sin efectos inesperados ni estados compartidos fuera del modelo de efectos.

El servidor arranca al ejecutar la cadena de instrucciones mostrada en la **Figura 32**.

Aquí, *serve* devuelve un *stream* de ejecución infinita (el servidor queda escuchando indefinidamente), que se compila y drena con *compile.drain*, para finalmente devolver un código de salida correcto (*ExitCode.Success*) una vez terminado.

En resumen, *BlazeServerBuilder* es el componente que, una vez configuradas todas las rutas y lógicas de la aplicación, se encarga de convertir esa estructura en un servidor HTTP real y funcional, listo para recibir y procesar peticiones externas. Su integración con *http4s* e *IOApp* permite que el resultado sea totalmente funcional y esté alineado con los principios de diseño del resto del sistema.

En el apéndice **Instrumentación de otros endpoints** se explica cómo se ha llevado a cabo la instrumentación de otros endpoints del dominio propio que pueden resultar interesantes para entender mejor algunos de los conceptos utilizados en la aplicación en relación con esto.

3.3. Instrumentación de endpoints que se comunican con servicios externos

Al no haberse implementado un sistema de persistencia en la aplicación, una cuestión crucial a resolver era el lugar del que se iban a obtener los datos relacionados con los contenidos multimedia que el sistema usa como eje central para realizar sus operaciones.

Se han contemplado algunas opciones, como la de crear archivos de prueba con datos inventados, o bien con información real resultante de llevar a cabo un proceso de *data scrapping* respecto a los servicios que nos pueden facilitar estos datos. Sin embargo, se ha optado, finalmente, por obtener esa información necesaria en tiempo real mediante solicitudes HTTP a las APIs mencionadas en la sección **Arquitectura del sistema**.

3.3.1. Definición de un endpoint

La manera de llevar a cabo peticiones a dichas APIs es mediante endpoints definidos también con la librería Tapir, y que serán lanzados por clientes específicos para cada API. Por tanto, la definición de estos endpoints se enfoca principalmente en establecer qué necesita el servicio externo en cuestión para entender una petición y devolver la información que nuestro sistema necesita.

Por ejemplo, para recibir, desde la API de TMDb, información de una película concreta, deberíamos definir un endpoint como el de la **Figura 33**.

```
val requestMovie: PublicEndpoint[(String, MovieId), UserError, MovieFromTMDb, Any] =
  Base.movieBaseEndpoint(
    "requestMovie",
    "This endpoint returns a movie from TMDb API by its ID"
  )
  .out(jsonRequestedMovieOut)
```

Figura 33. Definición del endpoint *requestMovie*.

Como se puede observar, *requestMovie* hace uso de *movieBaseEndpoint* (**Figura 34**), que indica que los endpoints que lo utilicen tendrán, en su ruta, el *String movie* y un parámetro de ruta para indicar el identificador de la película a la que se desea acceder.

```
val movieBaseEndpoint:
  (String, String) => PublicEndpoint[(String, MovieId), UserError, Unit, Any] =
  (name, description) =>
    tmdbBaseEndpoint(name, description, "movie")
    .in(PathInputs.pathMovieId)
```

Figura 34. Definición del endpoint base *movieBaseEndpoint*.

Este endpoint trabaja sobre otro, *tmdbBaseEndpoint* (**Figura 35**), que en este caso es común para todos los endpoints que realicen solicitudes a la API de TMDB.

```
private val tmdbBaseEndpoint:
  (String, String, String) => PublicEndpoint[String, UserError, Unit, Any] =
  (name, description, path) =>
    httpMethodEndpoint(name, description, path, "GET")
      .in(queryApiKey)
```

Figura 35. Definición del endpoint base *tmdbBaseEndpoint*.

Se puede observar que recibe como parámetro de consulta la clave de acceso a la API de TMDB, y hace uso del ya mencionado *httpMethodEndpoint*, al que le indica que se trata de una solicitud de tipo GET.

Por tanto, *requestMovie*, en esencia, recibe un *String* con la clave de acceso a la API y un *MovieId* con el identificador de la película dentro de la API de TMDB, y devuelve un JSON con la información recogida de la petición mediante el objeto *MovieFromTMDB* (**Figura 36**), que no es otra cosa que un DTO (Data Transfer Object) del que se sacarán los datos necesarios para formar el objeto *Movie*, el cual sí pertenece al dominio propio de la aplicación.

```
case class MovieFromTMDB(
  adult: Boolean,
  backdrop_path: Option[String],
  belongs_to_collection: Option[Collection],
  budget: Option[Long],
  genres: List[Genre],
  homepage: String,
  id: MovieId,
  imdb_id: String,
  origin_country: Option[List[String]],
  original_language: Option[String],
  original_title: String,
  overview: Option[String],
  popularity: Option[Double],
  poster_path: Option[String],
  production_companies: List[ProductionCompany],
  production_countries: List[ProductionCountry],
  release_date: Option[String],
  revenue: Option[Long],
  runtime: Option[Long],
  spoken_languages: List[SpokenLanguage],
  status: String,
  tagline: String,
  title: String,
  video: Option[Boolean],
  vote_average: Option[Double],
  vote_count: Option[Long]
)
```

Figura 36. Clase *MovieFromTMDB*.

En cuanto a los parámetros de entrada y salida de *requestMovie*, para poder incluirlos en la definición del endpoint, hace falta crear sus correspondientes *EndpointInput* y *EndpointOutput*. El proceso es el mismo que el que se sigue en la sección **Parámetros de E/S y códecs**, así como también en **Endpoint de tipo PUT** (se cita esta sección en lo referente a la creación del códec para las conversiones entre texto plano y objeto de identificador).

3.3.2. Creación de un cliente

Antes de pasar a explicar cómo utilizar el endpoint ya definido para realizar una petición a un servicio externo, es conveniente explicar el funcionamiento del cliente creado para gestionar dichas solicitudes. En este caso, tenemos el cliente que se comunica con TMDb (Figura 37).

```
object TMDBClient {
  private val apiKey = "4c004411730409c1a39b5f11582a04b7"
  private val responseMaxSize = 1024 * 1024
  private val baseUrl = uri("https://api.themoviedb.org/3")

  def executeRequest[I, O](
    endpoint: PublicEndpoint[I, UserError, O, Any],
    resource: MovieId |
              TvShowId |
              (TvShowId, TvSeasonNumber) |
              (TvShowId, TvSeasonNumber, TvEpisodeNumber) |
              String
  ): IO[Either[UserError, O]] = {

    val httpClientResource: Resource[IO, Client[IO]] =
      EmberClientBuilder.default[IO]
        .withChunkSize(responseMaxSize)
        .withTimeout(5.seconds)
        .build

    val requestResult = (endpoint, resource) match
      case (endpoint: PublicEndpoint[(String, MovieId), .., ..], movieId: MovieId) =>
        Right(Http4sClientInterpreter[IO]().toRequest(endpoint, baseUrl = Some(baseUrl)).apply(apiKey, movieId))

      case (endpoint: PublicEndpoint[(String, TvShowId), .., ..], tvShowId: TvShowId) =>
        Right(Http4sClientInterpreter[IO]().toRequest(endpoint, baseUrl = Some(baseUrl)).apply(apiKey, tvShowId))

      case (endpoint: PublicEndpoint[(String, TvShowId, TvSeasonNumber), .., ..], (tvShowId: TvShowId, tvSeasonNumber: TvSeasonNumber)) =>
        Right(Http4sClientInterpreter[IO]().toRequest(endpoint, baseUrl = Some(baseUrl)).apply(apiKey, tvShowId, tvSeasonNumber))

      case (endpoint: PublicEndpoint[(String, TvShowId, TvSeasonNumber, TvEpisodeNumber), .., ..], (tvShowId: TvShowId, tvSeasonNumber: TvSeasonNumber, tvEpisodeNumber: TvEpisodeNumber)) =>
        Right(Http4sClientInterpreter[IO]().toRequest(endpoint, baseUrl = Some(baseUrl)).apply(apiKey, tvShowId, tvSeasonNumber, tvEpisodeNumber))

      case (endpoint: PublicEndpoint[(String, String), .., ..], title: String) =>
        Right(Http4sClientInterpreter[IO]().toRequest(endpoint, baseUrl = Some(baseUrl)).apply(apiKey, title))

      case _ => Left(BadRequest("Wrong number of parameters for specified endpoint"))

    requestResult match {
      case Left(error) => IO.pure(Left(error))

      case Right((userRequest, parseResponse)) =>
        httpClientResource.use { httpClient =>
          httpClient.run(userRequest).use { response =>
            parseResponse(response).attempt.FlatMap {
              case Right(DecodeResult.Value(Right(result))) => IO.pure(Right(result))
              case Right(DecodeResult.Value(Left(error))) => IO.pure(Left(error))
              case Right(failure: DecodeResult.Failure) => IO.pure(Left(Unknown(500, s"Failed to decode response: $failure")))
              case Left(error) => IO.pure(Left(Unknown(500, s"Failed to parse response: ${error.getMessage}"))
            }
          }.handleErrorWith { error =>
            IO.pure(Left(Unknown(500, s"Unexpected error: ${error.getMessage}"))
          }
        }
    }
  }
}
```

Figura 37. Implementación de cliente (*TMDBClient*) para realizar peticiones a la API de TMDb.

El objeto *TMDBClient* tiene tres constantes y un método. Las constantes son la clave de acceso a la API, el tamaño máximo de las respuestas que recibe el cliente, y la URL sobre la que se lanzan las peticiones, añadiendo a cada endpoint el *String* necesario para el recurso deseado. Por ejemplo, el endpoint *requestMovie* añade *movie* a la URL base, además de *movie_id* como parámetro de ruta, por lo que la solicitud se realizaría a la dirección https://api.themoviedb.org/3/movie/{movie_id}.

Por otro lado, se define el método *executeRequest*, que recibe como parámetros el endpoint de la solicitud que se quiere realizar, y el identificador del recurso que se desea obtener; y que devuelve el objeto resultante de la petición a TMDB, o un error en caso de que algo haya salido mal.

Para realizar peticiones HTTP a los servicios externos, se emplea *EmberClientBuilder*, que forma parte de la librería *http4s*. Este *builder* permite crear clientes HTTP ligeros, eficientes y fácilmente integrables en entornos funcionales basados en Cats Effect. Ember está diseñado como un backend no bloqueante, con soporte tanto para HTTP/1 como para HTTP/2, y se caracteriza por su enfoque en la programación funcional pura (*http4s*, s.f. -b).

En el método *executeRequest*, lo primero que se hace es construir el cliente HTTP mediante el fragmento mostrado en la **Figura 38**.

```
val httpClientResource: Resource[IO, Client[IO]] =  
  EmberClientBuilder.default[IO]  
    .withChunkSize(responseMaxSize)  
    .withTimeout(5.seconds)  
    .build
```

Figura 38. Definición de la configuración básica del cliente.

Aquí se especifican parámetros como el tamaño máximo de los *chunks* de respuesta y el tiempo máximo de espera por petición. Además de estas, *EmberClientBuilder* proporciona múltiples opciones de configuración adicionales, como la gestión de la conexión persistente, límites de cabecera o ajustes de socket, todas ellas documentadas en su ScalaDoc oficial (*http4s*, s.f. -c).

La construcción del cliente se encapsula en un valor de tipo *Resource*, una abstracción propia de Cats Effect diseñada para gestionar de forma segura recursos que requieren adquisición y liberación explícitas (por ejemplo, conexiones de red). Esto garantiza que el cliente se abra y cierre correctamente, incluso en caso de error o cancelación. Además, *Resource*

permite componer múltiples recursos de manera declarativa, evitando problemas clásicos como las fugas de recursos o la anidación excesiva de llamadas de retorno (Typelevel, s.f. -b).

Para poder adaptarse a la flexibilidad de los endpoints definidos con Tapir, el método *executeRequest* recurre a una estructura de *pattern matching* que garantiza la correspondencia entre el endpoint recibido y el recurso proporcionado como argumento (**Figura 39**). Esto es necesario, ya que no todos los endpoints aceptan el mismo número ni tipo de parámetros.

```
val requestResult = (endpoint, resource) match
  case (endpoint: PublicEndpoint[(String, MovieId), .., ..], movieId: MovieId) =>
    Right(Http4sClientInterpreter[IO]().toRequest(endpoint, baseUri = Some(baseUri)).apply(apiKey, movieId))

  case (endpoint: PublicEndpoint[(String, TvShowId), .., ..], tvShowId: TvShowId) =>
    Right(Http4sClientInterpreter[IO]().toRequest(endpoint, baseUri = Some(baseUri)).apply(apiKey, tvShowId))

  case (endpoint: PublicEndpoint[(String, TvShowId, TvSeasonNumber), .., ..], (tvShowId: TvShowId, tvSeasonNumber: TvSeasonNumber)) =>
    Right(Http4sClientInterpreter[IO]().toRequest(endpoint, baseUri = Some(baseUri)).apply(apiKey, tvShowId, tvSeasonNumber))

  case (endpoint: PublicEndpoint[(String, TvShowId, TvSeasonNumber, TvEpisodeNumber), .., ..], (tvShowId: TvShowId, tvSeasonNumber: TvSeasonNumber, tvEpisodeNumber: TvEpisodeNumber)) =>
    Right(Http4sClientInterpreter[IO]().toRequest(endpoint, baseUri = Some(baseUri)).apply(apiKey, tvShowId, tvSeasonNumber, tvEpisodeNumber))

  case (endpoint: PublicEndpoint[(String, String), .., ..], title: String) =>
    Right(Http4sClientInterpreter[IO]().toRequest(endpoint, baseUri = Some(baseUri)).apply(apiKey, title))

  case _ => Left(BadRequest("Wrong number of parameters for specified endpoint"))
```

Figura 39. Comprobación de coincidencia en la estructura entre los argumentos recibidos por el cliente.

Cada caso del *pattern matching* construye, si corresponde, una función que genera la solicitud HTTP a partir del endpoint. Esta conversión se realiza mediante el objeto *Http4sClientInterpreter*, proporcionado por Tapir. Este componente permite transformar una definición de endpoint en una función que, al recibir los parámetros adecuados, genera una *Request[IO]* y una función para decodificar la respuesta (**Figura 40**).

```
Right(Http4sClientInterpreter[IO]().toRequest(endpoint, baseUri = Some(baseUri)).apply(apiKey, movieId))
```

Figura 40. Función que genera la solicitud HTTP a partir del endpoint.

Este proceso de interpretación del endpoint es el que recomienda la documentación oficial de Tapir para escenarios de cliente http4s, como se describe en su sección correspondiente (SoftwareMill, s.f. -c). La propia documentación explica cómo, a partir de un endpoint y una URI base opcional, se puede obtener una función que construye la solicitud y otra que interpreta la respuesta, devolviendo un resultado de tipo *DecodeResult* que incluye tanto el valor decodificado como los posibles errores de *parsing* o validación (SoftwareMill, 2024 -i).

Si el tipo de parámetro proporcionado no coincide con el esperado por el endpoint (por ejemplo, si se proporciona un identificador de serie donde debería ir uno de película), el método devuelve directamente un error *Bad Request*, evitando así la construcción de solicitudes incorrectas.

Una vez construida correctamente la solicitud (*userRequest*) y su correspondiente función de respuesta (*parseResponse*), se utiliza el cliente HTTP para lanzar la petición, tal y como se muestra en la **Figura 41**.

```
requestResult match {  
  case Left(error) => IO.pure(Left(error))  
  
  case Right((userRequest, parseResponse)) =>  
    httpClientResource.use { httpClient =>  
      httpClient.run(userRequest).use { response =>  
        parseResponse(response).attempt.flatMap {  
          case Right(DecodeResult.Value(Right(result))) => IO.pure(Right(result))  
          case Right(DecodeResult.Value(Left(error))) => IO.pure(Left(error))  
          case Right(failure: DecodeResult.Failure) => IO.pure(Left(Unknown(500, s"Failed to decode response: $failure")))  
          case Left(error) => IO.pure(Left(Unknown(500, s"Failed to parse response: ${error.getMessage}")))  
        }  
      }.handleErrorWith { error =>  
        IO.pure(Left(Unknown(500, s"Unexpected error: ${error.getMessage}")))  
      }  
    }  
}
```

Figura 41. Lanzamiento de la petición HTTP desde el cliente.

Aquí se hace uso de *use* dos veces: primero para abrir y cerrar correctamente el cliente HTTP (*httpClient*), y después para hacer lo mismo con la respuesta (*response*). Esto garantiza que no se mantiene ninguna conexión más tiempo del necesario. Una vez obtenida la respuesta, se intenta decodificar usando *parseResponse*.

El resultado de esa operación puede ser diverso: puede tratarse de un valor decodificado correctamente, de un error de usuario que ha sido capturado por el endpoint, o bien de un fallo en la propia decodificación. Cada uno de estos casos se recoge y se encapsula dentro de un *Either[UserError, O]*, lo cual permite propagar el resultado (ya sea válido o erróneo) hacia el exterior de forma segura.

Finalmente, ante cualquier error inesperado (por ejemplo, fallo de conexión, error del servidor externo, etc.), se captura la excepción y se transforma en un error *Unknown*, con un mensaje informativo para facilitar su trazabilidad.

3.3.3. Utilización del endpoint

Por último, para aplicar todo lo visto anteriormente en esta sección, únicamente hace falta realizar una llamada a *executeRequest*, pasándole el endpoint de la petición deseada y, en este caso, el identificador del objeto que queramos obtener (también existen endpoints en los

que se puede introducir un título para buscarlo por coincidencia de texto), tal y como se muestra en la Figura 42.

```
val getMovie: MovieId => IO[Either[UserError, Movie]] =
  movieId =>
    TMDBClient.executeRequest(Movies.requestMovie, movieId).flatMap {
      case Left(error) => IO.pure(Left(error))
      case Right(requestedMovie) =>
        val similarMoviesTmdb = TMDBClient.executeRequest(Movies.requestedSimilarMovies, movieId)
          .map(_._toOption.map(_._results))
          .handleError {
            case ex: Exception =>
              println(s"Some error occurred while requesting similar movies: ${ex.getMessage}")
              None
          }

        val recommendedMoviesTmdb = TMDBClient.executeRequest(Movies.requestedRecommendedMovies, movieId)
          .map(_._toOption.map(_._results))
          .handleError {
            case ex: Exception =>
              println(s"Some error occurred while requesting recommended movies: ${ex.getMessage}")
              None
          }

        val creditsTmdb = TMDBClient.executeRequest(Movies.requestedCreditsForMovie, movieId)
          .map(_._toOption)
          .handleError {
            case ex: Exception =>
              println(s"Some error occurred while requesting credits for the movie: ${ex.getMessage}")
              None
          }

        val metrics = getMetricsForMedia(movieId)
```

Figura 42. Implementación de la lógica del endpoint *getMovie*, realizando varias solicitudes a la API de TMDB.

Después, simplemente habría que recoger el resultado de la solicitud al servicio externo y procesarlo dentro de la lógica implementada para un endpoint del dominio propio de la aplicación. En el ejemplo mostrado, lo que se está haciendo es crear un objeto *Movie* a partir de la información que TMDB tiene de la película solicitada, de sus créditos y de películas similares y recomendadas.

4. EXPERIMENTOS Y VALIDACIÓN

En este capítulo se describen una serie de pruebas diseñadas para validar el comportamiento y la funcionalidad de los distintos endpoints desarrollados en la aplicación. A través de estas pruebas, se pretende comprobar que las respuestas obtenidas se ajustan a lo esperado, tanto en situaciones estándar como en casos límite o con entradas erróneas.

Los experimentos simulan distintas interacciones que un usuario podría realizar en un recorrido típico por la aplicación, incluyendo operaciones como la creación de registros, la obtención de listados filtrados, la interacción con otros usuarios mediante respuestas, la gestión de contenido, o la búsqueda y obtención de información sobre dicho contenido (prestando atención a los tres servicios externos utilizados). Para ello, se ha hecho uso de la herramienta Postman, que permite enviar peticiones HTTP con distintos parámetros y cuerpos de entrada, así como visualizar de forma clara las respuestas obtenidas.

Las entradas y salidas de los endpoints, así como los datos de prueba, se documentan mediante capturas de pantalla o ejemplos en formato JSON, cuando sea necesario, con el fin de mostrar de forma transparente y comprensible el funcionamiento del sistema en condiciones reales. Todos estos objetos JSON se incluirán en el repositorio de GitHub mencionado, en el directorio *documentation/ExperimentosYValidacion*, de manera que se pueden ver tal cual se reciben de la aplicación, ya que aquí se mostraran acortados para facilitar la lectura.

Además, se compararán los resultados de cada operación con los datos de prueba definidos en la aplicación, con el fin de que cada operación pueda ser validada con precisión. En aquellos casos en los que el comportamiento de un endpoint dependa de factores específicos (como la existencia de ciertos datos previos o la validez del identificador proporcionado), se ha indicado y justificado debidamente.

4.1. Endpoint `createUser`

El primer endpoint que se valida es *createUser*, que permite registrar un nuevo usuario en el sistema. Esta operación representa el punto de entrada natural de cualquier persona que desee comenzar a interactuar con la aplicación, ya que proporciona las credenciales necesarias para el acceso y la posterior utilización del resto de funcionalidades disponibles.

Para su correcto funcionamiento, este endpoint requiere un cuerpo de petición en formato JSON que incluya los datos básicos del nuevo usuario (**Figura 43**). Este JSON debe

tener la estructura de la clase *UserProfile*, dado que es la única información que exista inicialmente dentro de la clase *User* creada por el endpoint.

```
{
  "username": "Diego Carrera",
  "password": "89nkj43nhj_9j22az",
  "email": "d.carrerah.2019@alumnos.urjc.es",
  "biography": "Hey there! I'm user Diego Carrera",
  "location": "Fuenlabrada, Madrid, España"
}
```

Figura 43. Cuerpo de petición introducido al endpoint *createUser*.

Aparte del cuerpo de petición, también hay que especificar la dirección del endpoint al que se quiere hacer la solicitud. La ruta de todos los endpoints de la aplicación comienza por *http://localhost:8080/* y, a partir de ahí, se le añade el resto de la ruta según el endpoint al que se desee llamar. En este caso se añade *user/7/create*, que indica que lo que se quiere hacer es crear un usuario con un identificador de valor igual a 7. En caso de que ni el nombre de usuario ni el email introducidos en el cuerpo de petición, ni el identificador introducido en la ruta se encuentran ya dentro del sistema, y siempre que este último sea válido (número mayor que 0), la aplicación creará correctamente el usuario, devolviendo el código de estado *201 Created* que se puede ver en la **Figura 44**, así como el objeto JSON correspondiente al *User* creado (**Figura 45**).

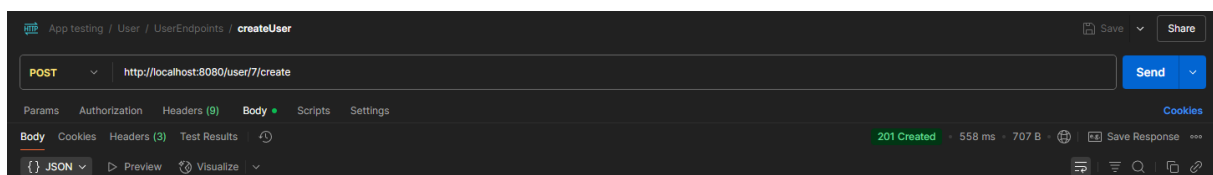


Figura 44. Llamada al endpoint *createUser* con resultado exitoso.

```
{
  "id": {
    "value": 7
  },
  "profile": {
    "username": "Diego Carrera",
    "password": "89nkj43nhj_9j22az",

```

```

    "email": "d.carrerah.2019@alumnos.urjc.es",
    "biography": "Hey there! I'm user Diego Carrera",
    "location": "Fuenlabrada, Madrid, España"
  },
  "favourites": {
    "movieId": null,
    "tvShowId": null,
    "videogameId": null,
    "bookId": null
  },
  "completedMediaIds": [],
  "pendingMediaIds": [],
  "inProgressMediaIds": [],
  "onHoldMediaIds": [],
  "droppedMediaIds": [],
  "mediaListsIds": [],
  "entriesIds": [],
  "reviewsIds": [],
  "ratingsIds": [],
  "likesIds": [],
  "repliesIds": [],
  "followingIds": [],
  "followersIds": [],
  "blockedIds": [],
  "chatsIds": [],
  "archivedChatsIds": []
}

```

Figura 45. Objeto JSON devuelto por el endpoint *createUser* con los datos del nuevo usuario.

Si, por el contrario, se diese alguno de los casos mencionados antes (ya sea por el identificador que se introduce en la ruta, o por el username o email introducidos en el cuerpo de petición tal y como se muestra en la **Figura 48** o en la **Figura 50**, respectivamente), el endpoint respondería a la petición con los códigos de estado y los mensajes que se muestran en la **Figura 46**, en la **Figura 47**, en la **Figura 49** o en la **Figura 51**.

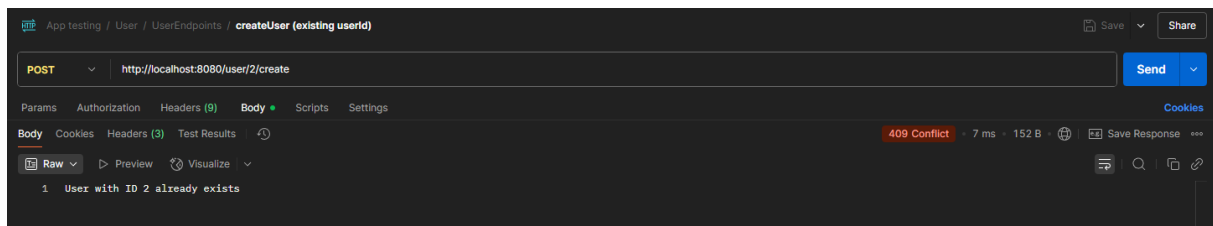


Figura 46. Resultado de ejecutar el endpoint *createUser* introduciendo un identificador de usuario que ya existe dentro de la aplicación.

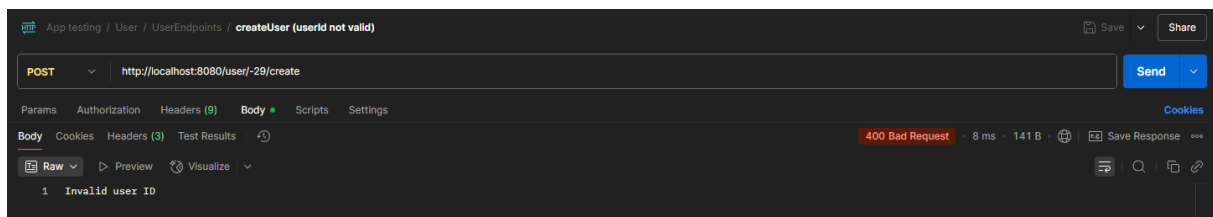


Figura 47. Resultado de ejecutar el endpoint *createUser* introduciendo un identificador de usuario no válido.

```
{
  "username": "User2",
  "password": "89nkj43nhj_9j22az",
  "email": "d.carrerah.2019@alumnos.urjc.es",
  "biography": "Hey there! I'm user Diego Carrera",
  "location": "Fuenlabrada, Madrid, España"
}
```

Figura 48. Cuerpo de petición, con un nombre de usuario existente, introducido al endpoint *createUser*.

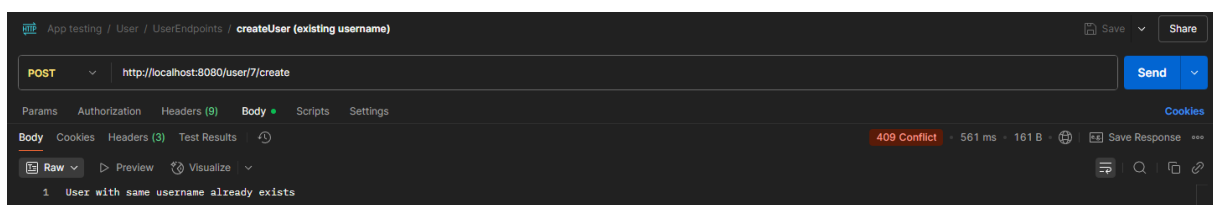


Figura 49. Resultado de ejecutar el endpoint *createUser* introduciendo un nombre de usuario que ya existe dentro de la aplicación.

```
{
  "username": "Diego Carrera",
  "password": "89nkj43nhj_9j22az",
  "email": "user4@gmail.com",
  "biography": "Hey there! I'm user Diego Carrera",
  "location": "Fuenlabrada, Madrid, España"
}
```

Figura 50. Cuerpo de petición, con un email existente, introducido al endpoint *createUser*.

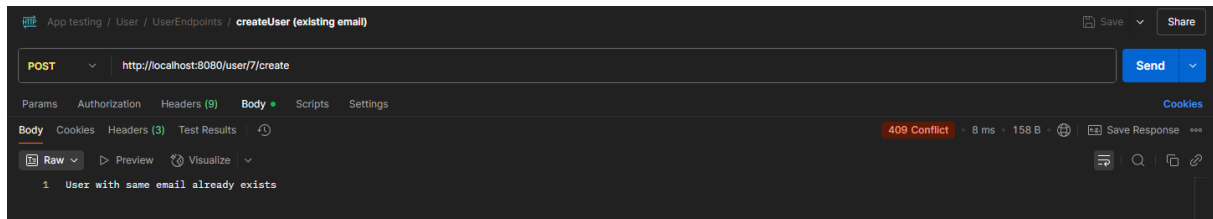


Figura 51. Resultado de ejecutar el endpoint *createUser* introduciendo un email que ya existe dentro de la aplicación.

Es posible comprobar que estas respuestas erróneas se dan debidamente comprobando los datos de prueba que han sido definidos dentro de la aplicación. En la **Figura 52**, se muestra el usuario que provoca que salten los errores de la **Figura 46** y de la **Figura 49**. Por otra parte, en la **Figura 53**, se puede observar que ya existe un usuario con el email introducido en la **Figura 50** y que provoca el error de la **Figura 51**.

```
val user2: User = User(  
    UserIds.userId2,  
    UserProfile(  
        "User2",  
        "kljhfsdg87ygfjsbhn",  
        "user2@gmail.com",  
        "Hey there! I'm user 2",  
        "London, UK"  
    ),  
    UserFavourites(  
        Some(MovieId(502033)),  
        Some(TvShowId(135918)),  
        Some(VideogameId(113112)),  
        Some(BookId("kHh_EAAAQBAJ"))  
    )  
)
```

Figura 52. Usuario 2 de los datos de prueba.

```

val user4: User = User(
    UserIds.userId4,
    UserProfile(
        "User4",
        "78hb7njmdsoil0",
        "user4@gmail.com",
        "Hey there! I'm user 4",
        "London, UK"
    ),
    UserFavourites(
        Some(MovieId(55347)),
        None,
        Some(VideogameId(103298)),
        None
    ),
)

```

Figura 53. Usuario 4 de los datos de prueba.

4.2. Endpoint getAllEntries

En la sección de **Análisis funcional**, se mencionaba que, desde la página principal, se pueden llevar a cabo distintas acciones. Una de ellas es la de registrarse y la acabamos de ver contemplada en el endpoint *createUser*. Otra de las cosas que se pueden hacer desde esa hipotética pantalla es la de ver una selección con los registros más recientes de la aplicación.

Esta funcionalidad se consigue mediante el endpoint *getAllEntries*, que nos devuelve una colección con todos los objetos *Entry* de la aplicación. Para este caso, a la petición realizada se le aplican dos parámetros de consulta: uno para ordenar los elementos de la lista obtenida de más recientes a más antiguos (*sort_by*), y otro para filtrar los registros de manera que solo se muestren los que hacen referencia a películas o libros (*categories*).

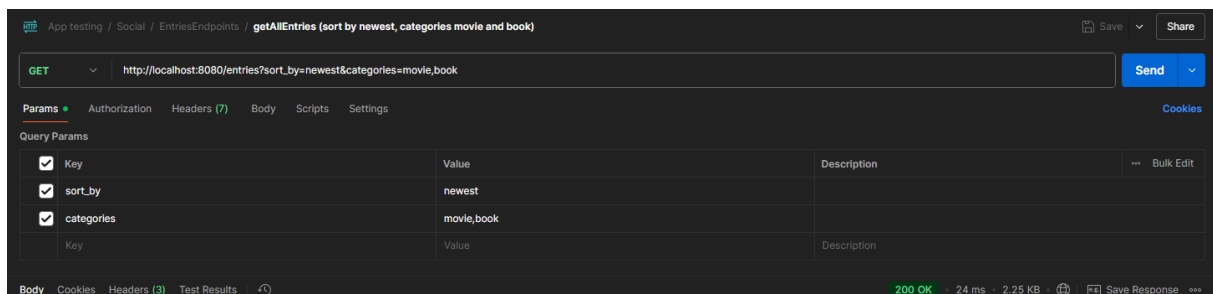


Figura 54. Petición exitosa al endpoint *getAllEntries* con parámetros *sort_by* y *categories*.

Como este endpoint, en cuanto a la lógica que implementa, es bastante sencillo, lo que realmente merece la pena comprobar no es si se han devuelto o no todos los objetos *Entry* de la

aplicación (podemos darlo por hecho), sino más bien si la lista de objetos devuelta satisface los criterios de ordenación y filtrado especificados. Para eso, hace falta observar el JSON devuelto por la aplicación, tal y como se ve en la **Figura 55**.

```
[
  {
    "id": {
      "value": 4
    },
    "userId": {
      "value": 1
    },
    "mediaId": {
      "type": "MovieId",
      "value": 502033
    },
    "ratingId": {
      "value": 4
    },
    "reviewId": {
      "value": 4
    },
    "completed": true,
    "inProgress": null,
    "onHold": null,
    "dropped": false,
    "repeat": true,
    "startedDate": null,
    "finishedDate": "2024-11-02",
    "platform": null,
    "creationDate": "2024-11-02T23:51:20"
  },
  {
    "id": {
      "value": 10
    },
    "userId": {
      "value": 1
    },
  },
]
```

```

    "mediaId": {
      "type": "BookId",
      "value": "ZefWEAAAQBAJ"
    },
    "ratingId": {
      "value": 10
    },
    "reviewId": null,
    "completed": true,
    "inProgress": false,
    "onHold": false,
    "dropped": false,
    "repeat": false,
    "startDate": "2024-10-19",
    "finishedDate": "2024-10-24",
    "platform": null,
    "creationDate": "2024-10-24T13:20:39"
  },
  ...
]

```

Figura 55. Respuesta del endpoint *getAllEntries* mostrando solo objetos *Entry* que hacen referencia a películas y libros, ordenados de más recientes a menos.

4.3. Endpoint *searchBook*

Otra funcionalidad de la aplicación que se encontraría en la pantalla principal es la de buscar elementos de la aplicación. Estos elementos pueden ser: contenidos multimedia (películas, series, libros o videojuegos), listas de contenidos o usuarios.

La herramienta de búsqueda desbloquea una gran variedad de posibilidades, permitiendo el posterior acceso a cada uno de estos tipos de elementos, y todas las acciones que un usuario puede llevar a cabo a partir de ellos. Esta sección, concretamente, se centra en el endpoint *searchBook*, que representa un ejemplo típico del funcionamiento de la herramienta. Lo interesante aquí no es solo la funcionalidad descrita, sino la forma que tiene el sistema de alcanzarla (realiza una petición HTTP a la API de Google Books). Esto aplica no solo a otros endpoints que usan este servicio, como *getBook*, sino también a endpoints que realizan la misma tarea, pero contenidos multimedia de distinto tipo, para lo que se deberán conectar a otras APIs.

Por último, antes de empezar con esta fase de los experimentos, es oportuno señalar que, dado que no existen datos de prueba sobre contenidos multimedia dentro de la aplicación, no existe una manera completamente precisa de validar los endpoints que se comunican con servicios externos (o no al menos prestando atención únicamente al tipo de respuesta recibida). Por este motivo, la verificación se limitará a comprobar el correcto funcionamiento de dichos endpoints en función del formato de la respuesta y el código de estado HTTP devueltos. Un correcto funcionamiento implicaría la correcta decodificación de un objeto JSON, recibido desde una API, en el DTO definido para recoger sus datos, y la posterior codificación de un objeto que combina datos de la API con datos de la aplicación, en el JSON que se le devuelve finalmente al usuario. Es este proceso el más propenso a errores en tiempo de ejecución, debido a que el DTO puede no contemplar la existencia o la opcionalidad de ciertos valores devueltos por la API. Es decir, si un DTO, como el de la **Figura 36**, contiene un campo que no se ha marcado como opcional, y ese campo no existe en el objeto JSON devuelto, se devolverá un mensaje de error debido a la imposibilidad de decodificar correctamente la información recibida.

Hecha esta aclaración, procedemos con la prueba que se quiere realizar del endpoint en cuestión. Para ello, ha de hacerse una llamada al endpoint *searchBook* con una dirección como la que se muestra en la **Figura 56**. Su correcto funcionamiento se demuestra por sí mismo al recibir un objeto JSON válido como respuesta, como el de la **Figura 57**, pues, al contener información de múltiples libros, se pone a prueba la robustez de la aplicación para decodificar valores que puedan ser opcionales.

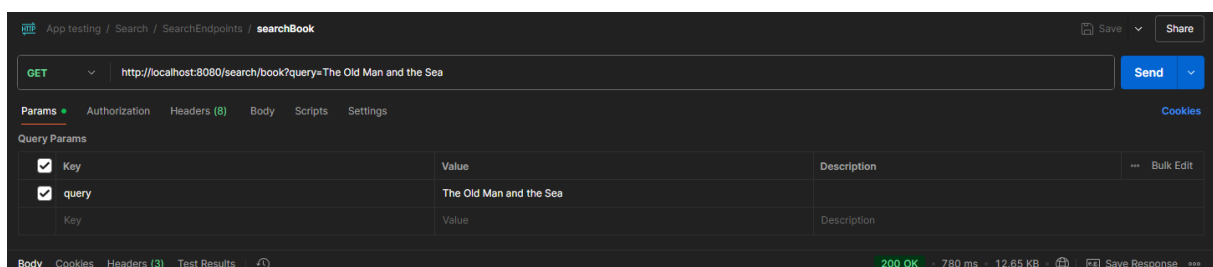


Figura 56. Petición al endpoint *searchBook*.

```
[
  {
    "kind": "books#volume",
    "id": "5SIAEAAAQBAJ",
    "etag": "1NMGEcFOVZQ",
    "selfLink": "https://www.googleapis.com/books/v1/volumes/5SIAEAAAQBAJ",
```



```

    "volumeInfo": {
      "title": "El viejo y el mar",
      "subtitle": null,
      "authors": [
        "Ernest Hemingway"
      ],
      "publisher": "LUMEN",
      "publishedDate": "2020-10-01",
      "description": "Una de las historias más grandes jamás contadas. En esta nueva y magistral traducción de Miguel Temprano García, El viejo y el mar recobra todo el esplendor del clásico imperecedero que..."
    }
  },
  {
    "kind": "books#volume",
    "id": "vV23SgAACAAJ",
    "etag": "+05bzHLxnk0",
    "selfLink": "https://www.googleapis.com/books/v1/volumes/vV23SgAACAAJ",
    "volumeInfo": {
      "title": "El viejo y el mar",
      "subtitle": null,
      "authors": [
        "Ernest Hemingway",
        "Miguel Temprano García"
      ],
      "publisher": null,
      "publishedDate": "2011-06",
      "description": "'El viejo y el mar' narra la historia de un viejo pescador cubano a quien la suerte parece haber abandonado, y..."
    }
  },
  ...
]

```

Figura 57. Objeto JSON devuelto por la aplicación al llamar al endpoint *searchBook* para buscar el libro *The Old Man and The Sea*.

4.4. Endpoint deleteBook

Para finalizar el capítulo de experimentos, se prueba un endpoint de tipo DELETE, pues ya se han visto endpoints de tipo GET, de tipo POST (los de tipo PUT son bastante parecidos), y endpoints en los que, internamente, se lanzan solicitudes a APIs externas.

El endpoint *deleteBook* elimina el contenido multimedia especificado de la lista de contenidos marcados con un determinado progreso que se especifica en la URL de la petición. Este endpoint, cuya ejecución se muestra en la **Figura 58**, funciona cuando el objeto referenciado en la URL sí que ha sido añadido previamente a la colección relativa al estado de progreso señalado, a diferencia de lo que sucedería con un endpoint de tipo POST como el que se ha probado en la sección **Endpoint *createUser***. Si se diese el caso de haber incluido en la URL el identificador de un libro que no existe dentro de esa lista de contenidos, el usuario obtendría el error *Bad Request* que se ilustra en la **Figura 59**.

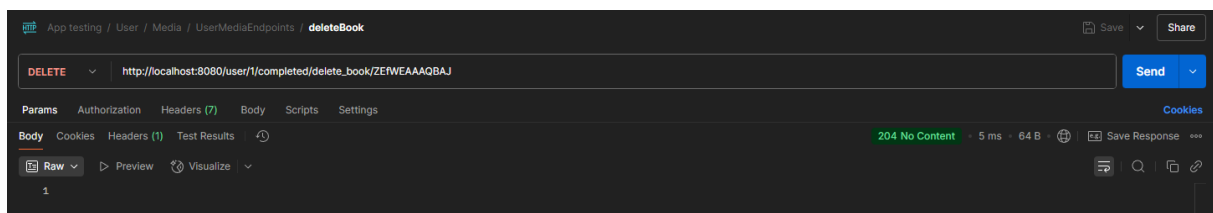


Figura 58. Petición al endpoint *deleteBook* para un libro que ya ha sido marcado previamente con ese estado de progreso.

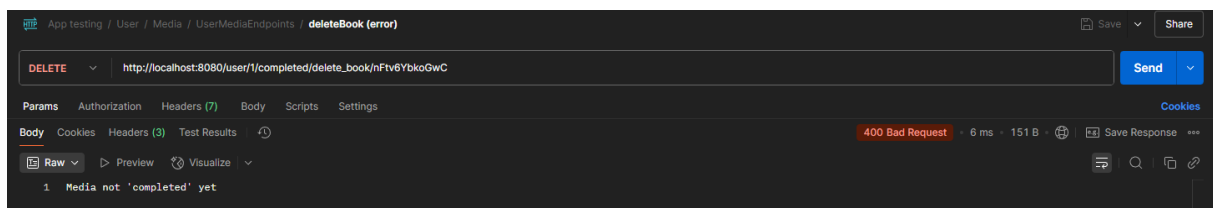


Figura 59. Petición al endpoint *deleteBook* para un libro que no ha sido marcado previamente con ese estado de progreso.

Debido al gran número de endpoints que se pueden probar, se ha decidido realizar algunas pruebas más en el apéndice de **Experimentos adicionales**, donde se recogen algunos de los casos más relevantes.

5. CONCLUSIONES

En el presente Trabajo de Fin de Grado se ha llevado a cabo el desarrollo del Back-End de una aplicación web cuyo objetivo es permitir a los usuarios realizar un seguimiento y valoración de distintos tipos de contenidos multimedia, como películas, series, videojuegos y libros, así como interactuar con otros usuarios en torno a ellos. En este capítulo se recogen las conclusiones generales del proyecto, estructurándolas de acuerdo con los objetivos marcados inicialmente, así como una reflexión sobre los problemas encontrados, el conocimiento adquirido, lo que ha podido ser reutilizado de la carrera y las posibles líneas de trabajo futuro.

El desarrollo del proyecto se ha estructurado en torno a tres grandes objetivos que fueron definidos en el capítulo **Objetivos**. A continuación, se detalla de manera resumida lo alcanzado en cada uno de ellos:

- **Análisis del sistema (Análisis y arquitectura del sistema).** Se ha llevado a cabo una descripción detallada de las distintas funcionalidades que debía ofrecer el sistema, realizando un análisis exhaustivo del modelo de dominio necesario para cubrir las necesidades planteadas. Este análisis incluyó tanto la identificación de las entidades principales como la definición de los casos de uso que deberían ser soportados por la aplicación. Además, se ha realizado un estudio de la arquitectura de la aplicación final, comparándola con la propuesta inicial que se había diseñado en fases previas al inicio del proyecto. Esta comparación ha permitido constatar cómo las decisiones técnicas y funcionales tomadas durante el desarrollo han influido en la estructura definitiva del sistema.
- **Instrumentación de endpoints del dominio propio.** Se han definido, implementado y expuesto, utilizando la librería Tapir, todos los endpoints necesarios para permitir a los usuarios interactuar con el sistema y llevar a cabo las funcionalidades previstas. Estos endpoints abarcan aspectos como la gestión de usuarios, reseñas, puntuaciones, registros de visionado o lectura, creación de listas personalizadas, marcación de *me gusta* y realización de comentarios. Cada uno de estos elementos está estrechamente vinculado con el modelo de dominio previamente analizado y definido.
- **Instrumentación de endpoints que se comunican con servicios externos.** Dada la ausencia de un sistema de persistencia en esta fase del desarrollo, se ha hecho necesario integrar la aplicación con servicios externos que proporcionan información sobre los contenidos multimedia. Para ello, se han definido e implementado endpoints que

permiten interactuar con tres APIs públicas: TMDb, IGDB y Google Books. Estos endpoints funcionan como clientes que lanzan peticiones HTTP hacia dichas APIs, procesando y adaptando las respuestas para integrarlas en el flujo de la aplicación.

Aunque los objetivos se han alcanzado de manera satisfactoria, a lo largo del desarrollo del proyecto se han presentado diversos desafíos técnicos y conceptuales que han requerido un proceso continuo de aprendizaje y adaptación:

- Complejidad del lenguaje Scala. El uso de Scala, aunque potente y versátil, ha supuesto una dificultad añadida debido a su menor popularidad respecto a otros lenguajes de programación. Esto se ha traducido en una comunidad más reducida y, en consecuencia, en una menor cantidad de recursos y ejemplos prácticos accesibles, lo que ha dificultado en algunos momentos la resolución de ciertos problemas.
- Curva de aprendizaje de la librería Tapir. La utilización de Tapir ha supuesto un reto importante, especialmente al principio del proyecto. Aunque la librería ofrece una gran cantidad de funcionalidades y una vez comprendida resulta bastante intuitiva, la escasez de ejemplos claros y específicos en la documentación oficial y en otros sitios web ha complicado la implementación de algunas partes del proyecto, especialmente en lo relativo a personalización de cuerpos de petición y de respuestas.
- Gestión de *encoders*, *decoders* y *schemas*. En relación con lo anterior, uno de los principales obstáculos ha sido comprender y gestionar adecuadamente los mecanismos de codificación y decodificación de datos. Inicialmente, se produjeron numerosos problemas derivados de una definición inadecuada de los tipos de datos y de la forma de mapear estos a estructuras JSON. Este problema fue especialmente evidente al trabajar con tipos complejos o uniones de tipos, lo que llevó a repensar y reorganizar parte de las *case classes* del dominio para simplificar el proceso de derivación automática de *encoders*, *decoders* y *schemas*.
- Cambios en las APIs externas. En el caso concreto de IGDB, se han producido modificaciones en los nombres de algunos campos y en la estructura de los datos devueltos por la API a lo largo del desarrollo. Estos cambios obligaron a realizar ajustes en el código para adaptar la estructura de las *case classes* correspondientes y asegurar la correcta recepción y tratamiento de las respuestas.

Además del aprendizaje que ha sido necesario a consecuencia de las dificultades mencionadas, también ha sido necesario instruirse en otras áreas que, aunque no hayan ocasionado grandes problemas, resultaban desconocidos en el punto de partida:

- Creación y utilización de clientes HTTP en Scala para interactuar con APIs externas.
- Levantamiento y configuración de un servidor HTTP en Scala, permitiendo la ejecución y prueba de los endpoints desarrollados.

A pesar de las dificultades y aprendizajes nuevos, también ha sido posible aplicar y reutilizar conocimientos adquiridos a lo largo del Grado, entre los que destacan:

- La capacidad para realizar un análisis detallado de un sistema, incluyendo la identificación de su modelo de dominio y la definición de casos de uso.
- La experiencia previa en diseño de arquitecturas de software, facilitando así la elección y justificación de la arquitectura adoptada.
- Los conocimientos adquiridos sobre el funcionamiento de APIs, el protocolo HTTP y los distintos tipos de solicitudes HTTP que se pueden formular y cómo hacerlo.
- Conceptos fundamentales de programación funcional y técnicas vistas previamente en Scala, como el uso de *pattern matching* o la definición de *case classes*.

5.1. Líneas futuras

El proyecto, aunque funcional en su estado actual, presenta diversas posibilidades de ampliación y mejora que podrían abordarse en futuros trabajos:

- Implementación de una capa de persistencia. Añadir una base de datos que permita almacenar de manera permanente la información generada por los usuarios (usuarios registrados, reseñas, puntuaciones, etc.).
- Desarrollo de un Front-End. Crear una interfaz gráfica de usuario que permita a cualquier persona interactuar de forma sencilla y visual con la aplicación, mejorando así la experiencia de uso.
- Generación de documentación OpenAPI o Swagger. Aprovechar las capacidades de la propia librería Tapir para generar documentación interactiva de los endpoints desarrollados, lo cual facilitaría la integración con clientes externos y la comprensión por parte de desarrolladores terceros.

- Sistema de autenticación y autorización. Implementar un mecanismo de autenticación que permita validar la identidad de los usuarios y proteger ciertos endpoints de accesos no autorizados.
- Mejora del manejo de errores y validación de entradas. Ampliar la gestión de errores para proporcionar respuestas más detalladas y personalizadas según el tipo de fallo, así como implementar una validación más exhaustiva de los parámetros de entrada.

En definitiva, el proyecto ha cumplido los objetivos planteados inicialmente, aportando una solución funcional y extensible para la gestión de interacciones de usuarios en torno a contenidos multimedia, pero a su vez abre la puerta a futuras mejoras que permitirían dotar a la aplicación de un mayor nivel de robustez, usabilidad y escalabilidad.

BIBLIOGRAFÍA

Backloggd. (19 de abril de 2024). *About Backloggd*. Obtenido de Backloggd - A Video Game Collection Tracker: <https://backloggd.com/about/>

Goodreads. (s.f.). *About Goodreads*. Obtenido de Goodreads: <https://www.goodreads.com/about/us>

http4s. (s.f. -a). *Quick Start*. Obtenido de http4s: <https://http4s.org/v0.23/docs/quickstart.html>

http4s. (s.f. -b). *Integrations*. Obtenido de http4s: <https://http4s.org/v0.23/docs/integrations.html>

http4s. (s.f. -c). *EmberClientBuilder*. Obtenido de javadoc.io: https://javadoc.io/doc/org.http4s/http4s-ember-client_3/latest/org/http4s/ember/client/EmberClientBuilder.html

Lightbend. (s.f.). *Akka HTTP*. Obtenido de Akka: <https://doc.akka.io/libraries/akka-http/current/introduction.html>

Metacritic. (s.f.). *About Us*. Obtenido de Metacritic: <https://www.metacritic.com/about-us/>

Play Framework Team. (s.f.). *Play Framework Documentation*. Obtenido de Play Framework: <https://www.playframework.com/documentation/3.0.x/Home>

Scala. (s.f.). *Learn Scala*. Obtenido de Scala Documentation: <https://docs.scala-lang.org/>

Serializd. (s.f.). *FAQs*. Obtenido de Serializd: <https://www.serializd.com/about>

Simkl. (s.f.). *Overview of Simkl*. Obtenido de How to Use Simkle: <https://docs.simkl.org/how-to-use-simkl/core-features/overview-of-simkl>

SoftwareMill. (2024 -a). *tapir - tapir 1.x documentation*. Obtenido de SoftwareMill: <https://tapir.softwaremill.com/en/latest/>

SoftwareMill. (2024 -b). *Basics - tapir 1.x documentation*. Obtenido de SoftwareMill: <https://tapir.softwaremill.com/en/latest/endpoint/basics.html>

SoftwareMill. (2024 -c). *One-of variants - tapir 1.x documentation*. Obtenido de SoftwareMill: <https://tapir.softwaremill.com/en/latest/endpoint/oneof.html>

SoftwareMill. (2024 -d). *Inputs/outputs - tapir 1.x documentation*. Obtenido de SoftwareMill: <https://tapir.softwaremill.com/en/latest/endpoint/ios.html>

SoftwareMill. (2024 -e). *Codecs - tapir 1.x documentation* . Obtenido de SoftwareMill: <https://tapir.softwaremill.com/en/latest/endpoint/codecs.html>

SoftwareMill. (2024 -f). *Schema derivation - tapir 1.x documentation*. Obtenido de SoftwareMill: <https://tapir.softwaremill.com/en/latest/endpoint/schemas.html>

SoftwareMill. (2024 -g). *Adding support for custom types - tapir 1.x documentation*. Obtenido de SoftwareMill: <https://tapir.softwaremill.com/en/latest/endpoint/customtypes.html>

SoftwareMill. (2024 -h). *Running as an http4s server - tapir 1.x documentation*. Obtenido de SoftwareMill: <https://tapir.softwaremill.com/en/latest/server/http4s.html>

SoftwareMill. (2024 -i). *Using as an http4s client - tapir 1.x documentation*. Obtenido de SoftwareMill: <https://tapir.softwaremill.com/en/latest/client/http4s.html>

SoftwareMill. (s.f.). *softwaremill/tapir: Rapid development of self-documenting APIs*. Obtenido de GitHub: <https://github.com/softwaremill/tapir>

Typelevel. (s.f. -a). *Custom Codecs*. Obtenido de GitHub Pages: <https://circe.github.io/circe/codecs/custom-codecs.html>

Typelevel. (s.f. -b). *Resource · Cats Effect*. Obtenido de Typelevel: <https://typelevel.org/cats-effect/docs/std/resource>

Typelevel. (s.f. -c). *IO · Cats Effect*. Obtenido de Typelevel: <https://typelevel.org/cats-effect/docs/2.x/datatypes/io>

Varvarigos, E. (18 de marzo de 2024). *Introduction to Tapir*. Obtenido de Baeldung: <https://www.baeldung.com/scala/tapir>

Wikipedia. (5 de junio de 2024). *Metacritic*. Obtenido de Wikipedia, la enciclopedia libre: <https://es.wikipedia.org/wiki/Metacritic>

Wikipedia. (2025 -a). *Letterboxd*. Obtenido de Wikipedia, la enciclopedia libre: <https://es.wikipedia.org/wiki/Letterboxd>

Wikipedia. (2025 -b). *FilmAffinity*. Obtenido de Wikipedia, la enciclopedia libre: <https://es.wikipedia.org/wiki/FilmAffinity>

Wikipedia. (2025 -c). *Goodreads*. Obtenido de Wikipedia, la enciclopedia libre: <https://es.wikipedia.org/wiki/Goodreads>

APÉNDICES

1. Estado del Arte

Para aportar un poco de contexto sobre la motivación que hay detrás de este Trabajo de Fin de Grado, en este apéndice se realiza un análisis del contexto actual en el que se enmarca.

Existen numerosas plataformas en línea orientadas al seguimiento de contenido cultural y de entretenimiento, que permiten a los usuarios llevar un control de lo que ven, juegan o leen, además de descubrir nuevas obras y compartir opiniones con otros usuarios. Cada una de estas plataformas tiende a especializarse en un tipo de contenido concreto: Letterboxd y Filmaffinity en cine, Goodreads en libros, Backloggd y HowLongToBeat en videojuegos, o Serializd en series de televisión. Otras, como Simkl, ofrecen enfoques más transversales dentro del ámbito audiovisual, pero siguen sin cubrir de forma integral todos los formatos de entretenimiento.

En esta sección se analizan algunas de las plataformas más representativas en las que se ha tomado inspiración para el desarrollo de esta aplicación. La revisión permitirá identificar sus fortalezas y limitaciones, y servirá como punto de partida para justificar las decisiones de diseño y funcionalidad tomadas durante el proyecto.

1.1. Letterboxd

Letterboxd es una plataforma social en línea lanzada en 2011, diseñada específicamente para personas que desean llevar un seguimiento de las películas que han visto, descubrir nuevos títulos y compartir sus opiniones con otros usuarios (Wikipedia, 2025 -a).. Su crecimiento en popularidad durante la última década la ha convertido en una de las principales referencias dentro del ámbito de las aplicaciones de seguimiento de contenido audiovisual.

Entre sus características más destacadas se encuentran:

- **Diario de visionado:** La funcionalidad principal de Letterboxd es permitir a los usuarios registrar las películas que han visto, asociándolas a una fecha concreta. Esto facilita llevar un historial detallado del visionado a lo largo del tiempo.
- **Listas personalizadas:** Los usuarios pueden crear listas temáticas personalizadas, como por ejemplo “Mejores películas de 2024”, “Pendientes de ver” o “Películas dirigidas por mujeres”, lo que permite una organización flexible según sus intereses.

- Valoraciones y reseñas: Cada película puede ser valorada mediante un sistema de estrellas (de 0,5 a 5 estrellas) y los usuarios pueden escribir reseñas para acompañar su puntuación.
- Sistema social: Letterboxd tiene un fuerte componente social. Los usuarios pueden seguir a otros, ver sus listas, diarios y reseñas, dar *me gusta* y comentar en las publicaciones de otros miembros de la comunidad. Esto fomenta el descubrimiento de nuevas películas a través de las recomendaciones y actividades de otros usuarios.
- Exploración y descubrimiento: La plataforma ofrece diferentes herramientas para descubrir contenido nuevo, como rankings globales, tendencias de la comunidad, listas populares o recomendaciones basadas en la actividad del usuario.
- Interfaz web y aplicación móvil: Letterboxd está disponible tanto en versión web como en aplicaciones móviles (iOS y Android), ofreciendo una experiencia de usuario consistente y cuidada en todas sus plataformas.

Uno de los aspectos diferenciadores de Letterboxd es su enfoque exclusivo en el cine. A diferencia de otras plataformas, no permite el seguimiento de series, videojuegos o libros. Esta especialización le ha permitido construir una comunidad muy enfocada en la cinefilia, pero también ha limitado su ámbito de aplicación a quienes buscan una solución más generalista para el seguimiento de otros tipos de contenido.

1.2. Goodreads

Goodreads es una de las plataformas más conocidas y utilizadas a nivel mundial para el seguimiento y la recomendación de libros. Fue lanzada en 2007 y posteriormente adquirida por Amazon en 2013, lo que ha favorecido su integración con el ecosistema Kindle y el catálogo de Amazon.

Su principal objetivo es ofrecer a los usuarios un espacio donde puedan llevar un control de los libros que han leído, están leyendo o desean leer, además de facilitar la interacción social en torno a la literatura (Goodreads, s.f.; Wikipedia, 2025 -c).

Entre sus principales características se encuentran:

- Registro de lecturas: Goodreads permite a los usuarios registrar los libros que han leído, que están leyendo actualmente o que planean leer en el futuro. Este registro se organiza

a través de “estanterías virtuales” que se pueden personalizar según las preferencias del usuario.

- Valoraciones y reseñas: Cada libro puede ser valorado mediante un sistema de estrellas (de 1 a 5) y los usuarios pueden escribir reseñas más extensas en las que expresan su opinión sobre la obra.
- Sistema de recomendaciones: A partir del historial de lecturas y las valoraciones realizadas, Goodreads genera recomendaciones personalizadas de libros que podrían interesar al usuario.
- Listas y desafíos de lectura: La plataforma fomenta la gamificación de la lectura a través de funcionalidades como el Goodreads Reading Challenge, donde los usuarios se marcan un objetivo anual de libros a leer. Además, existen numerosas listas temáticas creadas por la comunidad, como “Los mejores libros de ciencia ficción” o “Novelas imprescindibles del siglo XXI”.
- Interacción social: Goodreads permite seguir a otros usuarios, comentar en sus reseñas, ver sus estanterías y recibir actualizaciones sobre su actividad lectora. Además, cuenta con un sistema de grupos y foros de discusión en torno a diferentes géneros, autores o temáticas.

Una de las principales limitaciones de Goodreads es su enfoque exclusivo en libros, sin incluir otros formatos como películas, series o videojuegos. Esta especialización lo convierte en una referencia dentro del ámbito literario, pero también ha creado una necesidad entre los usuarios que buscan una herramienta más global para gestionar el seguimiento de diferentes tipos de contenido cultural.

1.3. Backloggd

Backloggd es una plataforma lanzada en 2021 y orientada específicamente al seguimiento y descubrimiento de videojuegos. Inspirada en aplicaciones como Letterboxd y Goodreads, pero centrada en el ámbito de los videojuegos, Backloggd ha ganado popularidad dentro de la comunidad gracias a su diseño sencillo y su enfoque social.

Su principal propósito es permitir a los jugadores llevar un control de los juegos que han jugado, que están jugando o que desean jugar, además de fomentar la interacción entre usuarios con gustos similares (Backloggd, 2024).

Entre sus principales características destacan:

- Registro de juegos: Los usuarios pueden añadir videojuegos a diferentes categorías dentro de su colección personal, tales como *Playing*, *Completed*, *Backlog*, *Abandoned* o *Wishlist*. Esto permite llevar un seguimiento detallado del estado de cada juego.
- Valoraciones y reseñas: Cada juego puede ser valorado mediante un sistema de puntuación (de 0 a 5 estrellas) y se pueden escribir reseñas, tanto breves como extensas, para compartir impresiones o análisis personales.
- Listas personalizadas: Al igual que otras plataformas similares, Backlogg'd permite crear listas personalizadas, como por ejemplo “Mejores juegos de 2023”, “Juegos independientes favoritos” o “Juegos pendientes de completar”.
- Sistema social: Los usuarios pueden seguir a otros jugadores, ver sus bibliotecas de juegos, sus valoraciones, reseñas y actividad reciente. También pueden dar *me gusta* y comentar en las publicaciones de otros.
- Estadísticas y análisis: Backlogg'd ofrece algunas métricas simples sobre el tiempo dedicado a cada juego, el número total de juegos completados, pendientes, abandonados, etc. También permite generar informes básicos del progreso del usuario a lo largo del tiempo.
- Descubrimiento de juegos: La plataforma incluye rankings, tendencias y reseñas populares, lo que facilita el descubrimiento de nuevos títulos, tanto actuales como retro.

A diferencia de plataformas más generalistas, Backlogg'd se centra exclusivamente en videojuegos y no incluye otros tipos de contenido como películas, series o libros. Esta especialización le ha permitido ganarse una base de usuarios fiel dentro del sector de los videojuegos, pero también limita su alcance para aquellos usuarios que buscan un sistema unificado de seguimiento de diferentes tipos de medios culturales.

1.4. Serializd

Serializd es una plataforma relativamente reciente, lanzada en 2021, que está enfocada en el seguimiento de series de televisión. Se presenta como una solución moderna y minimalista para aquellos usuarios que buscan una alternativa a aplicaciones más complejas o sobrecargadas de funciones.

Su funcionamiento y filosofía están claramente inspirados en otras plataformas de seguimiento como Letterboxd, pero adaptada al consumo episódico de las series (Serializd, s.f.).

Entre sus características principales destacan:

- Seguimiento de series y episodios: Serializd permite a los usuarios registrar qué series están viendo, cuáles han completado y en qué episodio se encuentran actualmente. La aplicación incluye un sistema de progreso que facilita retomar el visionado justo donde el usuario lo dejó.
- Diario de visionado: Cada vez que un usuario marca un episodio como visto, este queda registrado en su historial, lo que permite llevar un diario detallado del consumo de contenido televisivo.
- Valoraciones y reseñas: Los usuarios pueden valorar tanto series completas como temporadas individuales, utilizando un sistema de puntuación de 0 a 5 estrellas. También pueden escribir reseñas o comentarios cortos sobre cada serie o temporada.
- Listas personalizadas: Serializd permite crear listas temáticas personalizadas, como por ejemplo “Series favoritas”, “Pendientes de ver” o “Mejores temporadas de 2024”, ofreciendo así una mayor flexibilidad para organizar el contenido.
- Componente social: Los usuarios pueden seguir a otros miembros, ver sus actividades recientes, valorar publicaciones, y comentar en reseñas o actualizaciones. Además, existe una sección de “Feed” donde se muestra la actividad de las personas seguidas, lo que fomenta el descubrimiento de nuevas series a través de la interacción social.
- Interfaz sencilla y moderna: Uno de los aspectos más valorados de Serializd es su diseño limpio, intuitivo y centrado en la experiencia de usuario, ofreciendo una navegación rápida y sin complicaciones.

A diferencia de otras plataformas más generalistas o complejas (como Trakt), Serializd se caracteriza por su enfoque exclusivo en series de televisión, sin incluir películas, videojuegos o libros. Esta especialización, si bien lo hace atractivo para quienes consumen principalmente series, también representa una limitación para aquellos usuarios que buscan una herramienta de seguimiento de contenido más amplia y unificada.

1.5. Filmaffinity

Filmaffinity es una plataforma española fundada en 2002 que se ha consolidado como una de las mayores bases de datos y comunidades de opinión sobre cine en el ámbito hispanohablante.

A diferencia de otras plataformas de carácter más social como Letterboxd, Filmaffinity ha mantenido un enfoque más centrado en la valoración y la consulta de información, con una menor orientación hacia la interacción entre usuarios.

Entre sus principales características destacan:

- Base de datos de películas y series: Filmaffinity ofrece un extenso catálogo de películas y series de televisión, incluyendo tanto producciones internacionales como nacionales. Cada ficha incluye información detallada como sinopsis, reparto, equipo técnico, género, país de origen y año de estreno.
- Valoraciones personales: Los usuarios pueden calificar películas y series mediante una escala de 1 a 10. Estas valoraciones permiten a la plataforma generar recomendaciones personalizadas basadas en la afinidad entre usuarios.
- Sistema de afinidad: Una de las funcionalidades distintivas de Filmaffinity es su sistema de "almas gemelas". Basado en las puntuaciones dadas, la plataforma calcula el grado de afinidad entre diferentes perfiles (Wikipedia, 2025 -b), sugiriendo películas o series que podrían gustar al usuario en función de los gustos de otros con puntuaciones similares.
- Listas y tops personalizados: Los usuarios pueden consultar rankings generales (por ejemplo, “Mejor valoradas”, “Más populares”, “Estrenos más esperados”) y también crear listas personales de películas o series favoritas.
- Consultas avanzadas: Filmaffinity permite realizar búsquedas avanzadas combinando múltiples criterios (año, país, género, director, etc.), lo que facilita a los usuarios encontrar contenido que se ajuste a sus preferencias específicas.
- Reseñas y críticas: La plataforma incluye críticas de medios profesionales y también permite a los usuarios escribir sus propias reseñas, aunque estas están menos integradas en el flujo social que en otras plataformas como Letterboxd.

A pesar de también contar con series en su catálogo, Filmaffinity sigue estando principalmente orientada al cine y carece de funcionalidades específicas para el seguimiento por temporadas o episodios. Además, no incluye otros tipos de contenido como videojuegos o libros, y su componente social es bastante limitado en comparación con otras alternativas.

1.6. Simkl

Simkl es una plataforma de seguimiento de contenidos que ofrece a los usuarios la posibilidad de gestionar y registrar su progreso en series de televisión, películas y anime (Simkl, s.f.). Lanzada en 2015, Simkl se presenta como una solución todo-en-uno para quienes consumen múltiples tipos de contenido audiovisual y desean tener un control centralizado de su historial de visionado.

Una de sus principales ventajas frente a otras plataformas más especializadas es precisamente su enfoque multiformato, lo que la convierte en una de las pocas alternativas que permiten el seguimiento tanto de series como de películas y anime en un mismo entorno.

Entre sus características más destacadas se encuentran:

- Seguimiento de series, películas y anime: Simkl permite llevar un registro detallado de los episodios vistos, las películas vistas y los animes completados. El sistema de seguimiento incluye diferentes estados como *Watching*, *Completed*, *Plan to Watch*, *On Hold*, y *Dropped*.
- Historial y diario de visionado: Cada visualización queda registrada, lo que permite al usuario consultar su historial completo de visionado, ver estadísticas de tiempo dedicado y revisar tendencias de consumo.
- Listas personalizadas: Los usuarios pueden crear listas propias para organizar su contenido de manera personalizada, como por ejemplo “Series favoritas”, “Películas pendientes” o “Animes para ver en verano”.
- Sistema de recomendaciones: Simkl genera recomendaciones personalizadas en función del historial de visionado y las valoraciones del usuario, lo que facilita el descubrimiento de nuevos títulos.
- Valoraciones y reseñas: Aunque permite calificar los contenidos, su sistema no tiene un enfoque fuerte en reseñas textuales extensas.
- Integración con otros servicios: Simkl ofrece opciones de sincronización con plataformas externas como Trakt.tv, Netflix, Crunchyroll o Hulu, lo que permite importar o actualizar el historial de visionado de forma automática.
- Extensiones y API: La plataforma dispone de una extensión para navegadores que facilita el seguimiento de lo que el usuario está viendo en tiempo real, además de ofrecer

una API pública para desarrolladores interesados en integrar Simkl en sus propias aplicaciones.

A pesar de su cobertura más amplia respecto a otros servicios especializados, Simkl presenta algunas limitaciones. No incluye el seguimiento de videojuegos ni de libros, y, además, la parte social (seguimiento de amigos, interacción, cronología de actividad) está menos desarrollada que en plataformas como Letterboxd o Backloggd.

1.7. HowLongToBeat

HowLongToBeat es una plataforma en línea lanzada en 2011 que se ha convertido en una de las principales referencias para conocer la duración estimada de los videojuegos. Su enfoque principal no es tanto la gestión de bibliotecas personales o el componente social, sino ofrecer información detallada sobre el tiempo medio que requiere completar un juego.

A pesar de ello, también incorpora ciertas funcionalidades de seguimiento y registro que han llevado a muchos usuarios a utilizarla como una herramienta complementaria a otras plataformas de gestión de videojuegos.

Entre sus principales características destacan:

- Estimación de tiempos de juego: La función central de HowLongToBeat es proporcionar estimaciones del tiempo que un jugador necesitará para completar un videojuego. La plataforma muestra diferentes métricas según el tipo de jugador:
 - *Main Story* (Historia principal)
 - *Main + Extras* (Historia + contenido opcional)
 - *Completionist* (Completista)
 - *All PlayStyles* (Promedio general)

Estas estimaciones se basan en los datos introducidos por la comunidad.

- Seguimiento de progreso personal: HowLongToBeat permite a los usuarios crear y gestionar su propio perfil de juegos, clasificándolos según estados como *Playing*, *Completed*, *Backlog*, *Retired* o *Dropped*.
- Registro de tiempos personales: Los usuarios pueden introducir el tiempo que han tardado en completar un juego, lo que contribuye a mejorar las estadísticas globales de la plataforma.

- Listas y backlog: HowLongToBeat permite a los usuarios crear y gestionar su propio backlog de juegos pendientes, así como realizar un seguimiento de los juegos que ya han terminado.
- Sistema de búsqueda y filtros: La plataforma ofrece una herramienta de búsqueda avanzada que permite filtrar juegos por duración, género, plataforma o popularidad, lo que facilita a los usuarios encontrar títulos que se ajusten al tiempo del que disponen.
- Comparativas y estadísticas personales: Los usuarios pueden visualizar estadísticas detalladas sobre el tiempo total dedicado a jugar, número de juegos completados, tiempo medio por juego, etc.

A pesar de su utilidad para calcular el tiempo necesario para completar un videojuego, HowLongToBeat tiene algunas limitaciones importantes. Por ejemplo, su componente social es bastante básico comparado con plataformas como Backloggd. Además, está centrada exclusivamente en videojuegos y no permite el seguimiento de otros tipos de contenido como películas, series o libros.

1.8. Metacritic

Metacritic es una plataforma lanzada en el año 1999 que actúa como agregador de críticas y valoraciones de distintos medios de entretenimiento, incluyendo principalmente videojuegos, películas, series de televisión, música y, anteriormente, libros (Metacritic, s.f.; Wikipedia, 2024). Su enfoque principal es ofrecer una visión global de la recepción crítica de una obra, consolidando valoraciones de medios especializados y mostrando un promedio ponderado conocido como *Metascore*.

Entre sus principales características destacan:

- *Metascore* (puntuación agregada): Cada producto (juego, película, serie, etc.) recibe un *Metascore*, que es una media ponderada basada en múltiples análisis y críticas de medios profesionales. La puntuación va de 0 a 100 y está acompañada de un código de colores para facilitar la interpretación rápida:
 - Verde (buena recepción)
 - Amarillo (recepción mixta)
 - Rojo (recepción negativa)

- Críticas profesionales: Metacritic recopila reseñas de medios especializados y ofrece un resumen breve de cada una, con un enlace a la crítica original. Esto permite a los usuarios consultar diferentes perspectivas antes de consumir un producto.
- Valoraciones de usuarios: Además de la crítica profesional, los usuarios registrados pueden dejar sus propias valoraciones, con una puntuación de 0 a 10 y comentarios opcionales. No obstante, el sistema de valoraciones de usuarios ha sido objeto de críticas por su falta de control sobre prácticas como el *review bombing* (puntuaciones masivas negativas o positivas por motivos ajenos a la calidad del producto).
- Cobertura multiplataforma: A diferencia de otras aplicaciones más especializadas, Metacritic abarca diferentes tipos de contenido cultural: videojuegos, cine, televisión, música y, en algunos casos, libros. Esto le da un enfoque más transversal en cuanto a formatos de entretenimiento.
- Rankings y tops: La plataforma publica rankings periódicos como “Mejores juegos del año”, “Películas mejor valoradas”, o “Series con mejor puntuación de la crítica”, lo que la convierte en una fuente de referencia para conocer las tendencias del mercado.
- Historial de lanzamientos: Metacritic ofrece listados de lanzamientos recientes o próximos, junto con su puntuación de crítica y de usuarios, lo que facilita a estos últimos estar al día de las novedades.

A pesar de su amplio alcance, Metacritic no está orientada al seguimiento personal de contenidos. No permite registrar qué juegos has jugado, qué películas has visto o en qué capítulo de una serie te has quedado. Tampoco tiene funciones de listas personalizadas, diarios de consumo o backlog. Su orientación es puramente informativa y comparativa a nivel de crítica profesional y de comunidad general.

1.9. Conclusión

El análisis de las plataformas existentes revela que, aunque actualmente hay numerosas aplicaciones dedicadas al seguimiento de contenidos culturales, la mayoría se enfocan en un único tipo de medio o en ámbitos muy concretos. Esto genera que los usuarios interesados en múltiples formatos (películas, series, videojuegos y libros) deban recurrir a varias herramientas diferentes, lo que dificulta una gestión unificada y eficiente de su consumo cultural.

Asimismo, se observa que muchas de estas aplicaciones carecen de una integración social profunda o de funcionalidades que faciliten el descubrimiento y la recomendación personalizada en contextos multidisciplinares.

Estos vacíos y limitaciones justifican el desarrollo de una aplicación capaz de centralizar el seguimiento de diversos tipos de contenido, combinando lo mejor de cada plataforma y añadiendo características que mejoren la experiencia del usuario de forma integral. El proyecto presentado en este Trabajo de Fin de Grado pretende cubrir esta necesidad, ofreciendo una solución moderna, intuitiva y versátil.

1.10. Sitios web de las aplicaciones analizadas

A continuación, se listan las direcciones de las páginas web de las aplicaciones analizadas en este apéndice. Las páginas, según su orden de aparición, son:

- *Letterboxd · Social Film Discovery.* <https://letterboxd.com>
- *Goodreads | Meet your next favourite book.* <https://www.goodreads.com>
- *Backloggd - A videogame collection tracker.* <https://backloggd.com>
- *Serializd | Track all things TV.* <https://serializd.com>
- *FilmAffinity.* <https://www.filmaffinity.com>
- *Simkl - Watch and Track Movies, Anime, TV Shows.* <https://simkl.com>
- *HowLongToBeat.com | Game Lengths, Backlogs and more!* <https://howlongtobeat.com>
- *Movie Reviews, TV Reviews, Game Reviews and Music Reviews - Metacritic.* <https://www.metacritic.com>

2. Sitios web oficiales de las tecnologías y herramientas utilizadas

Este apéndice tiene el propósito de listar, por orden de aparición, las páginas web oficiales de las tecnologías y herramientas que se describen en la sección **Tecnologías, herramientas y metodología**.

- *The Scala Programming Language*: <https://www.scala-lang.org/>
- *tapir – tapir 1.x documentation*: <https://tapir.softwaremill.com/en/latest/>
- *http4s*: <https://http4s.org/>
- *circe/circe: Yet another JSON library for Scala*: <https://github.com/circe/circe>
- *Cats Effect · The pure asynchronous runtime for Scala*: <https://typelevel.org/cats-effect/>
- *IntelliJ Idea: El IDE para el desarrollo profesional en Java y Kotlin*: <https://www.jetbrains.com/es-es/idea/>
- *TMDB API – Getting Started*: <https://developer.themoviedb.org/docs/getting-started>
- *IGDB: Video Game Database API*: <https://www.igdb.com/api>
- *Google Books APIs | Google for Developers*: <https://developers.google.com/books?hl=es-419>
- *Postman API Platform*: <https://www.postman.com/>
- *Presentamos ChatGPT | OpenAI*: <https://openai.com/es-ES/index/chatgpt/>
- *Git*: <https://git-scm.com/>
- *GitHub · Build and ship software on a single, collaborative platform*: <https://github.com/>

3. Otros casos de uso

En este apéndice se muestran el resto de los casos de uso que describen cómo se relacionarán el usuario y la aplicación, y que no han sido incluidos en la sección de **Análisis funcional** por no hacerlo demasiado tedioso.

En la **Figura 60** se presenta el diagrama para los casos de uso relacionados con las puntuaciones.

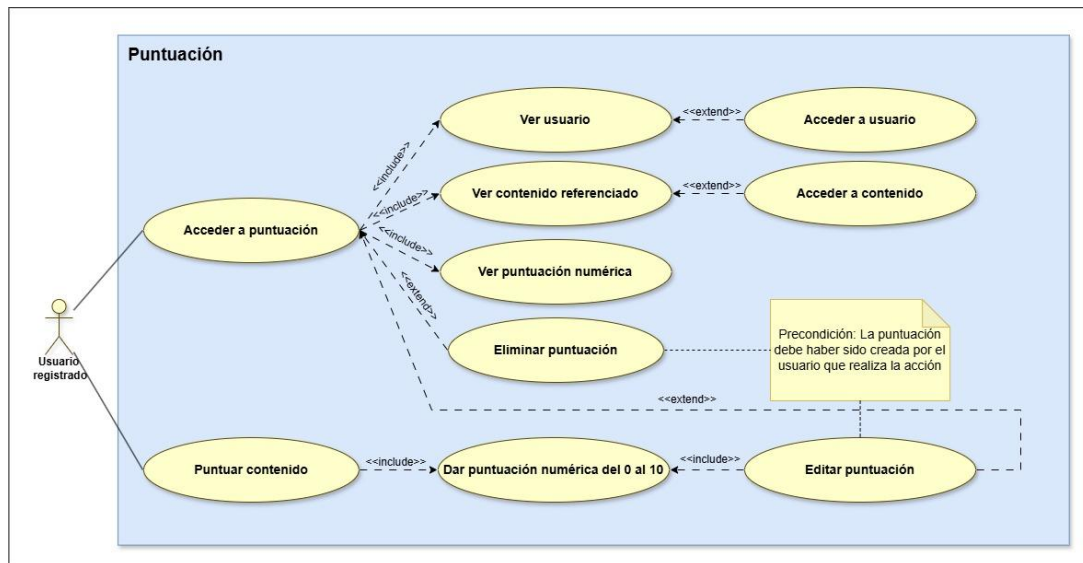


Figura 60. Diagrama de casos de uso relacionados con las puntuaciones.

Se puede ver a simple vista que la funcionalidad de las puntuaciones es muy sencilla, pues consisten solo en otorgar una puntuación entre el 0 y el 10 a cualquier tipo de contenido multimedia. Además de eso, podemos gestionar nuestras propias puntuaciones y también podemos acceder a otros elementos a través de ellas, como el usuario que la ha dado o el contenido al que se hace referencia.

De la misma manera, tenemos los casos de uso para las reseñas (**Figura 61**). Las interacciones que un usuario puede hacer respecto a las reseñas son parecidas a las que se pueden llevar a cabo con las puntuaciones, solo que estas ofrecen más posibilidades, como “dar me gusta” o “publicar un comentario” en reseñas ajenas, o “especificar si se permiten respuestas” e “indicar si hay spoilers” en reseñas propias.

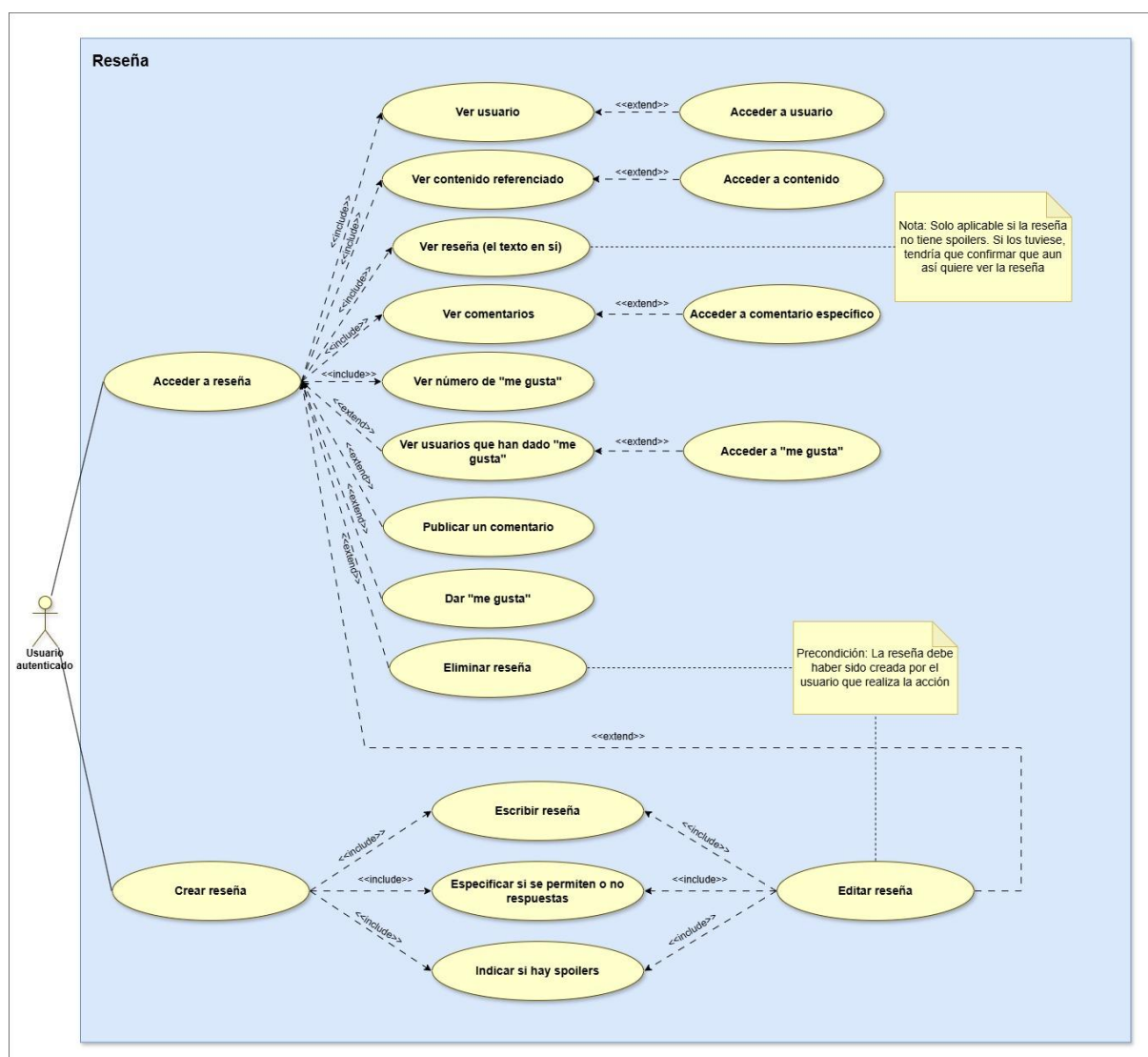


Figura 61. Diagrama de casos de uso relacionados con las reseñas.

A continuación, se muestran casos de uso para un elemento algo más elaborado, como son los registros (**Figura 62**). A rasgos generales, estos permiten indicar el estado del progreso en el que nos encontramos respecto a un contenido (pudiendo indicar más condiciones que puedan darse), así como incluir una reseña y/o una puntuación de las que hemos visto antes. Por lo demás, el carácter de las interacciones que un usuario puede llevar a cabo sigue siendo parecido al de esos otros elementos de la aplicación, excluyendo, eso sí, la posibilidad de dar *me gusta* o de responder con un comentario.

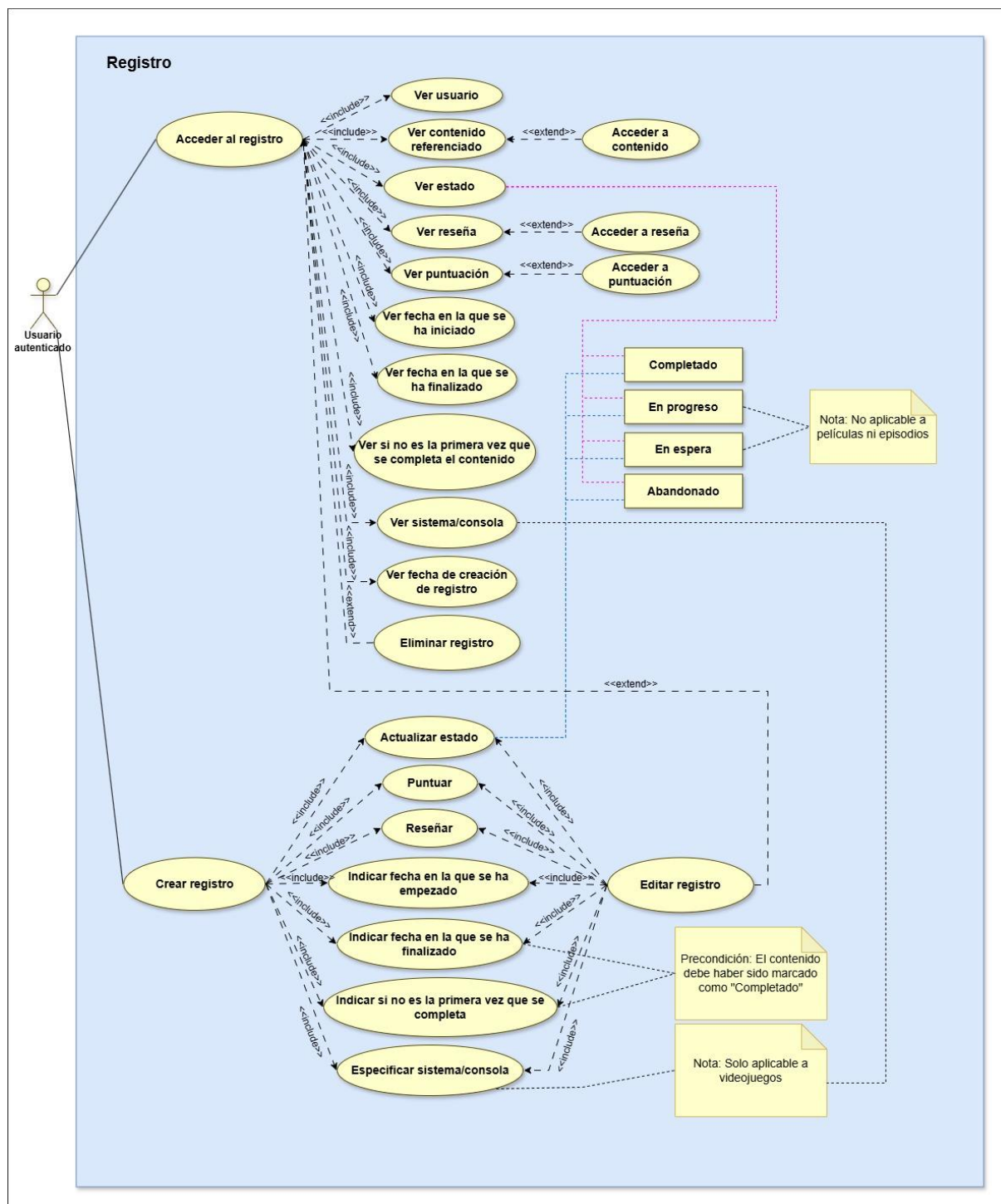


Figura 62. Diagrama de casos de uso relacionados con los registros.

Otro conjunto de interacciones que el usuario puede llevar a cabo es el que se produce en relación con las listas de contenidos multimedia que este puede ver y gestionar (**Figura 63**). Las listas ofrecen algo más de variedad en las interacciones que se pueden llevar a cabo, en cuanto a que cuentan con más información visible al usuario. El hecho de que puedan albergar muchos contenidos multimedia y de distintos tipos abre la posibilidad a realizar filtrados y ordenaciones sobre ellas a la hora de visualizarlas.

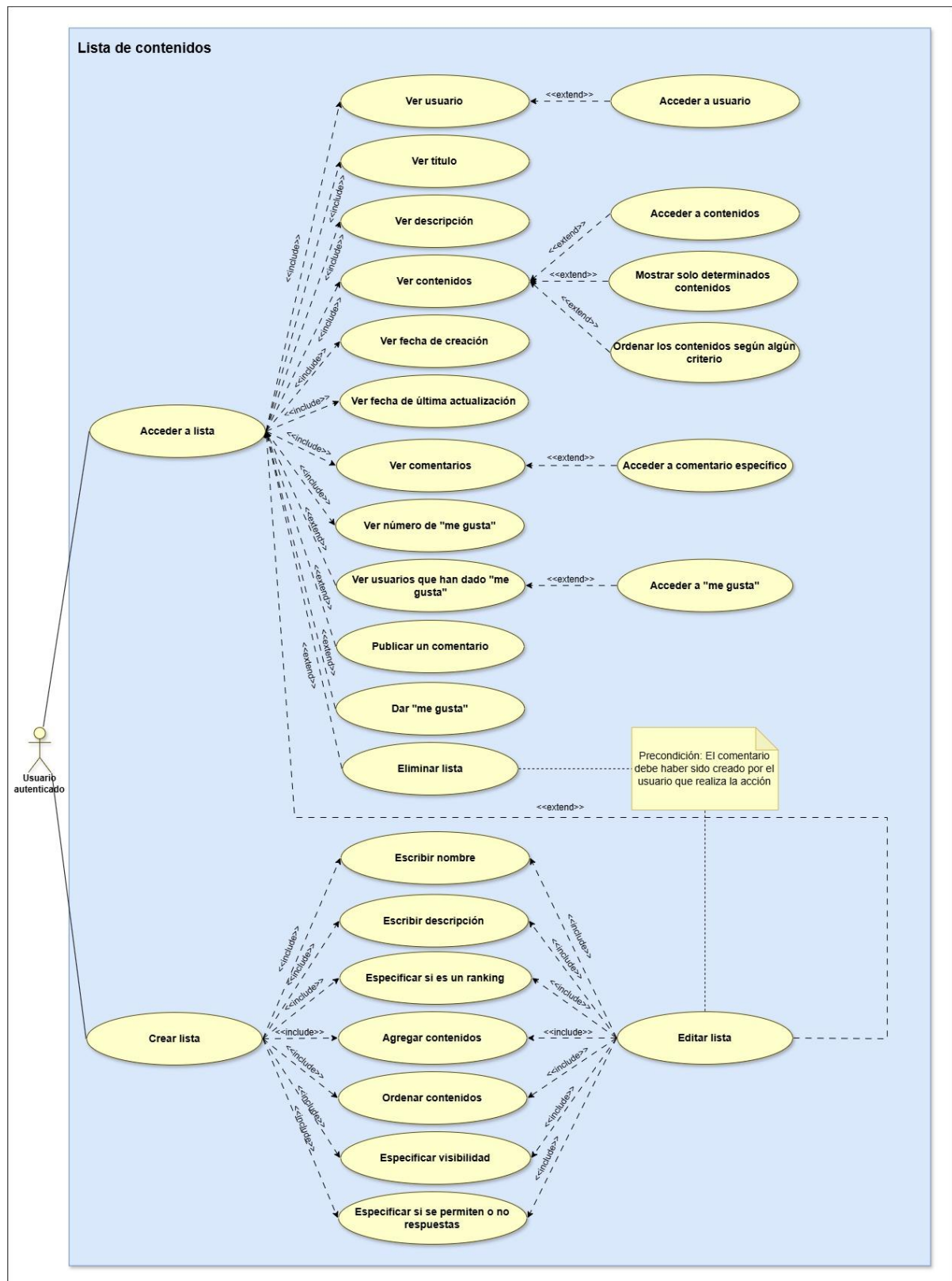


Figura 63. Diagramas de casos de uso relacionados con las listas de contenidos.

En varias ocasiones se ha mencionado la posibilidad de dar *me gusta* a algunos elementos del dominio de la aplicación, por lo que en el diagrama de la **Figura 64** se muestran las posibilidades que la aplicación ofrece respecto a esta funcionalidad.

Son esos mismos elementos sobre los cuales se pueden, además, publicar comentarios, cuyos casos de uso se muestran en la **Figura 65**.

Respecto a los objetos de este tipo, los casos de uso no distan mucho de los de algunos de los diagramas que hemos visto anteriormente. La única peculiaridad que tienen los comentarios es que permiten hacerse sobre otros comentarios, por lo que podríamos tener una cadena bastante larga de estos.

El último diagrama que queda es el que engloba a los casos de uso que se pueden realizar tomando al usuario como eje central de la acción (**Figura 66**).

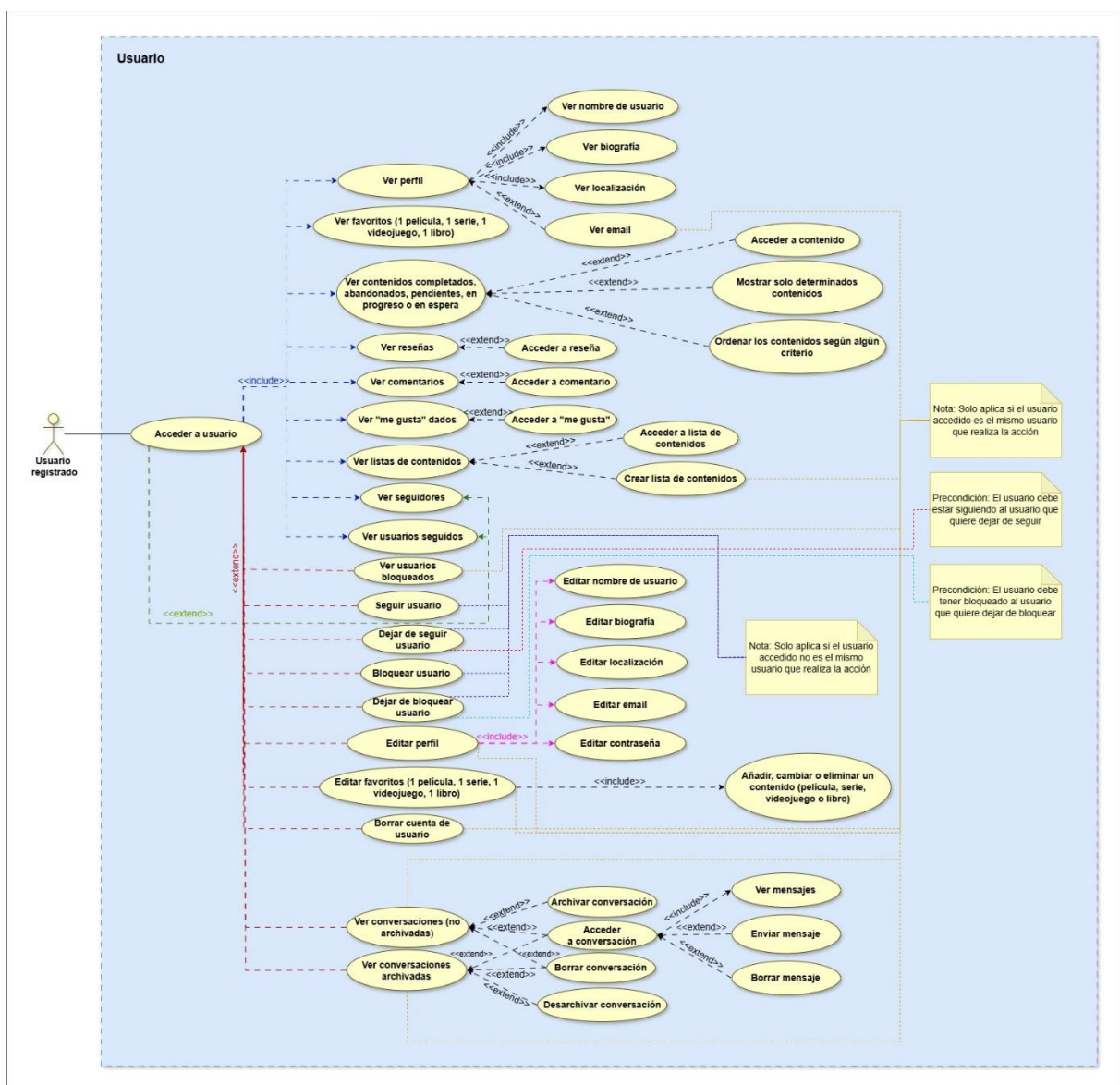


Figura 66. Diagramas de casos de uso relacionados con los usuarios.

Aquí es donde más interacciones podemos encontrar por el simple hecho de que el usuario contiene más información que el resto de los elementos de la aplicación. La mayoría de los casos de uso van enfocados a la visualización de las distintas interacciones que el usuario ha hecho respecto a los contenidos multimedia, así como respecto a otros elementos internos de la aplicación.

Además, se encuentran casos de uso que describen las interacciones directas entre usuarios. Estos son los que permiten seguir/bloquear a un usuario o dejar de seguir/bloquear a un usuario, y los que tienen que ver con las conversaciones, teniendo en cuenta que se puede acceder a estas tanto para visualizarlas como para enviar mensajes.

También hay casos de uso relacionados con la información del perfil de usuario y de los contenidos multimedia favoritos, ya sea para visualizarlos o editarlos.

4. Instrumentación de otros endpoints

Habiendo visto un endpoint bastante completo en cuanto a las características que presenta, considero interesante ver casos en los que se realicen métodos HTTP diferentes, pues en ese aspecto es en lo que puede haber más diferencias respecto al primero, principalmente. Como *getAllReviews* lleva a cabo un método HTTP de tipo GET, los endpoints que veremos a continuación serán de tipo POST, PUT, y DELETE. Se omitirán aquellas cosas que hay en común entre estos endpoints y el anterior, como la exposición.

4.1. Endpoint de tipo POST

Siguiendo con la temática de *getAllReviews*, se muestra *createReview* (**Figura 67**) para comentar aquellos aspectos distintivos de un endpoint de tipo POST. Este, permite crear un objeto *Review* en el sistema a partir de un JSON que tenga la misma estructura que este.

```
val createReview: PublicEndpoint[Review, UserError, Review, Any] =
  reviewBaseEndpoint(
    "createReview",
    "This endpoint creates a review",
    "POST"
  )
  .in("create")
  .in(JsonInputs.jsonReview)
  .out(SocialOutputs.reviewOutput)
```

Figura 67. Definición del endpoint *createReview*.

El endpoint base, en este caso (así como en los siguientes) es *reviewBaseEndpoint* (**Figura 68**; Error! No se encuentra el origen de la referencia.), que se diferencia de *reviewsBaseEndpoint*, únicamente, en la cadena de texto que se le añade a la ruta del endpoint. Como lo que se procesa es una reseña individual, la cadena de texto viene en singular para que sea más representativa de lo que se desea.

```
private val reviewBaseEndpoint:
  (String, String, String) => PublicEndpoint[Unit, UserError, Unit, Any] =
  (name, description, method) =>
    httpMethodEndpoint(name, description, "review", method)
```

Figura 68. Definición del endpoint base *reviewBaseEndpoint*.

Por debajo, este endpoint base hará lo mismo. Es decir, también utiliza el endpoint base *httpMethodEndpoint*, que, en este caso hará uso de *postEndpoint*, pues es la operación que se ha especificado en *createReview*.

```
private val postEndpoint:
  (String, String, String) => PublicEndpoint[Unit, UserError, Unit, Any] =
  (name, description, path) =>
    postBaseEndpoint(name, description, path)
      .out(statusCode(StatusCode.Created))
```

Figura 69. Definición del endpoint base *postEndpoint*.

Este endpoint únicamente define el código de estatus que se desea devolver en caso de que la solicitud se haya realizado correctamente, y utiliza el endpoint base *postBaseEndpoint* (Figura 70).

La razón de haber definido un endpoint base únicamente para definir el código de estatus deseado en un caso de éxito, es porque existe otro caso de endpoint de tipo POST que no devuelve un código de estado *Created*. Esta situación se produce en endpoints que realizan solicitudes a servicios externos. Concretamente, los que trabajan con la API de IGDB, dado que las peticiones a este servicio se formulan como métodos POST (con un cuerpo de petición en el que se pide la información deseada), pero el retorno que se obtiene es el de una petición GET (código de estado *Ok*).

Por tanto, *postBaseEndpoint* define la salida de error de los endpoints que lo utilicen, que devolverán un mensaje de error asociado a los códigos de estado *Not Found*, *Bad Request*, *Conflict* y *Unknown*.

```
private val postBaseEndpoint:
  (String, String, String) => PublicEndpoint[Unit, UserError, Unit, Any] =
  (name, description, path) =>
    baseEndpoint(name, description, path)
      .post
      .errorOut(
        oneOf[UserError](
          oneOfVariant(StatusCode.NotFound, ErrorOutputsTraits.notFoundOutput),
          oneOfVariant(StatusCode.BadRequest, ErrorOutputsTraits.badRequestOutput),
          oneOfVariant(StatusCode.Conflict, ErrorOutputsTraits.conflictOutput),
          oneOfDefaultVariant(ErrorOutputsTraits.unknownOutput)
        )
      )
```

Figura 70. Definición del endpoint base *postBaseEndpoint*.

En cuanto a los parámetros de *createReview*, la entrada del endpoint es un cuerpo de petición en formato JSON (Figura 71). Este JSON debe tener la misma estructura que la clase *Review*, es decir, los mismos campos con los mismos nombres y valores de un tipo que sea codificable a al tipo que tenga ese campo en la clase de Scala. Si *Review* tiene un atributo *id* de

tipo *Long*, en el JSON ese campo deberá ser un número; y lo mismo aplica para un *String*, por ejemplo.

```
val jsonReview: EndpointInput[Review] =  
  jsonBody[Review]
```

Figura 71. Definición del *EndpointInput* *jsonReview*.

El parámetro de salida es un objeto JSON que recoge la información del objeto *Review* creado en la misma petición (Figura 72).

```
val reviewOutput: EndpointOutput[Review] =  
  jsonBody[Review].description("The requested review")
```

Figura 72. Definición del *EndpointOutput* *reviewOutput*.

En lo que a la implementación de la lógica del endpoint se refiere, se sigue la misma línea que también tienen, por lo general, el resto de endpoints del sistema con método POST:

- Se comprueba si existe una reseña con el mismo identificador.
 - En caso afirmativo, se devuelve un error de tipo *Conflict*, indicando que ya existe un recurso de tipo *Review* con ese mismo identificador.
 - En caso contrario, se comprueba si el identificador de la reseña es válido.
 - Si no es válido, se devuelve un error *Bad Request* indicando el problema.
 - Si es válido, se actualiza el usuario que ha creado el objeto, y, si esa operación se realiza con éxito, se actualiza el sistema de persistencia con la nueva reseña.

```
val createReview: Review => IO[Either[UserError, Review]] =  
  newReview => IO.pure {  
    ReviewRepository.get(newReview.id) match  
    case Some(_) => Left(Conflict(s"Review with ID ${newReview.id.value} already exists"))  
    case None if newReview.id.value <= 0 => Left(BadRequest("Invalid review ID"))  
    case None =>  
      CommonFunctions.getUserAndApply(newReview.userId)(newReview, ReviewsAuxFunctions.addNewReviewToUser) match  
      case Left(error) => Left(error)  
      case Right(_) =>  
        ReviewRepository.put(newReview.id, newReview)  
        Right(newReview)  
  }.handleError {  
    case ex: Exception => Left(Unknown(500, s"Unexpected error: ${ex.getMessage}"))  
  }
```

Figura 73. Implementación de la lógica del endpoint *createReview*.

A lo largo de la ejecución del código mencionado puede producirse cualquier otro tipo de error inesperado en forma de excepción, cuyo mensaje será mostrado en un error de tipo *Unknown*.

4.2. Endpoint de tipo PUT

A modo de ejemplo de un endpoint cuyo método HTTP es PUT, se enseña el endpoint *editReview* (**Figura 74**), que permite al usuario editar una reseña ya existente en el sistema en caso de que dicho usuario sea realmente el creador de esa reseña.

```
val editReview: PublicEndpoint[(ReviewId, Review), UserError, Review, Any] =
  reviewBaseEndpoint(
    "editReview",
    "This endpoint edits a review",
    "PUT"
  )
  .in(PathInputs.pathReviewId)
  .in("edit")
  .in(JsonInputs.jsonReview)
  .out(SocialOutputs.reviewOutput)
```

Figura 74. Definición del endpoint *editReview*.

Este endpoint especifica que utiliza el método PUT, por lo que uno de sus endpoints base es *putBaseEndpoint* (**Figura 75**), que define como salida de error los mismos posibles errores que los de *postBaseEndpoint*, pero devolviendo, en caso de ejecución exitosa, un código de estatus *Ok*.

```
private val putBaseEndpoint:
  (String, String, String) => PublicEndpoint[Unit, UserError, Unit, Any] =
  (name, description, path) =>
    baseEndpoint(name, description, path)
      .put
      .errorOut(
        oneOf[UserError](
          oneOfVariant(StatusCode.BadRequest, ErrorOutputsTraits.badRequestOutput),
          oneOfVariant(StatusCode.NotFound, ErrorOutputsTraits.notFoundOutput),
          oneOfVariant(StatusCode.Conflict, ErrorOutputsTraits.conflictOutput),
          oneOfDefaultVariant(ErrorOutputsTraits.unknownOutput)
        )
      )
      .out(statusCode(StatusCode.Ok))
```

Figura 75. Definición del endpoint base *putBaseEndpoint*.

Por otra parte, *editReview* presenta otro parámetro de entrada más (**Figura 76**, en este caso es un parámetro de ruta).


```
val pathReviewId: EndpointInput[ReviewId] =
  path[ReviewId]("review_id")
```

Figura 76. Definición del *EndpointInput* *pathReviewId*.

Como el valor del parámetro se introduce en la dirección a la que se manda la solicitud, es necesario convertirlo a *Long*, pues es el tipo del valor que recoge la clase *ReviewId* (**Figura 77**).

```
case class ReviewId(value: Long)
```

Figura 77. Clase *ReviewId*.

Por ello, es necesario definir un códec personalizado (*custom codec*), tal y como recomienda la documentación oficial de Tapir (SoftwareMill, 2024 -g), que le indique a Scala cómo realizar esa conversión en ambos sentidos cuando nos encontremos con un *ReviewId*, tal y como se hace en la **Figura 78**.

```
implicit val reviewIdCodec: Codec[String, ReviewId, TextPlain] =
  Codec.string.mapDecode(s => DecodeResult.Value(ReviewId(s.toLong)))(_.value.toString)
```

Figura 78. Definición del códec *reviewIdCodec*.

La implementación de la lógica de este endpoint (y la de la mayoría de endpoints PUT) es muy parecida a la de los endpoints POST, con la diferencia de que, en este caso, el flujo de ejecución correcto se activa cuando sí que se encuentra en el sistema una reseña con el mismo identificador que el de la reseña que se quiere actualizar (además de que el valor introducido en el parámetro de ruta se corresponda con el del JSON de la reseña a actualizar).

```
val editReview: ((ReviewId, Review)) => IO[Either[UserError, Review]] =
  (reviewId, updatedReview) => IO.pure {
    if (reviewId.value != updatedReview.id.value)
      Left(BadRequest("Review ID in path and updated review ID did not match"))
    else
      CommonFunctions.getReview(reviewId) match
      case Left(error) => Left(error)
      case Right(existingReview) =>
        CommonFunctions
          .getUserAndApply(existingReview.userId)(existingReview, ReviewsAuxFunctions.updateUserFromReview) match
          case Left(error) => Left(error)
          case Right(value) =>
            ReviewRepository.put(reviewId, updatedReview)
            Right(updatedReview)
  }.handleError {
    case ex: Exception => Left(Unknown(500, s"Unexpected error: ${ex.getMessage}"))
  }
```

Figura 79. Implementación de la lógica del endpoint *editReview*.

4.3. Endpoint de tipo DELETE

En cuanto a los endpoints de tipo DELETE, se muestra como ejemplo *deleteReview* (Figura 80), que permite al usuario borrar una reseña que ya exista en el sistema, siempre y cuando dicho usuario sea el creador de la reseña mencionada.

```
val deleteReview: PublicEndpoint[ReviewId, UserError, Unit, Any] =  
  reviewBaseEndpoint(  
    "deleteReview",  
    "This endpoint deletes a review",  
    "DELETE"  
  )  
  .in(PathInputs.pathReviewId)  
  .in("delete")
```

Figura 80. Definición del endpoint *deleteReview*.

Al indicarse que se lleva a cabo una operación DELETE, hace uso del endpoint base *deleteBaseEndpoint* (Figura 81), que, al igual que los endpoints base definidos para los otros métodos HTTP, define el código de estatus que se devuelve en una ejecución exitosa, así como en una ejecución no exitosa acompañado de un mensaje de error.

En el caso de los endpoints que hacen uso de *deleteBaseEndpoint*, el código de estado devuelto en una operación realizada correctamente es un *No Content*. En operaciones que no se han podido completar satisfactoriamente, se devolverá alguno de los siguientes errores: *Not Found*, *Bad Request* o *Unknown*.

```
private val deleteBaseEndpoint:  
(String, String, String) => PublicEndpoint[Unit, UserError, Unit, Any] =  
  (name, description, path) =>  
    baseEndpoint(name, description, path)  
      .delete  
      .errorOut(  
        oneOf[UserError](  
          oneOfVariant(StatusCode.NotFound, ErrorOutputsTraits.notFoundOutput),  
          oneOfVariant(StatusCode.BadRequest, ErrorOutputsTraits.badRequestOutput),  
          oneOfDefaultVariant(ErrorOutputsTraits.unknownOutput)  
        )  
      )  
      .out(statusCode(StatusCode.NoContent))
```

Figura 81. Definición del endpoint base *deleteBaseEndpoint*.

La lógica definida para este tipo de endpoints, como la que se puede ver en la Figura 82, se encarga principalmente de comprobar que existe dentro del sistema el objeto al que hace referencia el identificador pasado como parámetro, en cuyo caso se actualiza el usuario referenciado en este y se actualiza el sistema de persistencia borrando el objeto. Si en algún

momento se produjese una excepción que interrumpiese el flujo de ejecución, se mostraría el mensaje de su causa con un error *Unknown*.

```
val deleteReview: ReviewId => IO[Either[UserError, Unit]] =
  reviewId => IO.pure {
    CommonFunctions.getReview(reviewId) match
    case Left(error) => Left(error)
    case Right(review) =>
      CommonFunctions.getUserAndApply(review.userId)(review, ReviewsAuxFunctions.removeReviewFromUser) match
      case Left(error) => Left(error)
      case Right(_) =>
        ReviewRepository.delete(review.id)
        Right(())
  }.handleError {
    case ex: Exception => Left(Unknown(500, s"Unexpected error: ${ex.getMessage}"))
  }
```

Figura 82. Implementación de la lógica del endpoint *deleteReview*.

5. Experimentos adicionales

En este apéndice se realizarán experimentos sobre otros endpoints que pueden resultar interesantes de analizar. En el capítulo **Experimentos y validación** se han revisado endpoints que llevan a cabo los distintos métodos HTTP existentes, salvo PUT, cuyos endpoints implementados son lo suficientemente parecidos a los del método POST, que podemos obviar su validación en esta memoria, con el propósito de hacerla un poco más dinámica.

Sin embargo, en cuanto a los endpoints que lanzan solicitudes a servicios externos, solo se ha visto uno, el de la sección **Endpoint *searchBook***. Si bien es suficiente para recoger en esta memoria la manera de probar este tipo de endpoints a nivel general, puede ser interesante también mostrar otros endpoints que interactúan con todas las APIs, y no solo con Google Books, dado que para cada uno de estos servicios se ha implementado un cliente distinto en la aplicación, siendo conveniente validar todos.

Aprovechando estas pruebas, también se realizarán experimentos sobre otros endpoints de interés, tanto si guardan relación con los contenidos multimedia extraídos de las APIs mencionadas, como si no.

5.1. Endpoints relacionados con contenidos de TMDB

Para empezar, se hará un recorrido por algunos de los endpoints que, internamente, realizan solicitudes a la API de TMDB. Eso incluye en este caso, los endpoints usados para

obtener información de la serie *The Bear*, de una de sus temporadas y de uno de sus capítulos, además de comprobar los registros y listas que la referencian o incluyen.

Por tanto, el primer endpoint que va a ser puesto a prueba es *searchTvShow* (**Figura 83**), que, mediante el parámetro de consulta *query* permite buscar el contenido especificado en la cadena de texto posterior (todos los endpoints de la herramienta de búsqueda funcionan de esta manera).

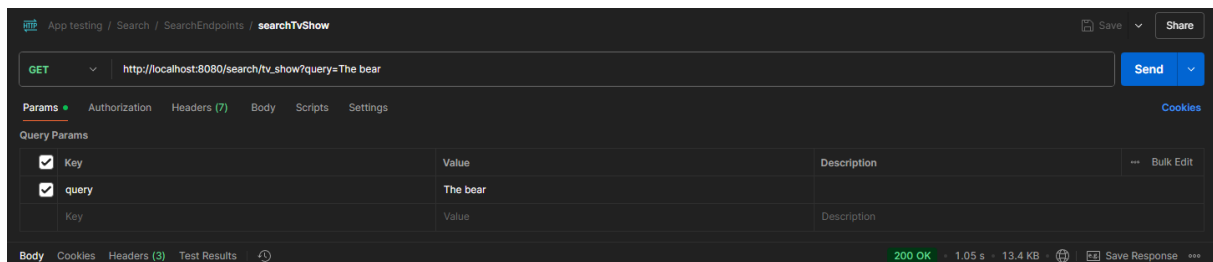


Figura 83. Llamada al endpoint *searchTvShow*.

De esta búsqueda, se obtiene un objeto JSON con todos los resultados obtenidos de dicha búsqueda (**Figura 84**). Estos resultados, al igual que ocurre en otros endpoints, podrían ser ordenados según algún criterio soportado por el tipo de contenido buscado mediante el parámetro de consulta *sorted_by*. En el caso de este endpoint, así como de todos los que realizan búsquedas, el objeto devuelto contiene una gran variedad de contenidos, por lo que no haría falta realizar varias búsquedas para cerciorarse de que la decodificación de la respuesta de TMDb se ha realizado correctamente. Además, es posible comprobar cómo los campos *original_name* o *name* contienen la palabra *bear* que forma parte del texto buscado, por lo que se puede intuir que la búsqueda por coincidencia de texto se realiza bien, aunque eso está gestionado por el servicio externo.

```
[
  {
    "adult": false,
    "backdrop_path": "/wHNw1E6ftEpgjVbdhLX0tv1hLs0.jpg",
    "first_air_date": "2022-06-23",
    "genre_ids": [
      18,
      35
    ],
    "id": 136315,
    "media_type": null,
```

```

    "original_name": "The Bear",
    "original_language": "en",
    "original_title": null,
    "overview": "Carry, a young fine-dining chef, comes home to Chicago to run
his family sandwich shop. As he fights to transform...",
    "popularity": 44.5549,
    "poster_path": "/eKfVzzEazSIjJMrw9ADa2x8ksLz.jpg",
    "release_date": null,
    "title": null,
    "name": "The Bear",
    "video": null,
    "vote_average": 8.198,
    "vote_count": 1341
  },
  {
    "adult": false,
    "backdrop_path": null,
    "first_air_date": "1969-09-06",
    "genre_ids": [
      16,
      10762
    ],
    "id": 129827,
    "media_type": null,
    "original_name": "The Smokey Bear Show",
    "original_language": "en",
    "original_title": null,
    "overview": "",
    "popularity": 19.267,
    "poster_path": null,
    "release_date": null,
    "title": null,
    "name": "The Smokey Bear Show",
    "video": null,
    "vote_average": 0.0,
    "vote_count": 0
  },
  {
    "adult": false,
    "backdrop_path": null,

```

```

    "first_air_date": "",
    "genre_ids": [
        16
    ],
    "id": 253920,
    "media_type": null,
    "original_name": "The little polar bear",
    "original_language": "de",
    "original_title": null,
    "overview": "Follows the adventures of Lars the little polar bear and his friends Lena the snow hare and Peeps the snow goose with the fun and...",
    "popularity": 5.3419,
    "poster_path": "/AIwA9fKVGZKp8INJ8LTM0tz7IY.jpg",
    "release_date": null,
    "title": null,
    "name": "The Little Polar Bear",
    "video": null,
    "vote_average": 0.0,
    "vote_count": 0
  },
  ...
]

```

Figura 84. Objeto JSON devuelto por la aplicación al llamar al endpoint *searchTvShow* para buscar la serie *The Bear*.

A continuación, se realizaría la validación del endpoint *getTvShow*, cuya petición se muestra en la **Figura 85**. Se puede observar que el identificador introducido en la dirección de la solicitud HTTP es igual que el campo *id* del objeto *tvShowFromTMDB* mostrado en la **Figura 87**. Con eso es suficiente para comprobar el correcto funcionamiento del endpoint, pues la decodificación del JSON ya se ha validado en el endpoint anterior, y, habiendo definido el mismo DTO para las series de ambos endpoints, el proceso de decodificación es el mismo.

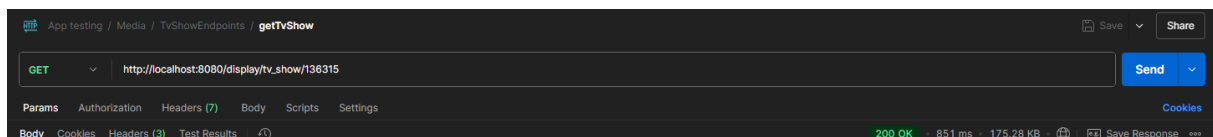


Figura 85. Ejecución del endpoint *getTvShow*.

```

case class TvShowFromTMDB(
    adult: Boolean,
    backdrop_path: Option[String],
    created_by: List[Creator],
    episode_run_time: Option[List[Int]],
    first_air_date: Option[String],
    genres: List[Genre],
    homepage: String,
    id: TvShowId,
    in_production: Boolean,
    languages: Option[List[String]],
    last_air_date: Option[String],
    last_episode_to_air: Option[TvEpisodeInTvShow],
    name: String,
    next_episode_to_air: Option[TvEpisodeInTvShow],
    networks: Option[List[Network]],
    number_of_episodes: Option[Int],
    number_of_seasons: Option[Int],
    origin_country: Option[List[String]],
    original_language: Option[String],
    original_name: String,
    overview: Option[String],
    popularity: Option[Double],
    poster_path: Option[String],
    production_companies: List[ProductionCompany],
    production_countries: List[ProductionCountry],
    seasons: List[TvSeasonInTvShow],
    spoken_languages: Option[List[SpokenLanguage]],
    status: String,
    tagline: String,
    `type`: String,
    vote_average: Option[Double],
    vote_count: Option[Long]
)

```

Figura 86. DTO utilizado para recoger información de una serie de TMDB.

```

{
  "tvShowFromTMDB": {
    "adult": false,
    "backdrop_path": "/wHNw1E6ftEpgjVbdhLX0tv1hLs0.jpg",
    "created_by": [
      {
        "id": 1057655,
        "credit_id": "62965a8b7e403d006682157d",
        "name": "Christopher Storer",
        "original_name": "Christopher Storer",
        "gender": 2,
        "profile_path": "/kG6I1G1aUtENJoJzrP7uxiJovy9.jpg"
      }
    ],
    "episode_run_time": [],
    "first_air_date": "2022-06-23",
    "genres": [
      {
        "id": 18,
        "name": "Drama"
      }
    ]
  }
}

```

```

    },
    {
      "id": 35,
      "name": "Comedy"
    }
  ],
  "homepage": "https://www.fxnetworks.com/shows/the-bear",
  "id": {
    "value": 136315
  },
  "in_production": true,
  "languages": [
    "en"
  ],
  "last_air_date": "2024-06-26",
  "last_episode_to_air": {
    "id": 5334931,
    "name": "Forever",
    "overview": "Another funeral.",
    "vote_average": 7.5,
    "vote_count": 14,
    "air_date": "2024-06-26",
    "episode_number": {
      "value": 10
    },
    "episode_type": "finale",
    "production_code": "XCBV3010",
    "runtime": 44,
    "season_number": {
      "value": 3
    },
    "show_id": {
      "value": 136315
    },
    "still_path": "/35uxXBcWcPknocsuLuggL6txyRe.jpg"
  },
  "name": "The Bear",
  "next_episode_to_air": {
    "id": 6231138,

```



```

    "name": "Episode 1",
    "overview": "",
    "vote_average": 0.0,
    "vote_count": 0,
    "air_date": "2025-06-25",
    "episode_number": {
      "value": 1
    },
    "episode_type": "standard",
    "production_code": "",
    "runtime": null,
    "season_number": {
      "value": 4
    },
    "show_id": {
      "value": 136315
    },
    "still_path": null
  },
  "networks": [
    {
      "id": 453,
      "logo_path": "/pqUTCleNUiTLAVlelGxUgWn1ELh.png",
      "name": "Hulu",
      "origin_country": "US"
    }
  ],
  "number_of_episodes": 38,
  "number_of_seasons": 4,
  "origin_country": [
    "US"
  ],
  "original_language": "en",
  "original_name": "The Bear",
  "overview": "Carmy, a young fine-dining chef, comes home to Chicago to run his family sandwich shop. As he fights to transform the shop and himself,...",
  "popularity": 44.5549,
  "poster_path": "/eKfVzzEazSIjJMrw9ADa2x8ksLz.jpg",
  "production_companies": [
    {

```

```

        "id": 15990,
        "logo_path": "/5cT4zwHA66uNAr2p3CcBDLddXu2.png",
        "name": "FX Productions",
        "origin_country": "US"
    }
],
"production_countries": [
    {
        "iso_3166_1": "US",
        "name": "United States of America"
    }
],
"seasons": [
    {
        "air_date": "2022-06-23",
        "episode_count": 8,
        "id": 215540,
        "name": "Season 1",
        "overview": "",
        "poster_path": "/l2NFRq7w6azQ2gyG0tDgJdkVACV.jpg",
        "season_number": {
            "value": 1
        },
        "vote_average": 8.1
    },
    ...
],
"spoken_languages": [
    {
        "english_name": "English",
        "iso_639_1": "en",
        "name": "English"
    }
],
"status": "Returning Series",
"tagline": "Back for seconds.",
"type": "Scripted",
"vote_average": 8.198,
"vote_count": 1341

```

```

},
"similarTvShows": [
  {
    "adult": false,
    "backdrop_path": "/oGTia7UGeU1A7QX7tsc0935SBwC.jpg",
    "first_air_date": "1997-09-11",
    "genre_ids": [
      35
    ],
    "id": 2575,
    "media_type": null,
    "original_name": "Between Brothers",
    "original_language": "en",
    "original_title": null,
    "overview": "Between Brothers is an American television situation comedy centered around four middle-class African American men in their late twenties, living in Chicago.",
    "popularity": 7.7487,
    ...
  },
  ...
],
"recommendedTvShows": [
  {
    "adult": false,
    "backdrop_path": "/bqopwxfw2Sd30gPDfUUFf1CpjtK.jpg",
    "first_air_date": "2022-09-26",
    "genre_ids": [
      18,
      80,
      9648
    ],
    "id": 121710,
    "media_type": "tv",
    "original_name": "Inside Man",
    "original_language": "en",
    "original_title": null,
    "overview": "A prisoner on death row in the US, a vicar in a quiet English town, and a maths teacher trapped in a cellar cross paths in the most unexpected way.",
    "popularity": 9.9818,

```

```

        ...
    },
    ...
],
"cast": [
    {
        "adult": false,
        "gender": 2,
        "id": 206905,
        "known_for_department": "Acting",
        "name": "Jeremy Allen White",
        "original_name": "Jeremy Allen White",
        "popularity": 4.5199,
        ...
    },
    ...
],
"crew": [
    {
        "adult": false,
        "gender": 2,
        "id": 2395784,
        "known_for_department": "Art",
        "name": "Steve Hull",
        "original_name": "Steve Hull",
        "popularity": 0.1879,
        ...
    },
    ...
],
"averageRating": 10.0,
"entriesIds": [
    {
        "value": 1
    },
    {
        "value": 19
    }
],

```

```

    "mediaListsIds": null,
    "completedCount": 1,
    "droppedCount": 0,
    "inProgressCount": 0,
    "onHoldCount": 0,
    "pendingCount": 0,
    "totalRatings": 1
}

```

Figura 87. Objeto JSON devuelto por la aplicación al llamar al endpoint *getTvShow* para obtener información de la serie *The Bear*.

Una vez que el usuario ha accedido al contenido multimedia deseado, puede llevar a cabo diversas acciones, como obtener todos los registros que lo referencian, tal y como se contempla en la **Figura 88**. En el caso de que el endpoint se ejecute correctamente, la aplicación responde a la solicitud HTTP con un objeto JSON que contiene una lista de los registros que han sido creados basándose en la serie en cuestión (**Figura 89**). En este caso, se incluyen los dos únicos objetos *Entry* existentes en la aplicación que apuntan al *TvShowId* con el valor que hemos usado para obtener la información de la serie *The Bear*.

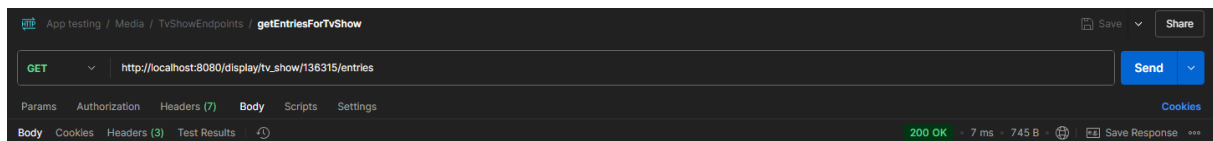


Figura 88. Petición al endpoint *getEntriesForTvShow*.

```

[
  {
    "id": {
      "value": 1
    },
    "userId": {
      "value": 5
    },
    "mediaId": {
      "type": "TvShowId",
      "value": 136315
    },
    "ratingId": {
      "value": 1
    }
  }
]

```

```

    },
    "reviewId": {
      "value": 1
    },
    "completed": true,
    "inProgress": false,
    "onHold": false,
    "dropped": false,
    "repeat": false,
    "startedDate": "2024-11-10",
    "finishedDate": "2024-12-02",
    "platform": null,
    "creationDate": "2024-12-02T20:16:41"
  },
  {
    "id": {
      "value": 19
    },
    "userId": {
      "value": 3
    },
    "mediaId": {
      "type": "TvShowId",
      "value": 136315
    },
    "ratingId": {
      "value": 5
    },
    "reviewId": null,
    "completed": true,
    "inProgress": false,
    "onHold": false,
    "dropped": false,
    "repeat": true,
    "startedDate": "2025-06-01",
    "finishedDate": "2025-06-15",
    "platform": null,
    "creationDate": "2025-06-15T17:34:01"
  }
}

```

]

Figura 89. Objeto JSON obtenido como resultado de la llamada al endpoint *getEntriesForTvShow* para la serie *The Bear*.

Si no existiese ningún objeto que apunte a dicho contenido, no se obtendría ningún error, sino que el resultado se sigue considerando válido devolviendo una lista vacía. Esto puede comprobarse llamando a un endpoint análogo al anterior, pero que, en vez de objetos *Entry*, devuelve contenidos multimedia.

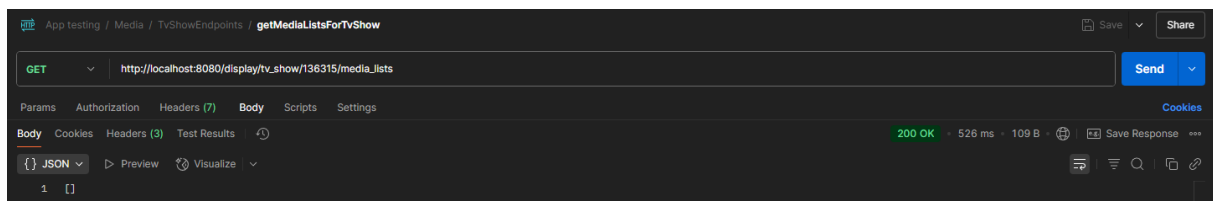


Figura 90. Resultado de ejecutar el endpoint *getMediaListsForTvShow* sin que exista ninguna lista que albergue ese contenido.

De la misma manera que un usuario accede a la información de una serie, también puede obtener los datos de una de sus temporadas (**Figura 91** y **Figura 92**) y de uno de sus episodios (**Figura 93** y **Figura 94**). La manera en que se comprueba el correcto funcionamiento de estos endpoints es el mismo que con las series (y las películas), y las operaciones que luego permiten estos contenidos son las mismas.

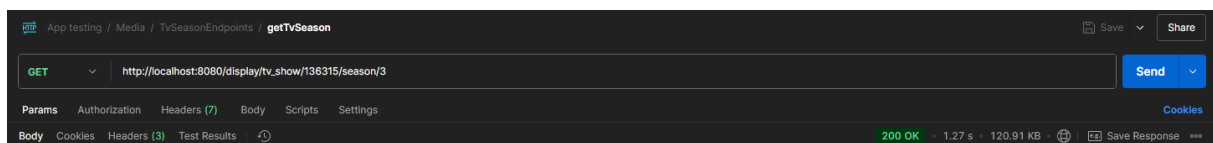


Figura 91. Ejecución del endpoint *getTvSeason*.

```
{
  "tvSeasonFromTMDB": {
    "_id": "663cfd855ad4ec9cf92fee4e",
    "air_date": "2024-06-26",
    "episodes": [
      {
        "air_date": "2024-06-26",
        "episode_number": {
          "value": 1
        },
        "episode_type": "standard",
```

```

    "id": 5334913,
    "name": "Tomorrow",
    "overview": "The next day and the days that led to it.",
    "production_code": "XCBV3001",
    "runtime": 37,
    "season_number": {
        "value": 3
    },
    "show_id": 136315,
    "still_path": "/qJ42DnApTQMFUQodRHWXWzomjvz.jpg",
    "vote_average": 8.121,
    "vote_count": 33,
    "crew": [
        {
            "adult": false,
            "gender": 2,
            "id": 1057655,
            "known_for_department": "Directing",
            "name": "Christopher Storer",
            "original_name": "Christopher Storer",
            "popularity": 1.0947,
            ...
        },
        ...
    ],
    "guest_stars": [
        {
            "adult": false,
            "gender": 2,
            "id": 17485,
            "known_for_department": "Acting",
            "name": "Oliver Platt",
            "original_name": "Oliver Platt",
            "popularity": 2.779,
            ...
        },
        ...
    ]
},

```



```

        ...
    ],
    "name": "Season 3",
    "overview": "In season three, Carmen 'Carry' Berzatto, Sydney Adamu, and Richard 'Richie' Jerimovich do what it takes to elevate The Bear, their beef stand turned fine dining establishment,...",
    "id": 392941,
    "poster_path": "/kmK0XOC6txne2tXjcnZPHNtsY9E.jpg",
    "runtime": null,
    "season_number": {
        "value": 3
    },
    "vote_average": 7.6
},
"cast": [
    {
        "adult": false,
        "gender": 2,
        "id": 206905,
        "known_for_department": "Acting",
        "name": "Jeremy Allen White",
        "original_name": "Jeremy Allen White",
        "popularity": 4.817,
        ...
    },
    ...
],
"crew": [
    {
        "adult": false,
        "gender": 1,
        "id": 4797619,
        "known_for_department": "Art",
        "name": "Angela Ngo",
        "original_name": "Angela Ngo",
        "popularity": 0.0553,
        ...
    },
    ...
],

```

```

    "averageRating": null,
    "entriesIds": null,
    "mediaListsIds": null,
    "completedCount": 0,
    "droppedCount": 0,
    "inProgressCount": 0,
    "onHoldCount": 0,
    "pendingCount": 0,
    "totalRatings": 0
}

```

Figura 92. Objeto JSON devuelto por la aplicación al llamar al endpoint *getTvSeason* para obtener información de una temporada de la serie *The Bear*.

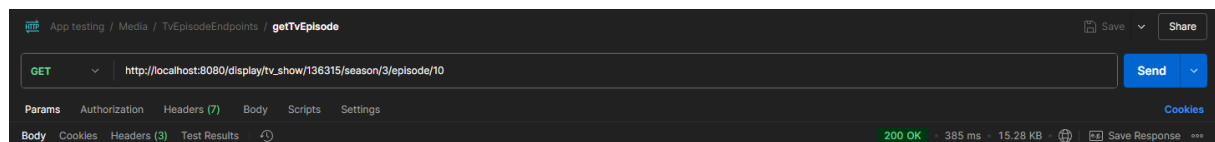


Figura 93. Ejecución del endpoint *getTvEpisode*.

```

{
  "tvEpisodeFromTMDB": {
    "air_date": "2024-06-26",
    "crew": [
      {
        "adult": false,
        "gender": 2,
        "id": 1057655,
        "known_for_department": "Directing",
        "name": "Christopher Storer",
        "original_name": "Christopher Storer",
        "popularity": 1.0947,
        ...
      },
      ...
    ],
    "episode_number": {
      "value": 10
    },
    "guest_stars": [
      {

```

```

        "adult": false,
        "gender": 2,
        "id": 74949,
        "known_for_department": "Acting",
        "name": "Joel McHale",
        "original_name": "Joel McHale",
        "popularity": 2.3903,
        ...
    },
    ...
],
"name": "Forever",
"overview": "Another funeral.",
"id": 5334931,
"production_code": "XCBV3010",
"runtime": 44,
"season_number": {
    "value": 3
},
"still_path": "/35uxXBcWcPknocsuLuggL6txyRe.jpg",
"vote_average": 7.5,
"vote_count": 14
},
"cast": [
    {
        "adult": false,
        "gender": 2,
        "id": 206905,
        "known_for_department": "Acting",
        "name": "Jeremy Allen White",
        "original_name": "Jeremy Allen White",
        "popularity": 4.817,
        ...
    },
    ...
],
"crew": [
    {
        "adult": false,

```

```

        "gender": 2,
        "id": 1057655,
        "known_for_department": "Directing",
        "name": "Christopher Storer",
        "original_name": "Christopher Storer",
        "popularity": 1.0947,
        ...
    },
    ...
],
"averageRating": null,
"entriesIds": null,
"mediaListsIds": null,
"completedCount": 0,
"droppedCount": 0,
"pendingCount": 0,
"totalRatings": 0
}

```

Figura 94. Objeto JSON devuelto por la aplicación al llamar al endpoint *getTvEpisode* para obtener información de una temporada de la serie *The Bear*.

5.2. Endpoints relacionados con contenidos de IGDB

Del mismo modo que se ha comprobado el correcto funcionamiento de los endpoints relacionados con los contenidos multimedia de TMDb (y algunos relacionados con todos los contenidos en general), se comprueba el funcionamiento de los endpoints que operan con contenidos multimedia provenientes de la API de IGDB. Para ello, el primer endpoint que se prueba es *searchVideogame*, que es igual que los endpoints de búsqueda que se han visto en la sección de **Endpoints relacionados con contenidos de TMDb**. Su correcto desempeño se comprueba de la misma manera, es decir, ejecutándolo y obteniendo un resultado que no sea un error, pues quiere decir que la respuesta de la API ha llegado exitosamente a la aplicación.

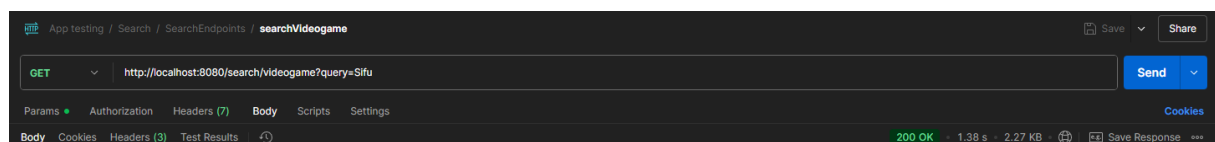


Figura 95. Ejecución del endpoint *searchVideogame*.

```
[
  {
    "id": 144022,
    "age_ratings": [
      56619,
      56621,
      ...
    ],
    "aggregated_rating": 78.85714285714286,
    "aggregated_rating_count": 13,
    "alternative_names": [
      79764
    ],
    "artworks": [
      50394
    ],
    "bundles": null,
    "category": 0,
    "collection": null,
    "cover": 208864,
    "created_at": 1614291022,
    "dlcs": null,
    "external_games": [
      2658785,
      1998663,
      ...
    ],
    "first_release_date": 1644278400,
    "follows": null,
    "franchises": null,
    "game_engines": [
      439
    ],
    "game_modes": [
      1
    ],
    "genres": [
      4,
      25,
```

```

    32
  ],
  "hypes": 17,
  "involved_companies": [
    120003,
    205058
  ],
  "keywords": [
    1985
  ],
  "name": "Sifu",
  "parent_game": null,
  "platforms": [
    169,
    48,
    ...
  ],
  "player_perspectives": [
    2
  ],
  "rating": 83.05934684868875,
  "rating_count": 141,
  "release_dates": [
    325753,
    389053,
    ...
  ],
  "screenshots": [
    424043,
    424044,
    ...
  ],
  "similar_games": [
    113895,
    111130,
    ...
  ],
  "slug": "sifu",
  "standalone_expansions": null,

```

```

    "storyline": "Sifu is the story of a young Kung Fu student on a path of
revenge, hunting for the murderers of his family...",
    "summary": "A third person action game featuring intense hand-to-hand
combat, it puts you in control of a young Kung-Fu student on his path of revenge
throughout the city.",
    "tags": [
        1,
        268435460,
        268435481,
        ...
    ],
    "themes": [
        1
    ],
    "total_rating": 80.95824485291581,
    "total_rating_count": 154,
    "updated_at": 1749953033,
    "url": "https://www.igdb.com/games/sifu",
    "videos": [
        46028,
        47133,
        ...
    ],
    "websites": [
        756581,
        756582,
        ...
    ],
    "checksum": "168bc905-197c-272f-3a1b-f85b6e7b652b",
    "remakes": null,
    "language_supports": [
        149147,
        149154,
        ...
    ],
    "game_localizations": [
        14101,
        26149
    ],
    "collections": null

```

```
}  
]
```

Figura 96. Objeto JSON devuelto por la aplicación al llamar al endpoint *searchVideogame* para buscar el videojuego *Sifu*.

Lo mismo aplica para el endpoint *getVideogame* de la figura **Figura 97** respecto a sus análogos de la sección anterior.

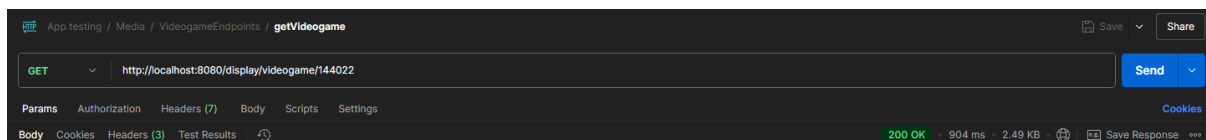


Figura 97. Ejecución del endpoint *getVideogame*.

Si obtenemos un objeto JSON válido con información del videojuego especificado, y el identificador de este coincide con el que se introduce en la ruta del endpoint, se considera válida su ejecución.

```
{  
  "videogameFromIGDB": {  
    "id": {  
      "value": 144022  
    },  
    "age_ratings": [  
      56619,  
      56621,  
      ...  
    ],  
    "aggregated_rating": 78.85714285714286,  
    "aggregated_rating_count": 13,  
    "alternative_names": [  
      79764  
    ],  
    "artworks": [  
      50394  
    ],  
    "bundles": null,  
    "category": 0,  
    "collection": null,  
    "cover": 208864,  
  }  
}
```



```
"created_at": 1614291022,
"dlds": null,
"external_games": [
    2658785,
    1998663,
    ...
],
"first_release_date": 1644278400,
"follows": null,
"franchises": null,
"game_engines": [
    439
],
"game_modes": [
    1
],
"genres": [
    4,
    25,
    32
],
"hypes": 17,
"involved_companies": [
    120003,
    205058
],
"keywords": [
    1985
],
"name": "Sifu",
"parent_game": null,
"platforms": [
    169,
    48,
    ...
],
"player_perspectives": [
    2
],
```

```

"rating": 83.05934684868875,
"rating_count": 141,
"release_dates": [
    325753,
    389053,
    ...
],
"screenshots": [
    424043,
    424044,
    ...
],
"similar_games": [
    113895,
    111130,
    ...
],
"slug": "sifu",
"standalone_expansions": null,
"storyline": "Sifu is the story of a young Kung Fu student on a path of
revenge, hunting for the murderers of his family...",
"summary": "A third person action game featuring intense hand-to-hand
combat, it puts you in control of a young Kung-Fu student on his path of revenge
throughout the city.",
"tags": [
    1,
    268435460,
    268435481,
    ...
],
"themes": [
    1
],
"total_rating": 80.95824485291581,
"total_rating_count": 154,
"updated_at": 1749953033,
"url": "https://www.igdb.com/games/sifu",
"videos": [
    46028,
    47133,

```

```

        ...
    ],
    "websites": [
        756581,
        756582,
        ...
    ],
    "checksum": "168bc905-197c-272f-3a1b-f85b6e7b652b",
    "remakes": null,
    "language_supports": [
        149147,
        149154,
        ...
    ],
    "game_localizations": [
        14101,
        26149
    ],
    "collections": null
},
"averageRating": 8.0,
"entriesIds": [
    {
        "value": 2
    }
],
"mediaListsIds": [
    {
        "value": 1
    },
    {
        "value": 5
    }
],
"completedCount": 1,
"droppedCount": 0,
"inProgressCount": 0,
"onHoldCount": 0,
"pendingCount": 0,

```

```
"totalRatings": 1
}
```

Figura 98. Objeto JSON devuelto por la aplicación al llamar al endpoint *getVideogame* para obtener información del videojuego *Sifu*.

Según los casos de uso definidos en la sección **Análisis funcional**, desde la página en que se visualiza la información de un contenido multimedia, también se debe poder agregar ese contenido a alguno de los objetos *MediaList* que haya creado el usuario previamente. Por tanto, esta acción implica una llamada al endpoint *editMediaList*, que permite editar una lista de contenidos multimedia, en este caso para añadirle el videojuego *Sifu*. Para completar esta operación, se debe introducir el JSON que represente una lista creada por el usuario en la aplicación, añadiéndole el identificador del videojuego, tal y como se puede ver en la **Figura 99**.

```
{
  "id": {
    "value": 4
  },
  "userId": {
    "value": 2
  },
  "title": "Movies, games, etc. to get to know me",
  "description": "Hope you enjoy them as much as I've done",
  "mediaIds": [
    {
      "type": "VideogameId",
      "value": 144022
    },
    {
      "type": "MovieId",
      "value": 10402
    },
    ...
  ],
  "visibility": "Followers",
  "allowReplies": true,
  "ranked": false,
```

```

    "creationDate": "2024-11-03T19:12:18",
    "updateDate": "2024-11-03T19:12:18",
    "likesIds": [
      {
        "value": 2
      }
    ],
    "repliesIds": []
  }

```

Figura 99. Cuerpo de petición (JSON) para el endpoint *editMediaList*, remarcando aquello que realmente se está editando en la lista.

Para que esto funcione, debemos asegurarnos de que el identificador del cuerpo de petición y el del parámetro de ruta coincidan como en la **Figura 100**. De lo contrario, se obtiene un *Bad Request* indicando que estos identificadores son distintos y que, por tanto, no se puede llevar a cabo la operación deseada (**Figura 101**).

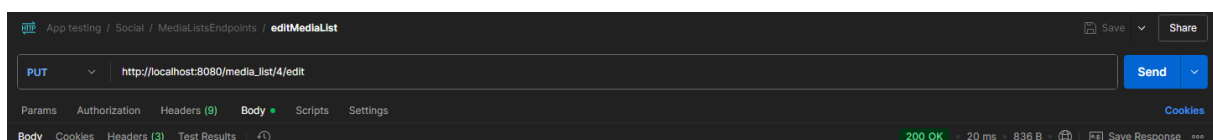


Figura 100. Llamada al endpoint *editMediaList*.

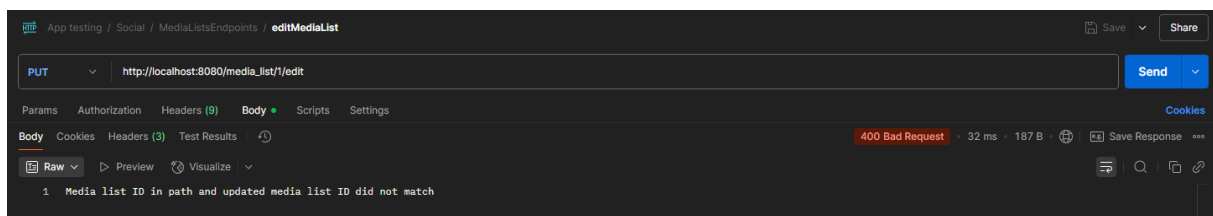


Figura 101. Resultado de ejecutar el endpoint *editMediaList* introduciendo un identificador en el cuerpo de la petición diferente al especificado en la ruta del endpoint.

5.3. Endpoints relacionados con contenidos de Google Books

Con el endpoint *getBook* se aplica el mismo criterio que con *getTvShow* o *getVideogame*, entre otros. Si la respuesta contiene información del libro cuyo identificador ha sido introducido en la URL de la petición, se considera que satisfactoria su ejecución.

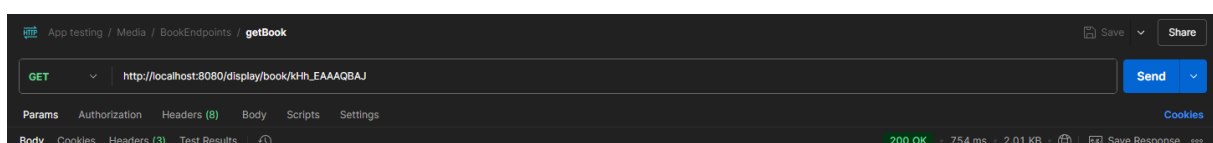


Figura 102. Solicitud al endpoint *getBook*.

```

{
  "bookFromGoogleBooks": {
    "id": {
      "value": "kHh_EAAQBAJ"
    },
    "volumeInfo": {
      "title": "The Old Man and the Sea",
      "subtitle": null,
      "authors": [
        "Ernest Hemingway"
      ],
      "publisher": "DigiCat",
      "publishedDate": "2022-08-01",
      "description": "In \"The Old Man and the Sea,\" Ernest Hemingway weaves a poignant tale of resilience and the human spirit, framed within..."
    }
  },
  "averageRating": null,
  "entriesIds": null,
  "mediaListsIds": [
    {
      "value": 4
    },
    {
      "value": 5
    }
  ],
  "completedCount": 0,
  "droppedCount": 0,
  "inProgressCount": 0,
  "onHoldCount": 0,
  "pendingCount": 0,
  "totalRatings": 0
}

```

Figura 103. Objeto JSON devuelto por la aplicación al llamar al endpoint *getBook* para obtener información del libro *The Old Man and The Sea*.

Otra acción que, según los casos de uso definidos, un usuario puede llevar a cabo directamente desde la página en que visualiza la información, es la de especificar el progreso

de dicho contenido, o el deseo de comenzar con su progreso en algún momento. Esto se hace mediante el endpoint *addBook* en este caso concreto, aunque se puede aplicar también a películas, videojuegos, etc.

El endpoint en cuestión realiza una solicitud HTTP de tipo PUT, dado que modifica la información interna del usuario que la realiza, añadiendo el identificador del contenido a uno de los atributos que marcan el estado progreso especificado.

En la **Figura 104** se muestra la ejecución del endpoint, indicando en la URL que se desea marcar el libro como completado. Por tanto, en el atributo *completedMediaIds* pasaríamos a encontrarnos la información devuelta por el endpoint, tal y como se muestra en el objeto JSON de la **Figura 105**.

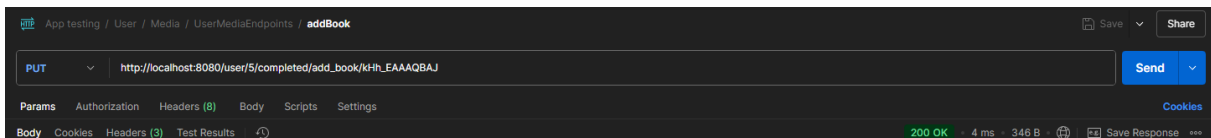


Figura 104. Solicitud al endpoint *addBook* para el estado de progreso *Completado*.

```
[
  {
    "type": "BookId",
    "value": "kHh_EAAAQBAJ"
  },
  {
    "type": "TvShowId",
    "value": 136315
  },
  {
    "type": "TvShowId",
    "value": 100565
  },
  {
    "type": "EpisodeNumber",
    "value": {
      "tvShowId": 136315,
      "seasonNumber": 2,
      "episodeNumber": 7
    }
  }
]
```

```

    },
    {
      "type": "VideogameId",
      "value": 19564
    }
  ]

```

Figura 105. Objeto JSON devuelto por el endpoint *addBook*, remarcando aquello que realmente se ha añadido a la categoría *Completado*.

Si se realizase la misma petición, pero introduciendo un identificador de un contenido que ya ha sido marcado con el progreso especificado, el sistema devolvería un error de tipo *Conflict* indicando que no se puede realizar la operación, tal y como se muestra en la **Figura 106**.

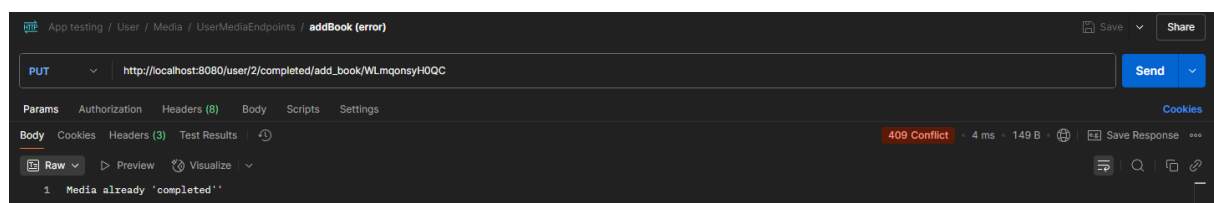


Figura 106. Petición al endpoint *addBook* para un libro que ya ha sido marcado previamente con ese estado de progreso.

5.4. Otros endpoints de interés

En último lugar, puede considerarse interesante mencionar una diferencia importante entre dos grupos de endpoints que parten, en un hipotético escenario, de la misma situación, que se produce al acceder a la información del propio usuario que ejecuta las solicitudes HTTP definidas en los diferentes endpoints. Para acceder a esa información, se debe realizar una llamada al endpoint *getUser*, cuya ejecución y resultado, en caso de éxito, se verían de la manera que se indica en la **Figura 107** y en la **Figura 108**.

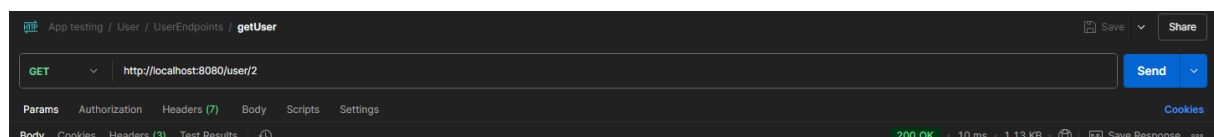


Figura 107. Petición al endpoint *getUser*.

```

{
  "id": {
    "value": 2
  },

```



```

"profile": {
  "username": "User2",
  "password": "kljhfsdg87ygfjsbhn",
  "email": "user2@gmail.com",
  "biography": "Hey there! I'm user 2",
  "location": "London, UK"
},
"favourites": {
  "movieId": {
    "value": 502033
  },
  "tvShowId": {
    "value": 135918
  },
  "videogameId": {
    "value": 113112
  },
  "bookId": {
    "value": "kHh_EAAAQBAJ"
  }
},
"completedMediaIds": [
  {
    "type": "MovieId",
    "value": 502033
  },
  {
    "type": "BookId",
    "value": "UU-VAAAACAAJ"
  },
  ...
],
"pendingMediaIds": [],
"inProgressMediaIds": [],
"onHoldMediaIds": [],
"droppedMediaIds": [
  {
    "type": "VideogameId",
    "value": 1082
  }
]

```

```
    }
  ],
  "mediaListsIds": [
    {
      "value": 4
    }
  ],
  "entriesIds": [
    {
      "value": 5
    },
    {
      "value": 6
    },
    ...
  ],
  "reviewsIds": [
    {
      "value": 5
    }
  ],
  "ratingsIds": [
    {
      "value": 5
    },
    {
      "value": 6
    },
    ...
  ],
  "likesIds": [
    {
      "value": 1
    }
  ],
  "repliesIds": [
    {
      "value": 2
    }
  ]
}
```

```

    ],
    "followingIds": [
      {
        "value": 4
      }
    ],
    "followersIds": [
      {
        "value": 4
      },
      {
        "value": 5
      }
    ],
    "blockedIds": [],
    "chatsIds": [
      {
        "value": 1
      }
    ],
    "archivedChatsIds": [
      {
        "value": 6
      }
    ]
  ]
}

```

Figura 108. Objeto JSON devuelto por la aplicación al llamar al endpoint *getUser*.

Tal y como se puede observar en el objeto JSON devuelto por la aplicación, el usuario contiene, principalmente, listas de identificadores que hacen referencia a elementos del dominio de la aplicación, como *Review* o *Like*, o a contenidos multimedia como *Movie* o *Book*. Estos últimos, como ya se ha mencionado anteriormente, necesitan de servicios externos para disponer de su información completa y actualizada, con campos que resulten relevantes para el usuario aparte de las métricas que la propia aplicación pueda generar.

La influencia de esto se observa en los grupos de endpoints que devuelven todos los objetos, de un determinado tipo, creados o recopilados por el usuario. Es decir, por una parte, tendríamos, por ejemplo, *getUserReplies*, *getUserChats*, o *getUserLikes*, y por otro lado

tendríamos las variables de *getAllMedia*, dando igual la categoría a la que hagan referencia entre *Completed*, *Pending*, etc.

En lo que se distinguen estos endpoints, entonces, es en la información que devuelven, en el sentido de que el primer grupo de endpoints mencionados devuelve una lista de objetos completos, mientras que *getAllMedia* responde a las peticiones con una lista que únicamente contiene los identificadores de los objetos completos a los que hacen referencia.

Por ejemplo, si llevamos a cabo una solicitud HTTP al endpoint *getUserReplies* (**Figura 109**), se obtiene una lista que contiene objetos *Reply* completos, tal y como se muestra en la **Figura 110**, que contiene toda la información existente de ese elemento del dominio, como el texto creado o los identificadores de las respuestas que ha tenido, y no solo su identificador propio.

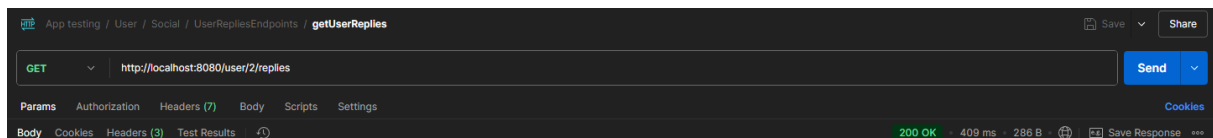


Figura 109. Petición al endpoint *getUserReplies*.

```
[
  {
    "id": {
      "value": 2
    },
    "userId": {
      "value": 2
    },
    "repliedObjectId": {
      "type": "ReviewId",
      "value": 5
    },
    "reply": "Well, at least it made fishing look less boring",
    "likesIds": [],
    "repliesIds": []
  }
]
```

Figura 110. Objeto JSON devuelto por la aplicación al llamar al endpoint *getUserReplies*.

Sin embargo, *getAllMedia* (**Figura 111**) responde a las peticiones con una lista que contiene exclusivamente los identificadores de los distintos contenidos multimedia referenciados. Es decir, esta colección, en vez de contener objetos de tipo *TvShow* o *Book*, entre otros, contiene objetos *TvShowId* o *BookId*, respectivamente (prueba de ello es el objeto JSON mostrado en la **Figura 112**). Con estos identificadores, el usuario podrá, después, acceder a la información completa de estos contenidos mediante endpoints como *getTvShow* o *getBook*.

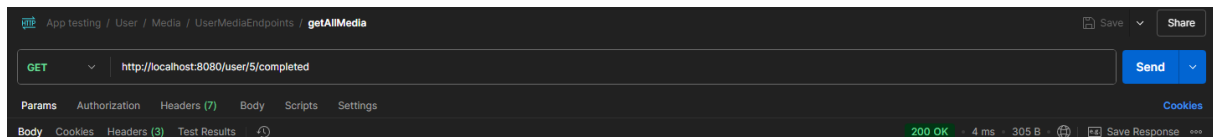


Figura 111. Petición al endpoint *getAllMedia*, especificando que se quiere obtener los contenidos marcados como completados.

```
[
  {
    "type": "TvShowId",
    "value": 136315
  },
  {
    "type": "TvShowId",
    "value": 100565
  },
  {
    "type": "EpisodeNumber",
    "value": {
      "tvShowId": 136315,
      "seasonNumber": 2,
      "episodeNumber": 7
    }
  },
  {
    "type": "VideogameId",
    "value": 19564
  }
]
```

Figura 112. Objeto JSON devuelto por la aplicación al llamar al endpoint *getAllMedia*.